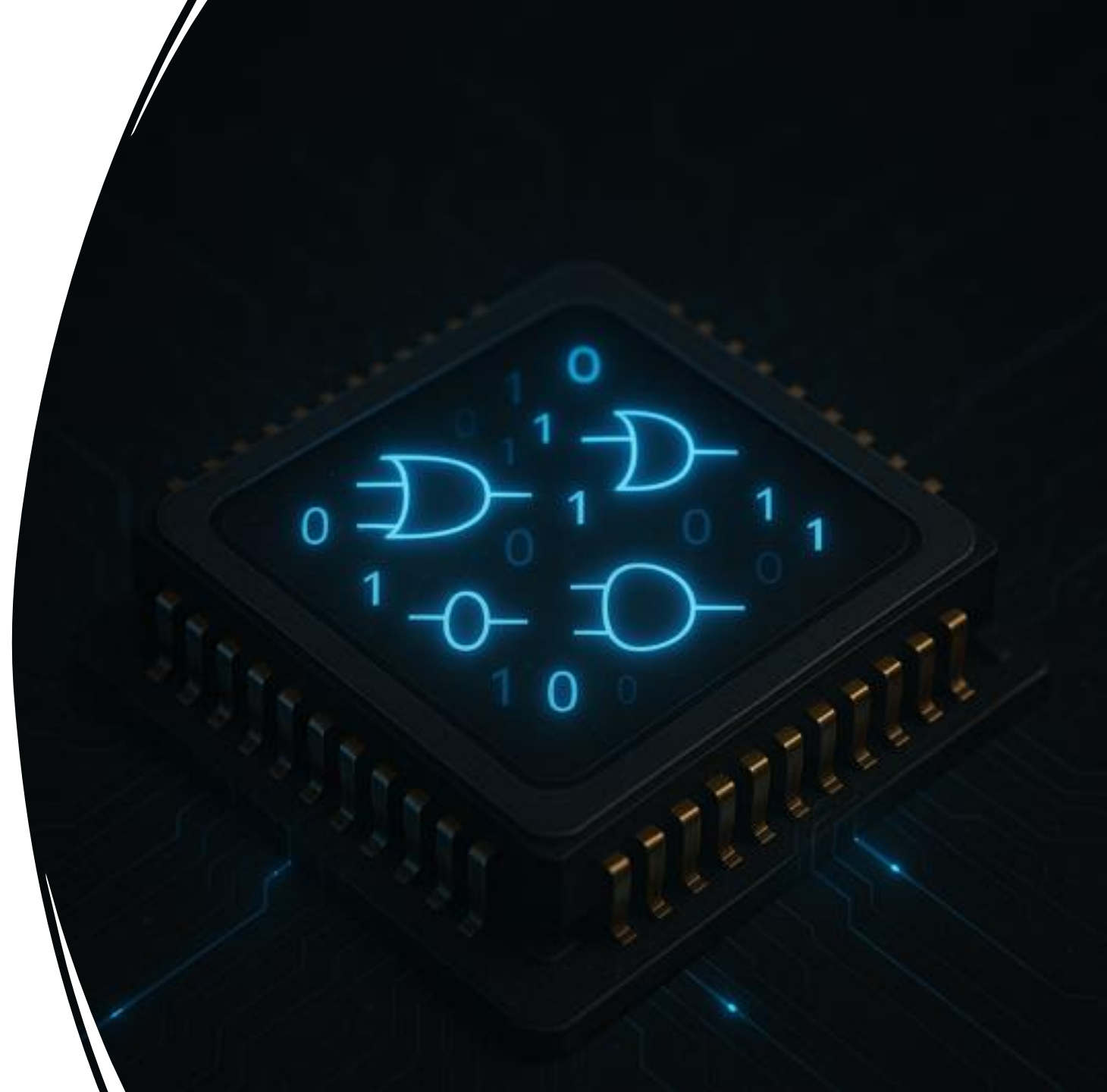


Eletrônica Digital

Do básico até o HDL



Introdução

Nesta apresentação vamos explorar a **base dos computadores**: A **eletrônica digital** e a **álgebra booleana**.

Partiremos do **básico e um pouco da história** até chegarmos ao **projeto de circuitos digitais** e ao uso de **VHDL para projetos de circuitos integrados**.

Aprender eletrônica digital ajuda a **desmistificar a programação de baixo nível** (como Assembly e C) e mostra como tudo se conecta:

👉 **programação + lógica + matemática + eletrônica.**

👉 Basicamente você vira um engenheiro ou programador melhor.



Agenda

Nesta apresentação iremos aprender os seguintes tópicos.

1. Raciocínio Logico, e Historia
2. Válvulas, Relés e Transistores
3. Logica/Números Booleans e Portas Universais
4. Circuitos Combinacionais
5. Mapas de Karnaugh
6. Osciladores 555 e NOT
7. Circuitos Sequenciais
8. Contadores
9. Maquinas de Estado (Moore e Mealy)
10. Metaestabilidade
11. VHDL e Verilog
12. Processadores
13. Criando seu próprio Chip

Raciocínio Logico

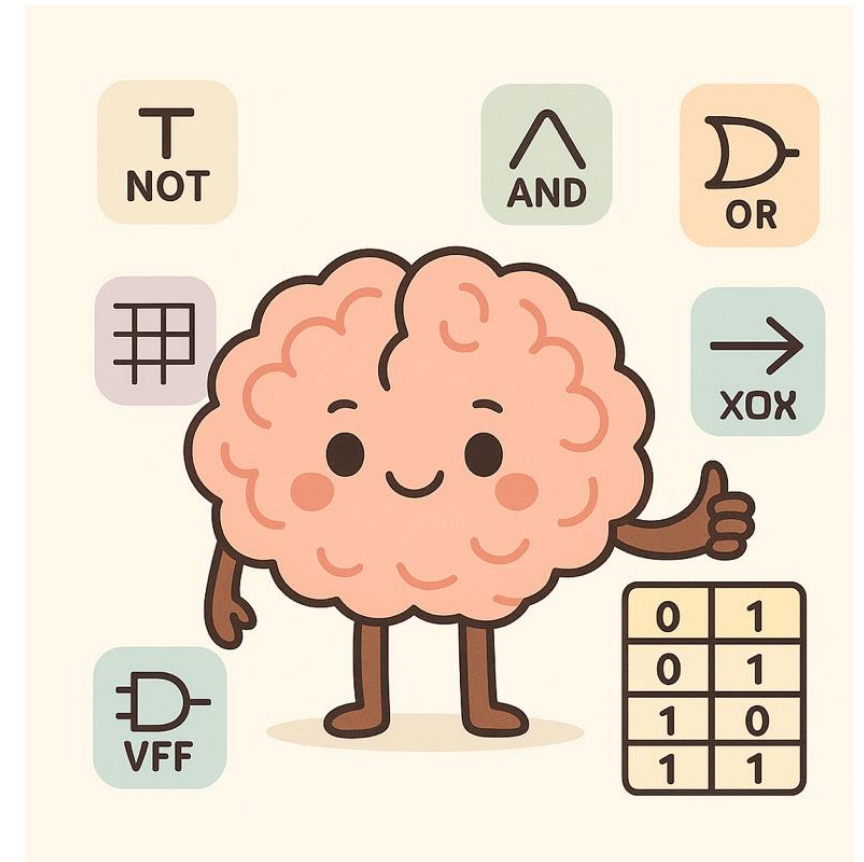
Lógica é o processo de partir de **premissas** (afirmações que assumimos verdadeiras) e chegar a uma **conclusão** seguindo regras válidas de inferência.

Ela também serve para **avaliar a validade de um raciocínio**. Foi sistematizada por **Aristóteles**, que a utilizava para verificar a consistência das ideias na filosofia.

Uma conclusão pode ser **verdadeira ou falsa**. Para expressar relações entre proposições, usamos os **conectivos lógicos**:

- $\neg$ (NÃO / NOT)
- $\wedge$ (E / AND)
- $\vee$ (OU / OR)
- $\oplus$ (OU-exclusivo / XOR)
- $\rightarrow$ (SE... ENTÃO / IF... THEN)
- $\leftrightarrow$ (SE E SOMENTE SE / IFF)

As **tabelas-verdade** organizam as premissas, mostrando todas as combinações possíveis de entrada e o resultado da saída, facilitando a análise e a construção de circuitos digitais.



Logica Dedutiva e Indutiva

Lógica Dedutiva

Parte de **premissas gerais(universais)** para chegar a conclusões específicas.

Se as premissas forem verdadeiras e o raciocínio válido → conclusão **necessariamente correta**.

Exemplo:

- Premissa: Todos os humanos são mortais.
- Premissa: Sócrates é humano.
- Conclusão: Sócrates é mortal. 

Lógica Indutiva

Parte de **observações específicas** para formular conclusões gerais. Suas conclusões são **prováveis**, não garantidas.

Exemplo:

- Observei 100 cisnes, todos brancos.
- Conclusão: "Todos os cisnes são brancos."  (falso se existir um cisne negro).

E nos Computadores?

- Computadores usam **lógica dedutiva** (booleana).
- Portas lógicas (AND, OR, NOT, etc.) garantem que a saída é **determinística** a partir das entradas.
- O problema surge quando lidamos com o **mundo real** → precisamos traduzir situações incertas (indutivas) em regras dedutivas.

	LÓGICA DEDUTIVA		LÓGICA INDUTIVA
	Premissa: Sócrates é mortal		Observei 100 cisnes todos brancos.
	Conclusão: Sócrates é mortal.		Conclusão: "Todos os cisnes são brancos."
			 (falso se existir um cisne negro)
	E NOS COMPUTADORES?		

Se agirmos logicamente sempre estaremos certos?

👉 Não necessariamente. A lógica garante apenas que a **conclusão segue corretamente das premissas**. Se as **premissas forem falsas**, a conclusão pode ser “lógica”, mas **não corresponde à realidade, por exemplo:**

Exemplo:

- Premissa 1: “Todos os gatos sabem falar.” ❌
- Premissa 2: “Mingau é um gato.” ✅
- Conclusão: “Logo, Mingau sabe falar.” ❌

Lógica está correta, mas o conteúdo é falso.

“A lógica garante validade, mas não garante verdade. A verdade depende das premissas.”

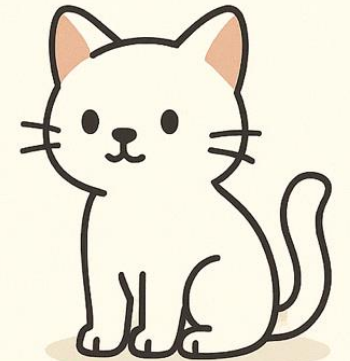
Se agirmos logicamente sempre estaremos certos?

Exemplo:

Premissa 1:
“Todos os gatos sabem falar.” ❌

Premissa 2:
“Mingau é um gato.” ✅

Conclusão:
“Logo, Mingau sabe falar.” ❌



Lógica está correta, mas o conteúdo é falso.

Se as premissas estiverem corretas e agirmos logicamente, estaremos sempre certos?

Se as **premissas forem verdadeiras** e o **raciocínio lógico for válido**, então a **conclusão será necessariamente correta**.

- Esse é justamente o poder da lógica dedutiva:
- **Premissas verdadeiras + regras de**
- **inferência corretas → conclusão verdadeira.**

⚠ **Mas atenção: isso vale para a lógica dedutiva formal.**

- **Em lógica indutiva (ex.: ciência experimental), mesmo premissas corretas podem levar a conclusões prováveis, mas não absolutas.**
- **Em vida real, nem sempre temos todas as premissas ou elas podem mudar com o tempo.**

Porem no mundo dos computadores... seguir a logica significa estar certo, o problema ocorre quando precisamos fazer programas para o mundo real, com premissas que mudam com o tempo.

Se as premissas estiverem corretas e agirmos logicamente, estaremos sempre certos?



Esse é justamente o poder da lógica dedutiva:

Premissas verdadeiras + regras de inferência corretas → conclusão verdadeira.



Mas atenção: isso vale para a lógica dedutiva formal.

Um Pouco de Historia

Em meados de 1800, o pesquisador [George Boole](#) buscava compreender a relação fundamental entre a matemática e o raciocínio lógico. Nesse contexto, ele desenvolveu um novo ramo da matemática conhecido como [Álgebra Booleana](#).

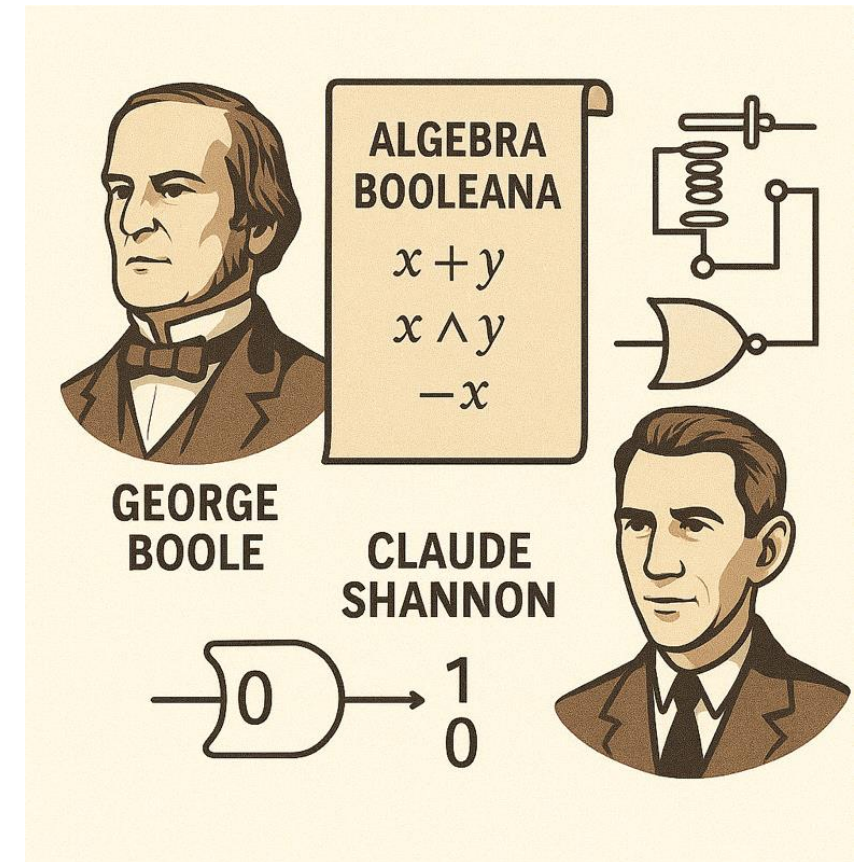
Décadas mais tarde, em 1937, [Claude Shannon](#) estabeleceu a conexão entre a Álgebra Booleana e os circuitos elétricos, utilizando chaves baseadas em dispositivos chamados **relés**.

Ele demonstrou, de forma prática, como as funções e postulados da Álgebra Booleana poderiam ser implementados fisicamente por meio de chaves elétricas, abrindo caminho para a eletrônica digital moderna.

👉 Resumindo:

- **Aristóteles** criou a lógica como ciência do raciocínio.
- **Boole** transformou a lógica em matemática.
- **Shannon** aplicou a lógica aos circuitos.

Hoje sabemos que toda informação pode ser convertida em 0s e 1s e transmitida para qualquer lugar do mundo.



Linha do Tempo da Eletrônica Digital

Segue aqui a linha do tempo dos componentes que definem a eletrônica digital, antes disso entre 1642 até 1835, tínhamos máquinas de calcular mecânica como as Máquinas de Blaise Pascal e Charles Babbage

Relés (anos 1930–1940)

- Computadores eletromecânicos (ex.: Harvard Mark I).
- Implementavam portas lógicas com chaves mecânicas.
- Lentos e barulhentos.

Válvulas Eletrônicas (anos 1940–1950)

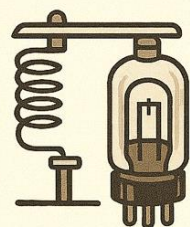
- Primeiros computadores eletrônicos (ex.: ENIAC).
- Muito mais rápidas que relés.
- Consumo alto e baixa durabilidade.
- **Transistores (anos 1950–1960)**
- Mais rápidos, confiáveis e eficientes.
- Permitem miniaturização.
- Base dos primeiros computadores comerciais de grande escala.

Circuitos Integrados (anos 1960–1970)

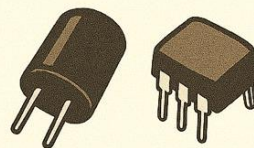
- Vários transistores em um único chip.
- Redução de custo, tamanho e aumento de performance.
- Revolução da microeletrônica.

FPGAs / ASICs (anos 1980 em diante)

- Projetos digitais altamente complexos.
- Configuráveis (FPGAs) ou dedicados (ASICs).
- Base dos sistemas embarcados e da computação moderna.



**VÁLVULAS
ELETRÔNICAS**
(ANOS 1940–1950)

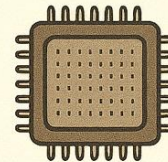


TRANSISTORES
(ANOS 1950–1960)

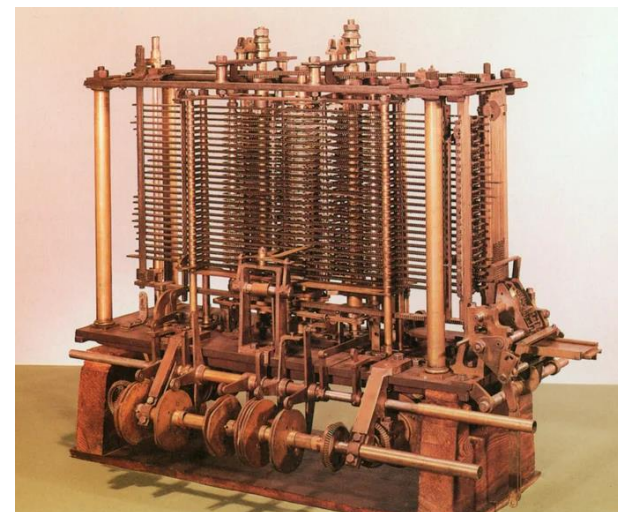
RELÉS
(ANOS 1930–1940)

**VÁLVULAS
ELETRÔNICAS**
(ANOS 1950–1960)

**CIRCUITOS
INTEGRADOS**
(ANOS 1960 EM DIANTE)



FPGAs / ASICs
(ANOS 1980 EM DIANTE)



Relés

Os relés são chaves ON/OFF usando dispositivos eletromecânicos controlados por uma corrente.

Uso Histórico

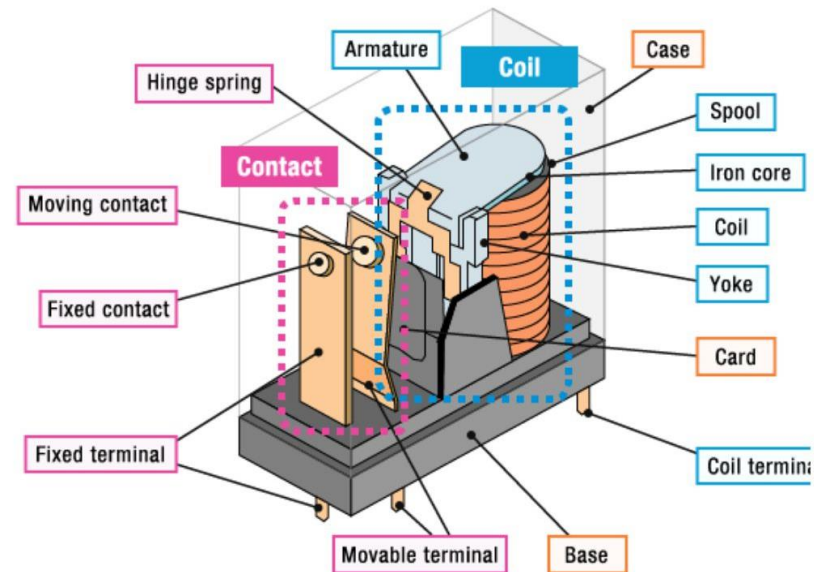
- Antes dos transistores e válvulas, os **relés eletromecânicos** implementavam funções lógicas.
- Funcionamento: a **bobina gera um campo magnético** → movimenta contatos.
- Tipos de contatos:
 - **NA** (Normalmente Aberto)
 - **NF** (Normalmente Fechado)
 - **C** (Comum)
- Com essas combinações era possível construir portas: **NOT, AND, OR, XOR**.

Problemas dos Relés

- **Lentos** (milissegundos por comutação).
- **Barulhentos** (cliques mecânicos).
- **Desgaste mecânico** → vida útil limitada.
- **Grandes e pesados**.
- **Consumo elevado de energia**.

Importância

- Primeira forma prática de **eletrônica digital**.
- Permitiram segurança: **circuitos de baixa corrente controlando cargas maiores**.
- Foram a base para a evolução → **válvulas** → **transistores** → **circuitos integrados**.



Válvulas Eletrônicas

Valvulas eletronicas, surgiram no início do século XX como **amplificadores e interruptores eletrônicos** os primeiros computadores digitais totalmente eletrônicos (ex.: **ENIAC, 1945**) usaram **milhares de válvulas**. Eles substituíram os relés por serem **muito mais rápidas** (tempo de comutação em microssegundos).

Uso em Portas Lógicas

Uma válvula podia funcionar como um **interruptor controlado por tensão**. Combinando válvulas, era possível implementar: NOT, AND, OR

Assim, criavam-se os primeiros **circuitos digitais eletrônicos**.

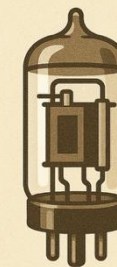
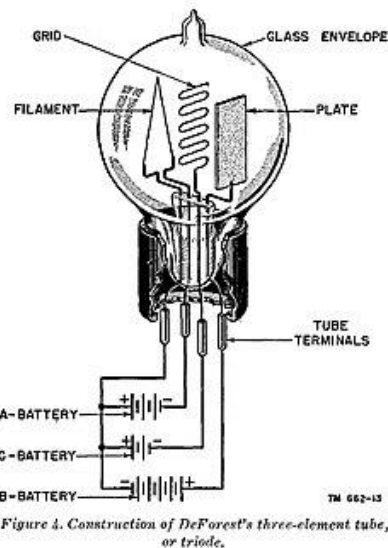
Problemas

- Consumo elevado de energia.
- Muito calor.
- Baixa confiabilidade (queima frequente).
- Grandes e caros.

Uso Atual das Válvulas

- Ainda usadas em nichos onde suas características são valorizadas
- ✓ **Áudio Hi-Fi e amplificadores de guitarra** (som “mais quente”).
- ✓ **Transmissão de rádio e TV de alta potência**.
- ✓ **Equipamentos militares e científicos antigos**.

Em lógica digital, foram totalmente substituídas por **transistores e CIs**.



⚡ Transistores MOSFETs

◆ O que é um MOSFET?

Metal-Oxide-Semiconductor Field-Effect Transistor atua como interruptor controlado por tensão (V_{gs}).

Dois tipos principais:

- **NMOS** → conduz quando $V_{gs} > V_{th}$ (liga ao nível lógico '1').
- **PMOS** → conduz quando $V_{gs} < V_{th}$ (liga ao nível lógico '0').

◆ Como usar em lógica digital

- **CMOS (Complementary MOS)** combina NMOS e PMOS.
- Saídas sempre conectadas à **Vdd** ou **GND** → baixo consumo estático.

◆ Vantagens do CMOS

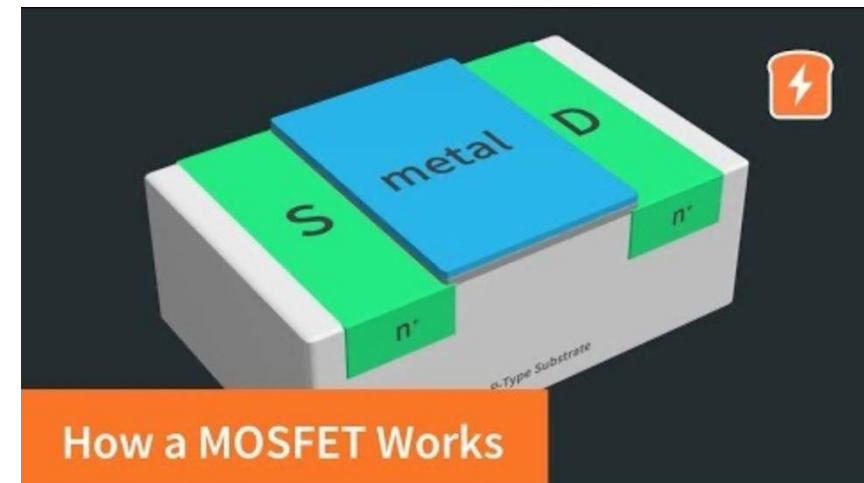
- Alta densidade de integração.
- Baixo consumo estático (só consome em transição).
- Base da fabricação de microprocessadores, memórias e ASICs.

◆ Por que não usar só PMOS (ou só NMOS)?

Se você montar portas **só com PMOS** (ou só com NMOS), a saída **nunca terá níveis lógicos “perfeitos”**.

- Com **PMOS-only**: a saída em nível baixo não desce até 0 V (fica “flutuando” por causa da queda V_{th}).
- Com **NMOS-only**: a saída em nível alto não sobe até Vdd (também fica degradada).

Devido ao tamanho quase atômico dos transistores, eles criaram a verdadeira revolução tecnológica do nosso século.



⚡ MOSFET como uma chave

◆ Estrutura do MOSFET

- **Gate (G):** controla o fluxo, como a “chave” do transistor. (V_{gs})
- **Source (S):** terminal por onde entram os portadores (elétrons no NMOS, lacunas no PMOS).
- **Drain (D):** terminal por onde saem os portadores.

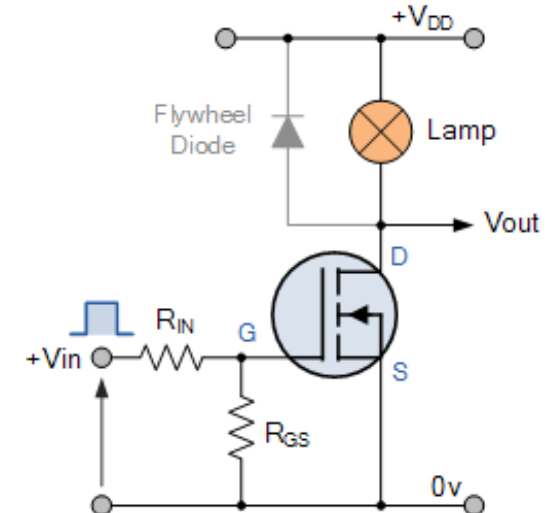
◆ Princípio de funcionamento

NMOS:

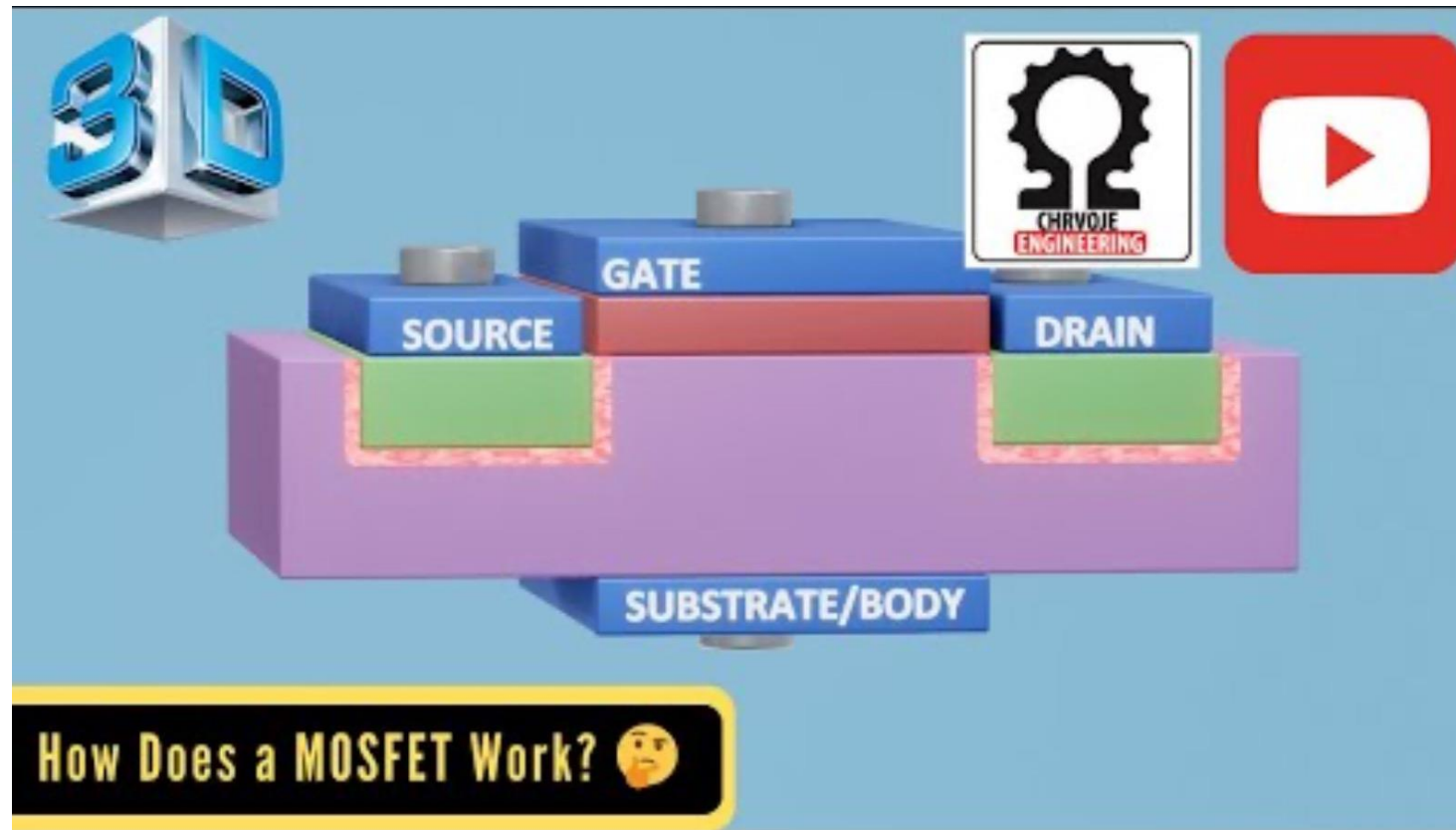
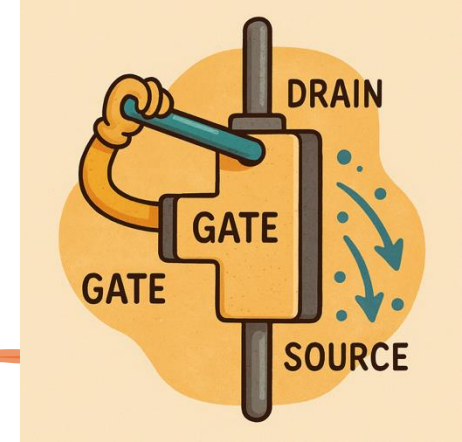
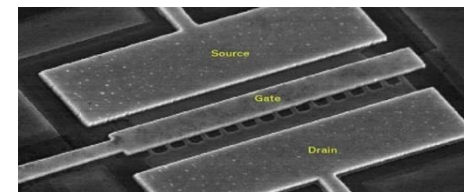
- Liga (conduz de Drain → Source) quando $V_{gs} > V_{th}$.
- Age como um “interruptor fechado” controlado pela tensão no Gate.

PMOS:

- Liga (conduz de Source → Drain) quando $V_{gs} < V_{th}$.
- Complementar ao NMOS.



Videos Mosfet



⚡ Limites da Eletrônica Digital

Hoje, os circuitos digitais são compostos por bilhões de transistores formando portas lógicas. Essa miniaturização trouxe enorme poder de processamento, mas também novos desafios.

◆ Escala Física

- **Elétron:** $\approx 10^{-15}$ m (tamanho efetivo, raio clássico $\sim 2,8$ femtômetros).
- **Átomo de silício:** $\sim 0,1$ nm (100 pm).
- **Transistores atuais (2024/2025):** ~ 3 nm (largura efetiva do canal ≈ 30 átomos).

◆ Problemas Atuais

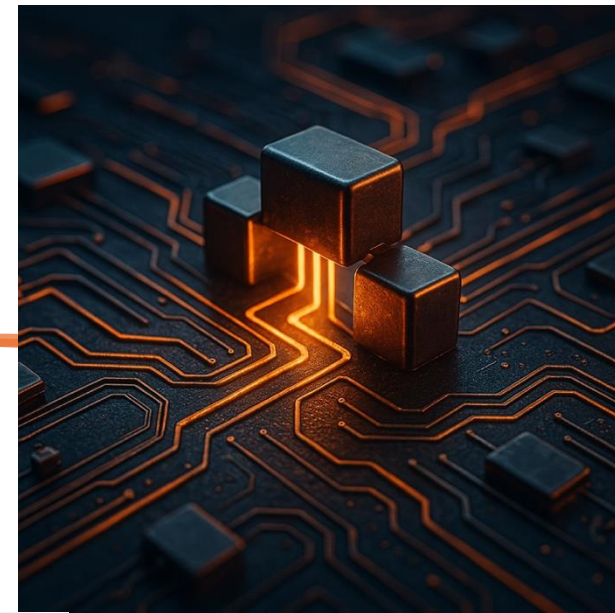
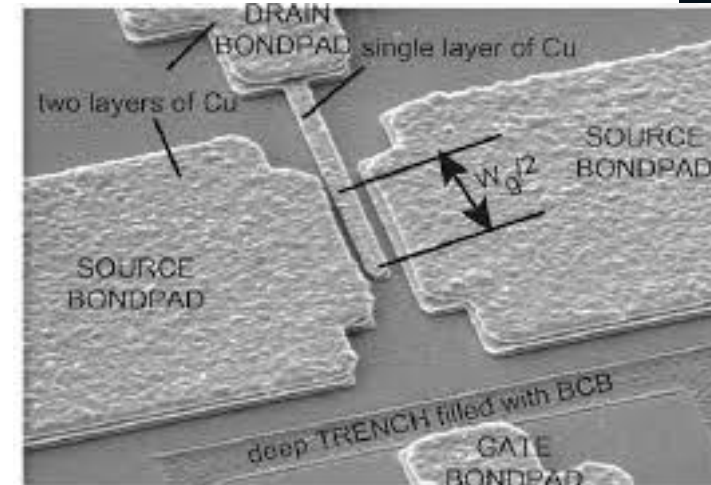
- **Efeito Túnel Quântico:** elétrons “atravessam” barreiras $\rightarrow$ vazamento de corrente.
- **Atraso nas interconexões:** fios metálicos internos limitam velocidade mais que o transistor.
- **Consumo de energia e calor:** bilhões de transistores = alto gasto energético.
- **Variabilidade:** pequenas imperfeições de fabricação alteram o comportamento.

◆ Soluções em uso

- **Arquiteturas 3D** (chipselets, empilhamento).
- **Novos transistores:**
- **FinFET** (usado em 16–7 nm).
- **GAAFET / Nanoribbons** (3 nm e abaixo).
- **Técnicas de energia:** clock gating, DVFS, power gating.
- **Materiais avançados:** grafeno, MoS_2 , nanotubos de carbono.

🧠 Futuro

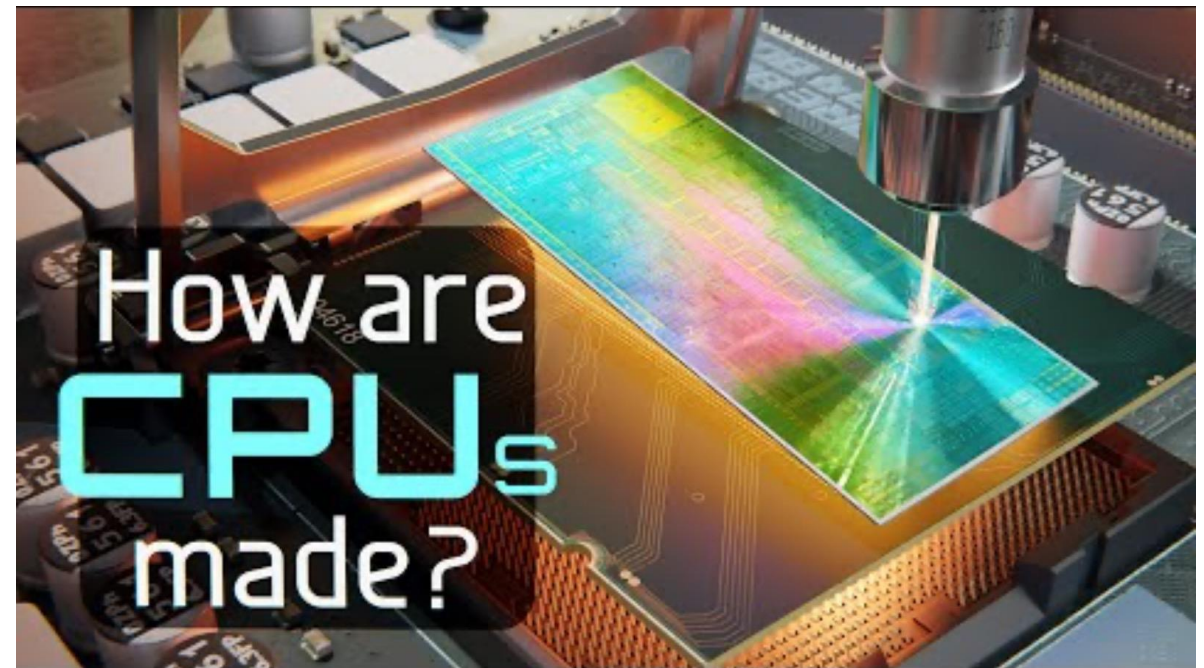
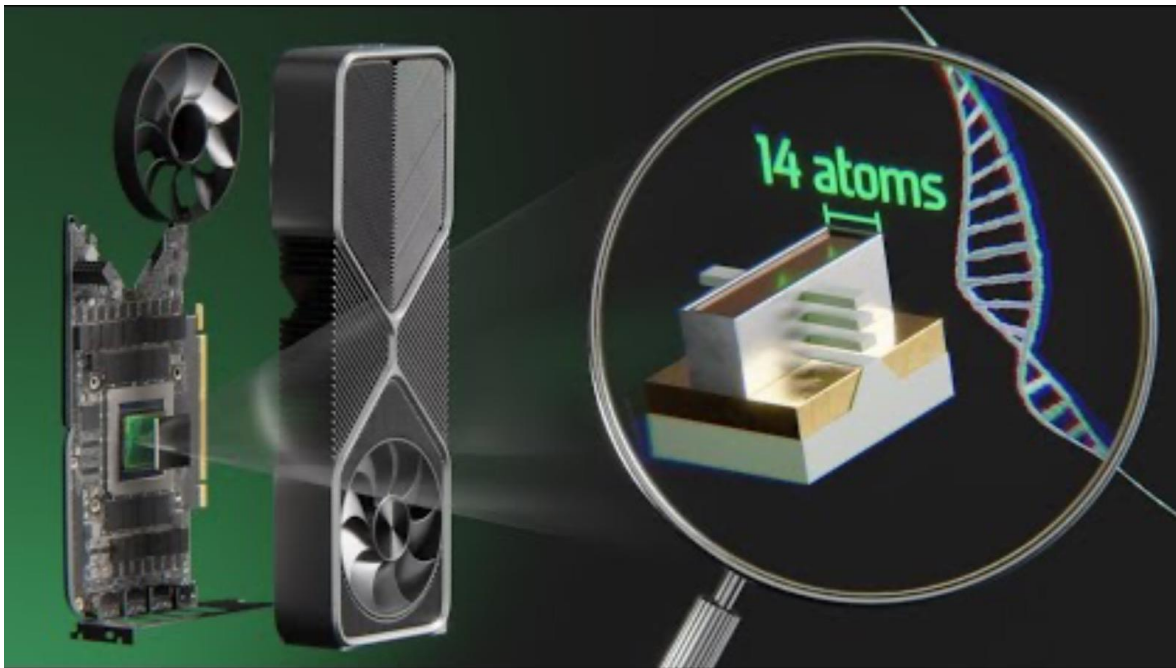
- Transistores menores que 1 nm já são **comparáveis ao tamanho de poucos átomos**.
- Caminhos promissores: **computação quântica, neuromórfica e óptica/fotônica**.



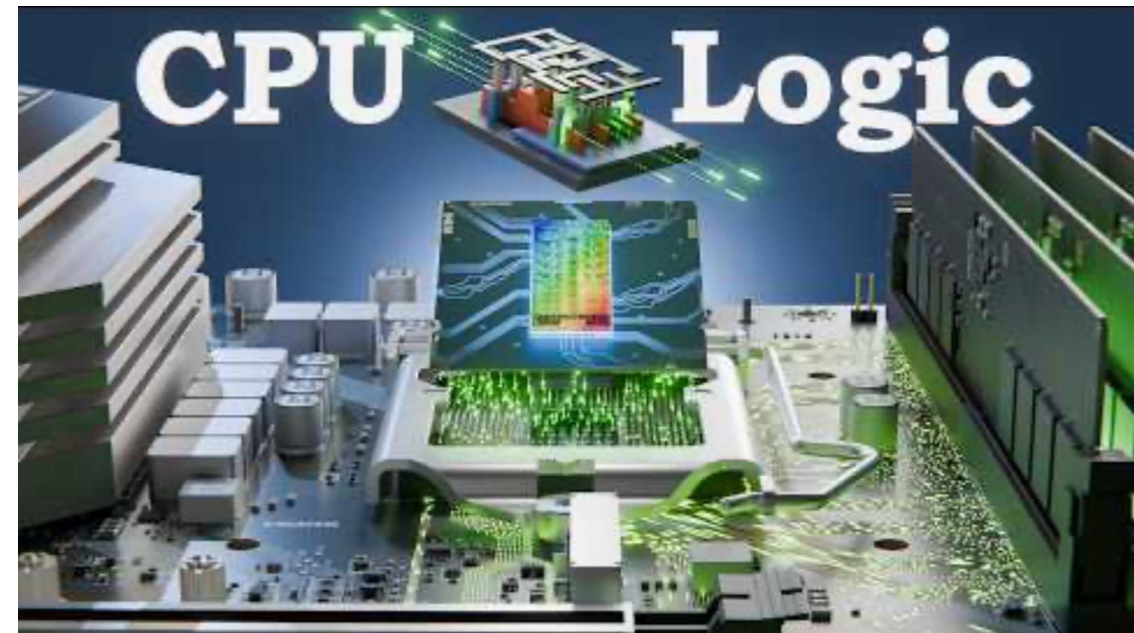
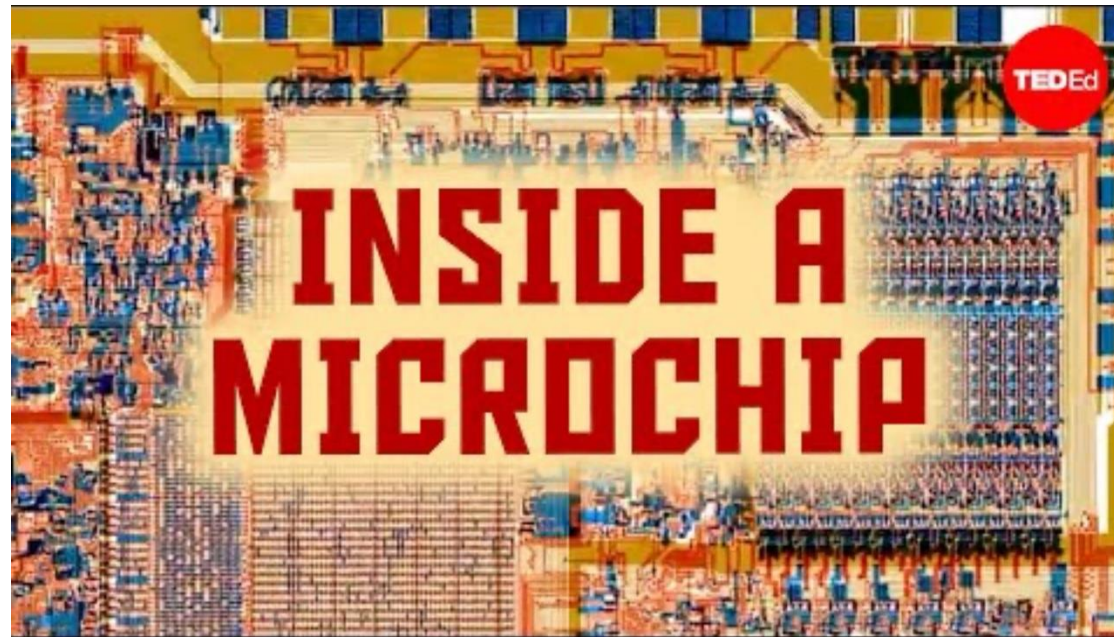
**Why the first
computers
were made
out of Light
Bulbs?**



⚡ Limites da Eletrônica Digital



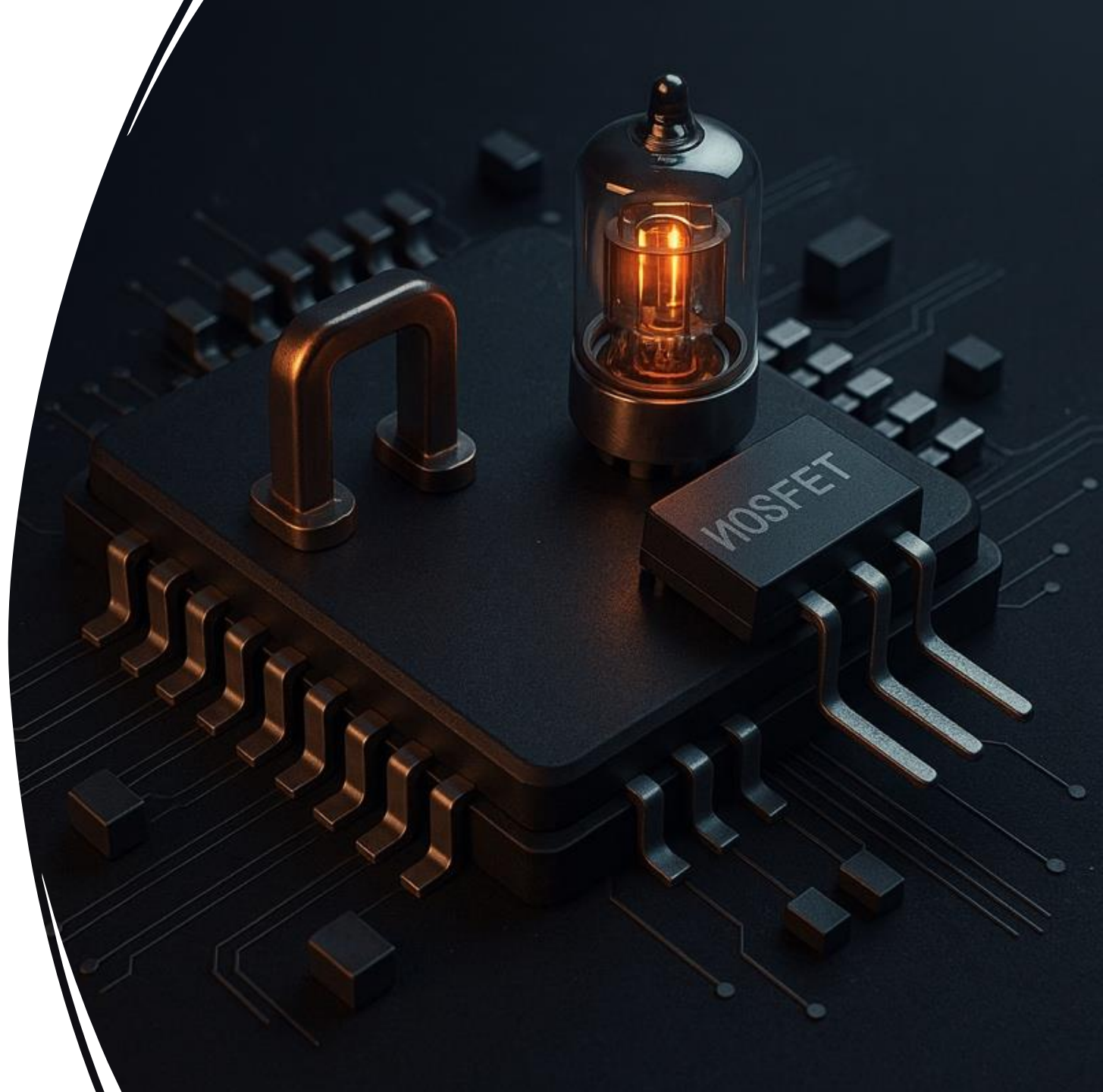
How Microchips are Made?



Fim da Parte 1

A partir deste ponto nos já sabemos:

- Base do Raciocínio lógico e tentativas de trazer seus fundamentos para Matemática.
- Um pouco da Historia e a linha de tempo da eletrônica digital
- Oque são os Reles, Válvulas e Transistores
- Limites da Eletrônica Digital





Números Booleanos



Hoje sabemos que é possível codificar qualquer informação (áudio, vídeo, texto, algoritmos) usando apenas **zeros e uns**.

Por isso, começamos nossa jornada aprendendo a representar os **números decimais** no **sistema binário**.

Conversão para Decimal

Exemplo: 1011_2

- $= (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$
- $= 8 + 0 + 2 + 1 = 11_{10}$

Conversão para Binário

Exemplo: $13_{10} \rightarrow ?$

- $13 \div 2 = 6$ resto **1**
- $6 \div 2 = 3$ resto **0**
- $3 \div 2 = 1$ resto **1**
- $1 \div 2 = 0$ resto **1**
- $\rightarrow 1101_2$

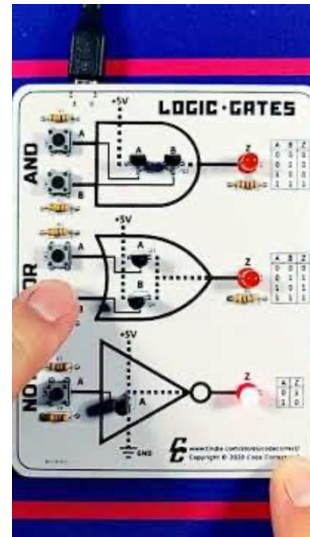
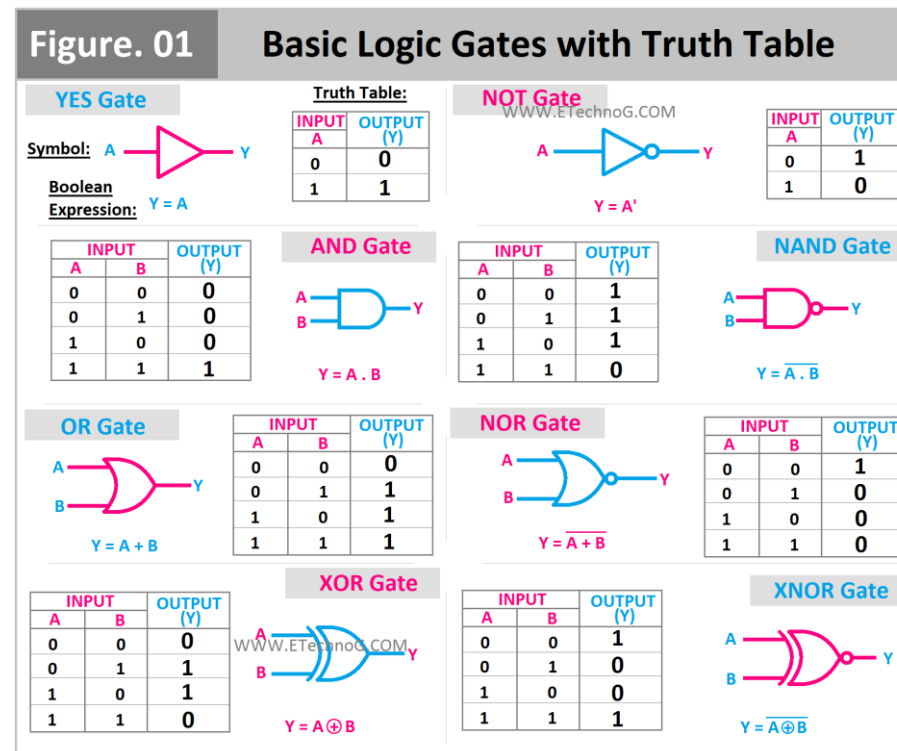
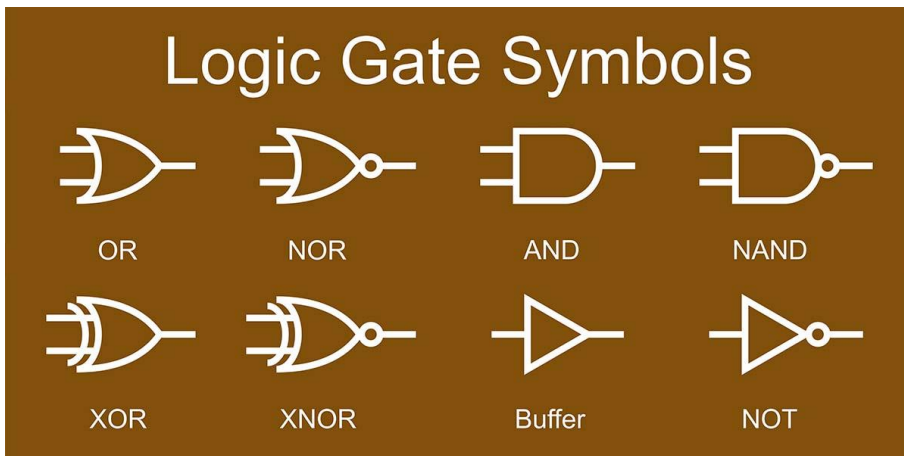
Contando 0, 1, 2, 3, 4, 5, 6, 7

000, 001, 010, 011, 100, 101, 110, 111

Portas Lógicas e Portas Universais

Independentemente de utilizarmos transistores, válvulas ou relés, os operadores booleanos básicos — AND, OR, NOT e XOR — podem ser representados como **portas lógicas**.

Também é importante destacar as **portas universais NAND e NOR**, que possuem a propriedade de poder implementar qualquer outra porta lógica.



Equações digitais (Ou Booleanas)

Representam funções lógicas usando **variáveis binárias** (0 ou 1) e são construídas a partir das portas lógicas.

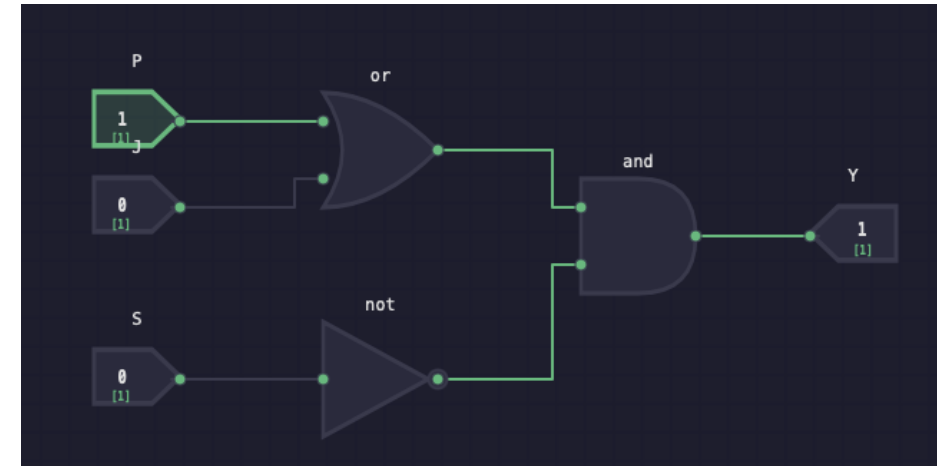
São a **linguagem matemática** usada para descrever circuitos digitais.

✨ Exemplo

Alarme simples $Y = (P + J) \cdot S'$

- Dispara se **porta ou janela abertas** e **não está no modo silencioso**.

Tabela Verdade representa todas as opções possíveis de entrada e saídas de uma equação.



Porta (P)	Janela (J)	Silencioso (S)	Alarme (Y)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0



Circuitos Combinacionais

Todo circuito cuja **saída depende apenas das entradas atuais**.

São **rápidos** porque não armazenam estados.

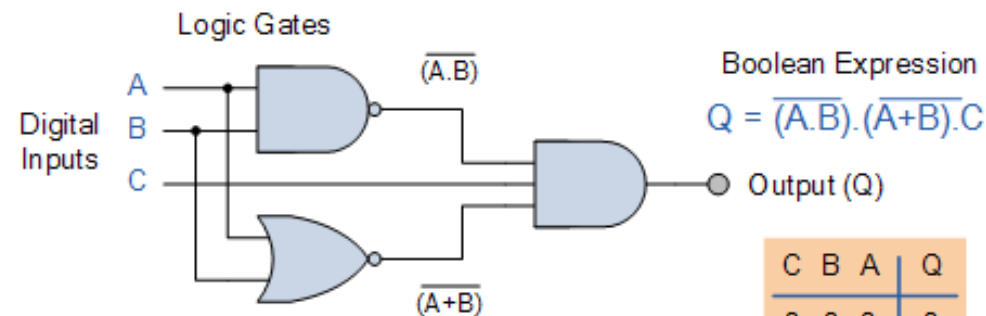
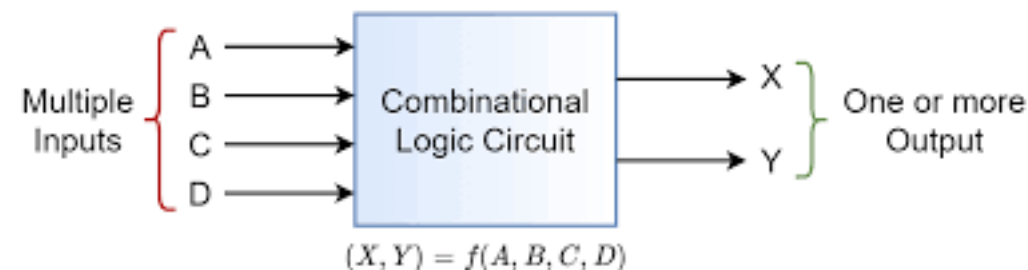
Exemplos:

- Somadores
- Multiplexadores
- ALUs (unidade de cálculo dos computadores)



Projeto de Circuitos Combinacionais

- Definir **entradas e saídas**.
- Construir a **tabela-verdade**.
- Simplificar as equações com **Mapa de Karnaugh**.
- Implementar as funções resultantes em **portas lógicas** (soma de produtos).



Logic Diagram

Typical Truth Table

C	B	A	Q
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0



Mapas de Karnaugh

Técnica visual para **simplificar expressões booleanas**.

Valores são organizados em uma **grade** (2, 3, 4 ou mais variáveis).

- Cada célula representa um **mintermo** (combinação de entradas).
- **Agrupamos 1's** em blocos de 1, 2, 4, 8, 16... → resultam em termos simplificados.

Aplicação prática

- Em software, o **sintetizador** já faz a simplificação automaticamente.
- Porém, conhecer a técnica ajuda a **entender e otimizar circuitos**.

No projeto de circuitos combinacionais:

- Criamos a **tabela-verdade**.
- Marcamos os elementos com valor **1**.
- Agrupamos os 1's em blocos de potência de 2.
- Escrevemos a equação final em forma de **Soma de Produtos**, eliminando variáveis redundantes.

a \ b	0	1
0		
1		

a \ b	00	01	11	10
0				
1				

a \ b	00	01	11	10
00				
01				
11				
10				

a \ b	000	001	011	010	110	111	101	100
00								
01								
11								
10								





CI 555

Circuito integrado clássico de **temporização e oscilação**. Internamente possui:

- Divisor de tensão
- Dois comparadores
- Um Latch SR

Um capacitor, junto à rede resistiva e comparadores, cria um ciclo de **carga/descarga triangular** → convertido em uma **onda quadrada** (clock). Podemos também usar um [Crystal de quartz](#) para frequência mais fixa.

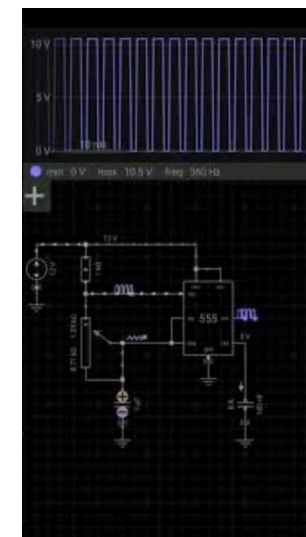
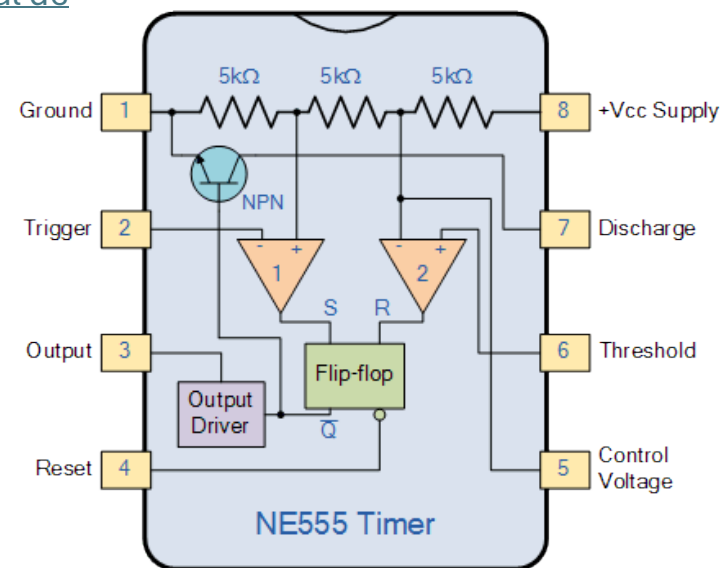
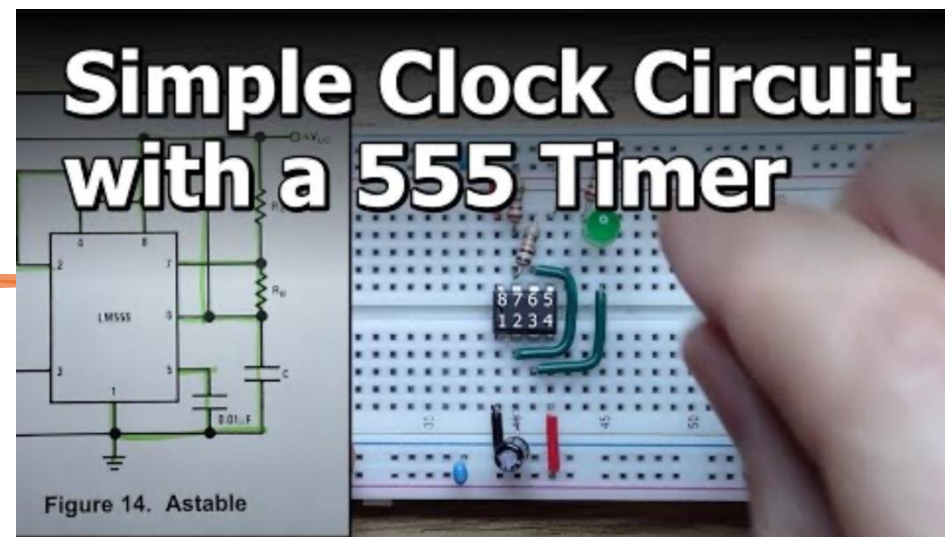
⚡ Modos de Operação

- **Astável** → gera onda quadrada (oscilador / clock)
- **Monoestável** → gera um pulso único (temporizador)
- **Biestável** → atua como flip-flop (set/reset)

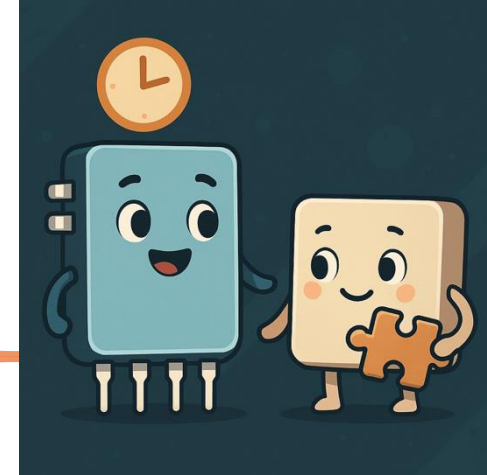
📌 Aplicações

- Geração de sinais de clock (Astável)
- Temporizadores e alarmes
- Divisores de frequência
- Pisca-pisca de LEDs

Osciladores são responsáveis por geral o sinal de clock usando em circuitos sequenciais, eles são uma mistura de circuitos analógicos e digitais, com feedback positivo.



Latches e Flip-Flops



Latches e Flip-flops são os elementos de armazenamento de 1-bit, eles são as unidades básicas necessárias para implementar circuitos sequenciais.

Latches

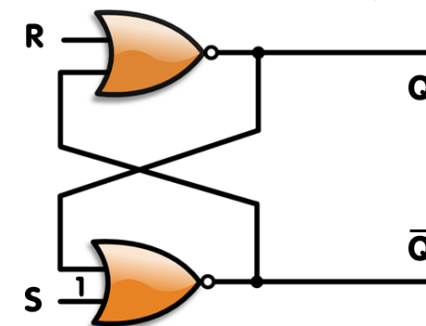
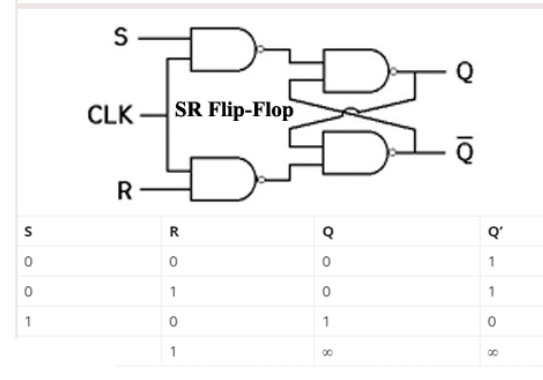
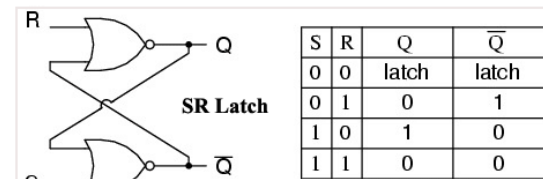
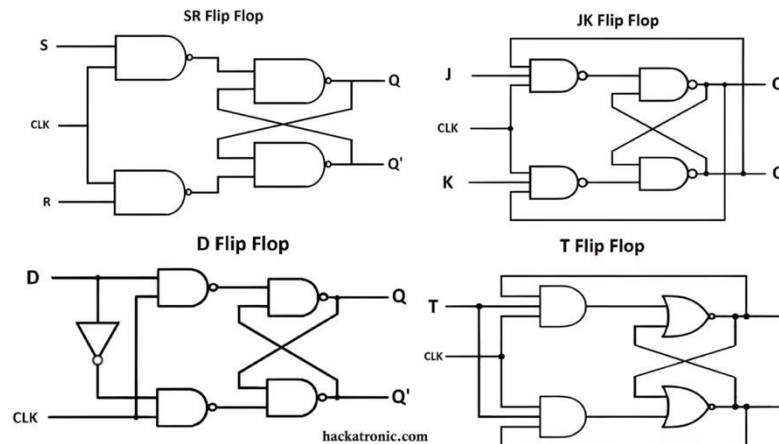
- **Armazenam 1 bit**
- Dependem do **nível do sinal de controle** (não da borda)
- Exemplos: **SR Latch**, **D Latch**
- **Não precisam de clock**

Flip-Flops

- Também armazenam **1 bit**
- **Sensíveis à borda do clock** (subida ou descida)
- Sincronizam dados em circuitos digitais

Importância

- **Latches**: simples, mas raramente usados em FPGAs
- **Flip-Flops (D)**: base de registradores, contadores e memórias
- **FPGAs modernos usam flip-flops D**, não latches, pois garantem sincronismo e análise de tempo confiável, você também consegue criar latches no FPGA dependendo do seu código, porem não é recomendado.



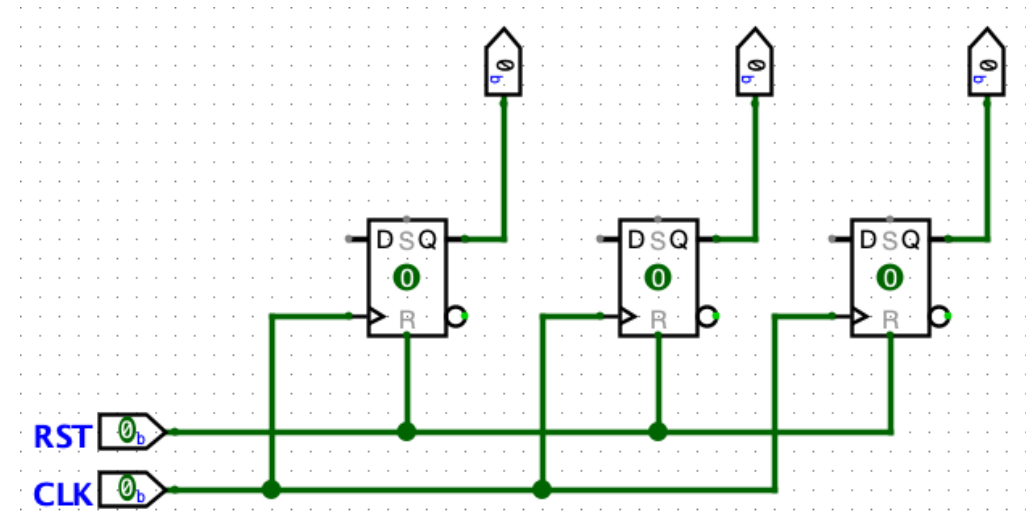
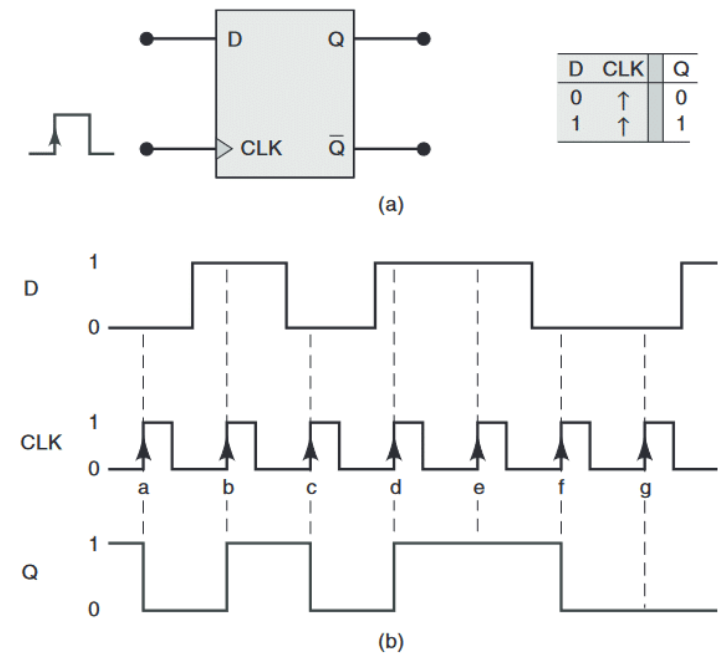
Flip-Flop D

O Flip-Flop D é o mais comum e usado, ele possui:

- Uma entrada D para definir o estado futuro, após a borda do clock ser detectada
- Uma saída Q para o estado atual
- Uma porta CLK para o clock
- Uma porta RST para o reset

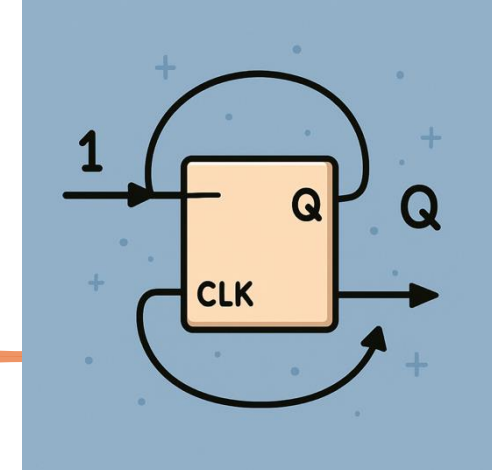
Os circuitos contadores, ou FSM tem vários flip-flops conectados, e irão ter lógica combinacional para definir o estado futuro, e o a saída atual, onde:

- Se a saída atual depender apenas do estado atual Q, teremos uma máquina Moore
- Se a saída atual depender do estado atual Q e das entradas, teremos uma máquina Mealy





Circuitos Sequenciais



Diferente dos **combinacionais**, onde a saída depende apenas das entradas atuais... Nos **sequenciais**, a saída depende de:

- **Entradas atuais**
- **Estado anterior (memória)**

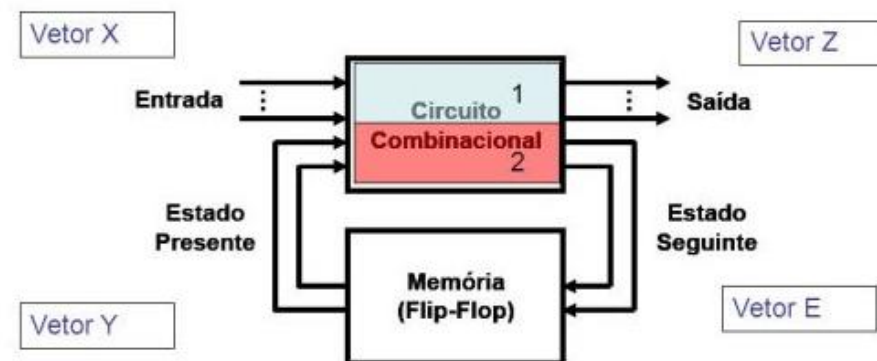
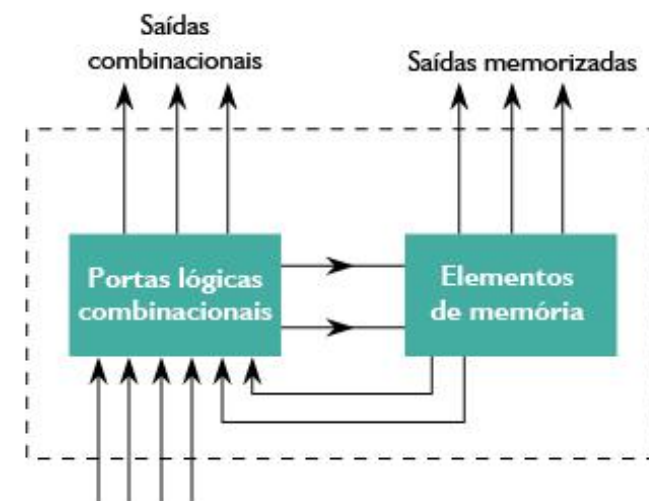
📌 Características

- Possuem **elementos de memória** (latches, flip-flops, registradores).
- São controlados por **sinais de clock**.
- Podem armazenar e processar **informações ao longo do tempo**.

🧩 Exemplos

- **Contadores** (binários, decimais, mod-N)
- **Registradores** (armazenamento temporário de dados)
- **Máquinas de Estado Finitas (FSMs)**
- **Memórias e processadores**

Circuitos sequenciais são a **ponte entre a lógica combinacional e sistemas digitais completos**, permitindo a criação de **computadores, controladores e autômatos**.





Máquinas de Estado Finitas

São [modelos](#) usados para projetar circuitos sequenciais



Tipos

Moore

- Saída associada **somente ao estado**.
- Mais simples, estável. (Melhor em FPGA evita metaestabilidade)
- Pode precisar de mais estados.

Mealy

- Saída depende de **estado + entrada**.
- Menos estados necessários.
- Saída pode mudar imediatamente com a entrada.



Como projetar uma FSM

- **Definir o problema** (entradas, saídas, condições).
- **Listar estados** necessários.
- **Desenhar o diagrama de estados**.
- Construir a **tabela de transições** (estado atual → próximo estado).
- Codificar os estados em binário.
- Derivar as equações (Karnaugh ou diretamente em HDL).
- Implementar com **flip-flops + lógica combinacional**.

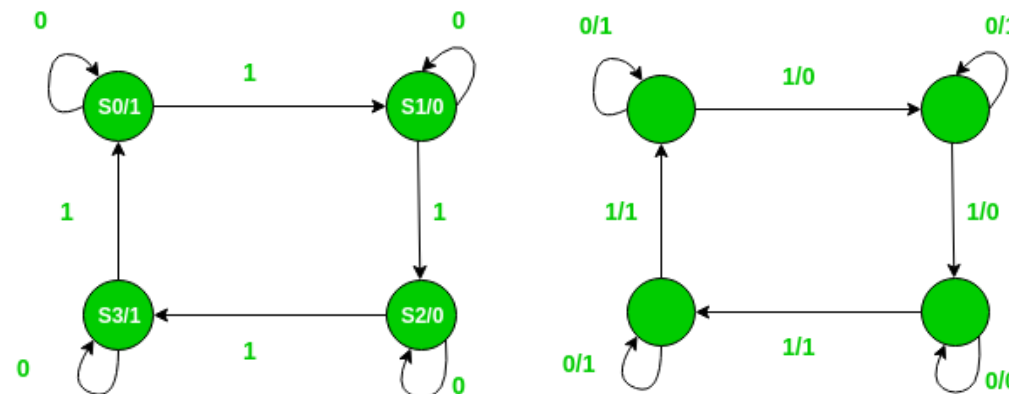
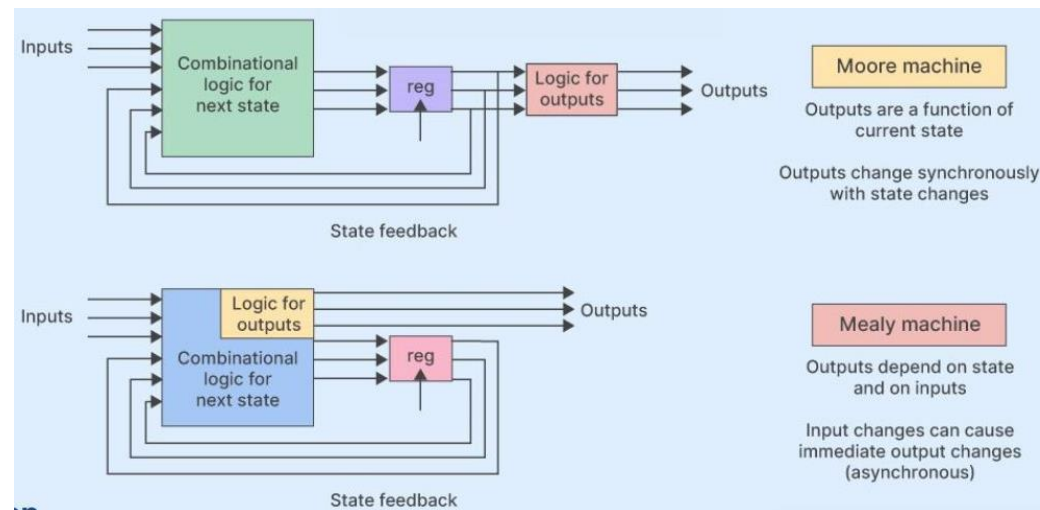


Figure - Moore machine

Figure - Mealy machine



Contadores

O que são?

- Circuitos [sequenciais](#) que **contam pulsos de clock**.
- Armazenam valores binários em **flip-flops**.
- Usados em **temporização, divisores de frequência, registradores de eventos e endereçamento de memória**.

Tipos

Assíncronos (Ripple)

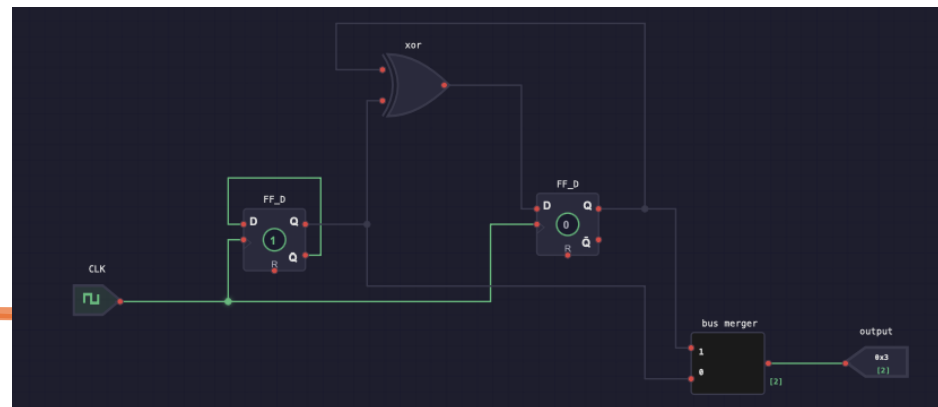
- Cada flip-flop é acionado pela saída do anterior.
- Mais simples, mas sofrem com **atrasos de propagação**.
- Na prática não são usados em circuitos modernos
- Aumentam o Risco da Metaestabilidade

Síncronos

- Todos os flip-flops recebem o **mesmo clock**.
- Mais rápidos e confiáveis, usados em sistemas modernos.

Resumindo:

- Contadores = base da temporização digital.
- **Assíncronos** → simples, mas lentos.
- **Síncronos** → mais robustos, padrão em **FPGAs**.



Metaestabilidade

Fenômeno que ocorre quando um **flip-flop recebe uma entrada que muda muito perto da borda de clock**.

O flip-flop pode ficar em um **estado indefinido (nem 0 nem 1)** por um tempo. Isso pode causar **glitches, travamentos ou comportamentos imprevisíveis** em FSMs e circuitos sequenciais.

Por que acontece?

- Entradas **assíncronas** vindas de fora do domínio de clock.
- Atrasos de propagação dentro do chip.
- Cross-domain clock (quando sinais passam entre blocos com clocks diferentes).

Consequências

- Estados inválidos em máquinas de estado.
- Erros intermitentes (difíceis de reproduzir).
- Sistemas que funcionam em teste, mas falham em operação real.

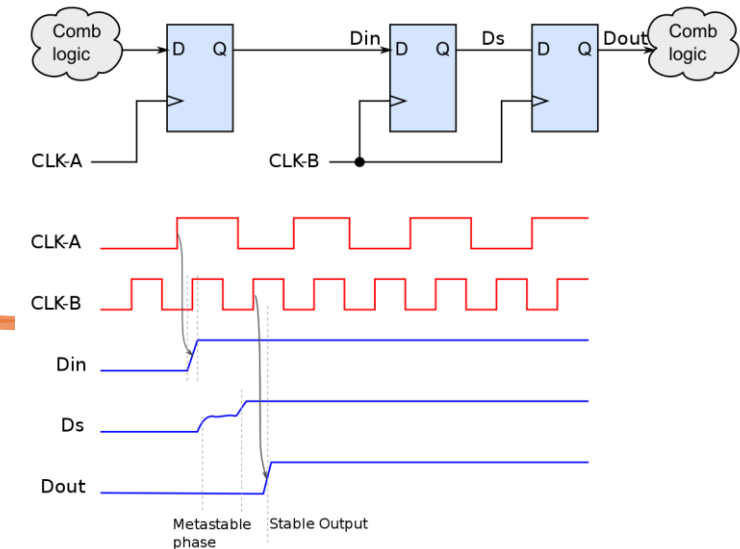
Como Evitar Metaestabilidade

- **Sincronizar entradas assíncronas** com 2 ou mais flip-flops em série.
- Usar sempre **um único domínio de clock** quando possível.
- Em comunicação entre domínios de clock, usar:
 - **FIFO assíncrona**
 - **Handshake (req/ack)**
 - **Gray code** para contadores.
- Projetar FSMs de forma **síncrona e estável** (Moore ajuda na previsibilidade).

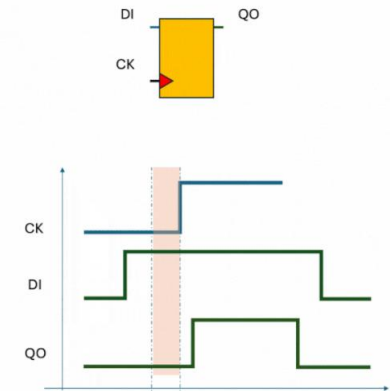
Resumindo

Devemos mudar a entrada do nosso flip-flop antes que o clock chegue.

Metaestabilidade **não pode ser 100% eliminada**, mas pode ser **reduzida a níveis aceitáveis** com boas práticas de design.



The SETUP time characterization



Same is done for different input slew ...

Logisim Evolution

◆ O que é?

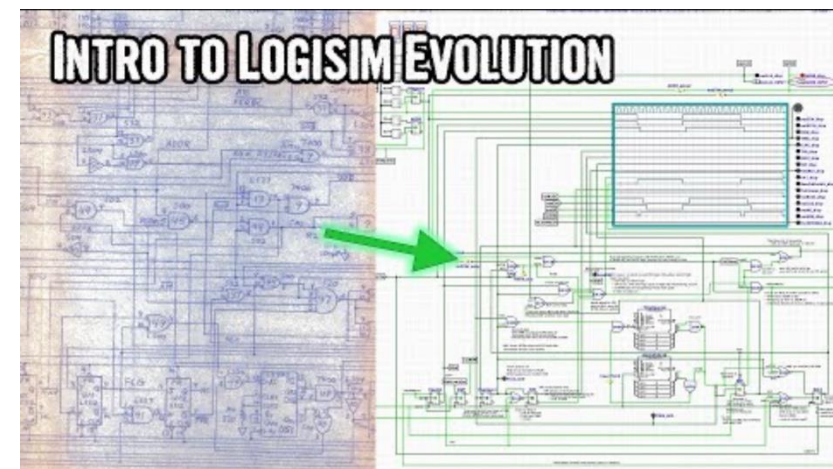
- Software **open source** para **projetar e simular** [circuitos digitais](#) a partir do esquemático.
- Muito usado em ensino de eletrônica digital.
- Multiplataforma (Windows, macOS, Linux).

◆ Principais Recursos

-  **Editor de circuitos** simples e intuitivo.
-  **Simulação lógica** em tempo real.
-  **Cronogramas** (visualizar evolução de sinais).
-  Biblioteca enorme: LEDs, switches, SoCs, TTLs, flip-flops, etc.
-  Integração com placas eletrônicas reais.
-  **Componentes em VHDL** → comportamento customizado.

◆ Onde usar?

- Aprendizado de lógica digital (portas, flip-flops, FSMs).
- Desenvolvimento de protótipos didáticos.
- Ensino de **VHDL/FPGA** de forma visual.

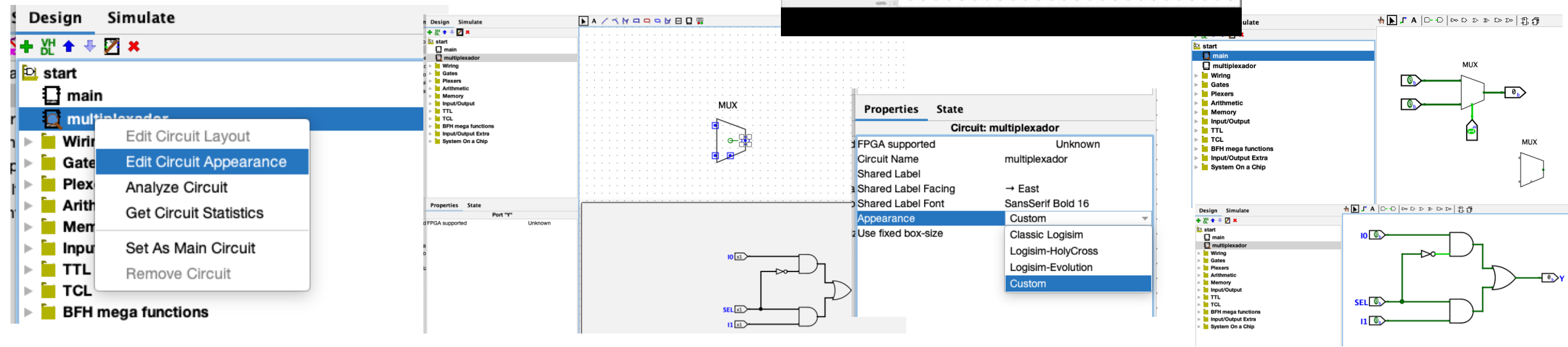
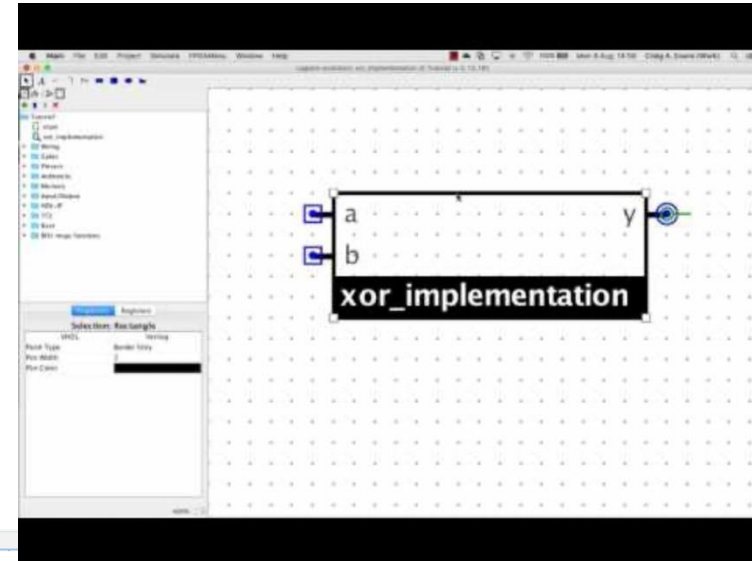


Logisim Sub-circuitos e Aparência

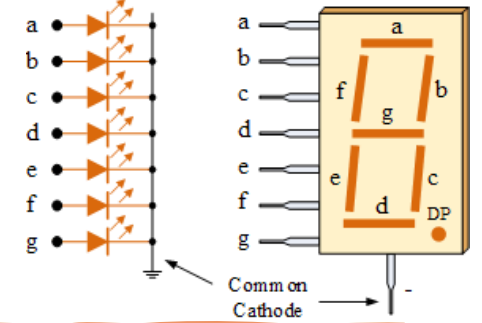
O Logisim permite criar **hierarquia de circuitos**, organizando projetos em subcircuitos reutilizáveis.

Cada subcircuito pode ter sua **aparência personalizada**, facilitando a leitura do diagrama.

Atenção: por padrão o Logisim não habilita a opção *Custom*. É necessário ativá-la manualmente para cada subcircuito.

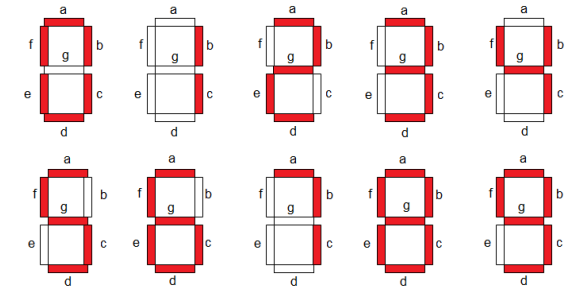


Display de 7 Segmentos

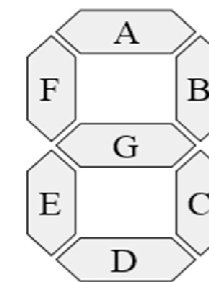
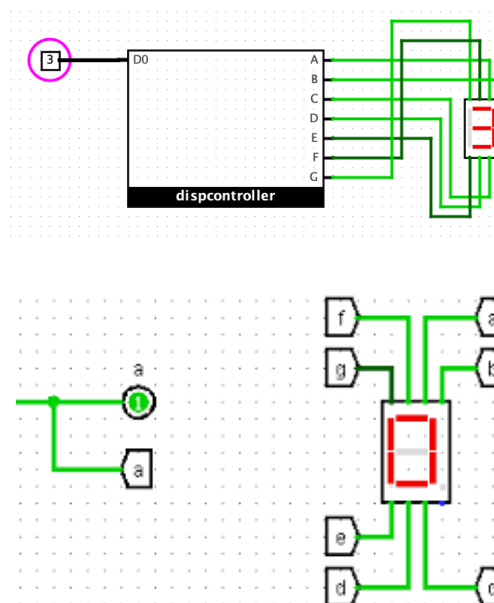
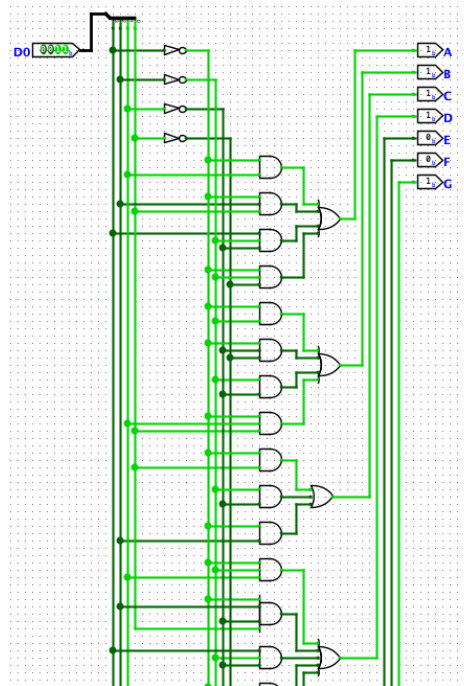


É um display formado por Leds usados para representar números de 0 a 9, ele tem 7 entradas (a .. g), com elas formamos os dígitos de 0 a 9, conforme a tabela ao lado.

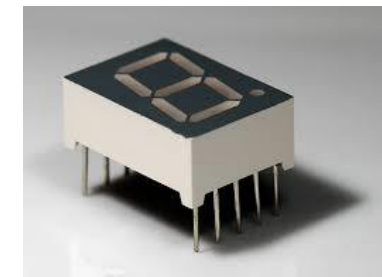
Vamos criar um circuito combinacional de 4 entradas e 7 saídas para controlar o display. Veja no Logisim, que podemos criar a tabela e já ter o circuito simplificado.



16 of 16 rows shown							
D0[3..0]	A	B	C	D	E	F	G
0000	1	1	1	1	1	1	0
0001	0	1	1	0	0	0	0
0010	1	1	0	1	1	0	1
0011	1	1	1	1	0	0	1
0100	0	1	1	0	0	1	1
0101	1	0	1	1	0	1	1
0110	1	0	1	1	1	1	1
0111	1	1	1	0	0	0	0
1000	1	1	1	1	1	1	1
1001	1	1	1	1	0	1	1
1010	0	0	0	0	0	0	0
1011	0	0	0	0	0	0	0
1100	0	0	0	0	0	0	0
1101	0	0	0	0	0	0	0
1110	0	0	0	0	0	0	0
1111	0	0	0	0	0	0	0



Decimal	Segments
0	ABCDEF
1	BC
2	ABDEG
3	ABCDG
4	BCFG
5	ACDFG
6	ACDEFG
7	ABC
8	ABCDEFG
9	ABCDG



Lista de Exercícios na Mão

Circuitos Combinacionais

1. Alarme de Porta Aberta (Sistema de Segurança)

- Um sistema de alarme deve acionar uma sirene (S) **se pelo menos uma das duas portas (P1, P2) estiver aberta** e o alarme estiver ativado (A).

2. Controle de Ventilador de Geladeira

- Um ventilador deve ligar (V=1) **quando a temperatura for alta (T=1) ou quando a porta estiver aberta (P=1), mas não se o modo econômico estiver ativo (E=1).**

3. Semáforo para Pedestres

- Um semáforo para pedestres deve acender a luz verde (G=1) quando:
- O botão de pedestre (B) for pressionado **e**
- O sinal de carros estiver vermelho (C=1).

4. Multiplexador de 3 Entradas

- Projete um circuito **multiplexador de 3 entradas** que seleciona um dos sinais I₀, I₁, I₂ para a saída Y, usando 2 linhas de seleção (S₀, S₁).
- 👉 Observação: como o multiplexador clássico é 2ⁿ entradas, aqui temos **3 entradas**, então usaremos as combinações 00, 01, 10. O caso 11 pode ser tratado como **saída 0 (desligada)**.

Sequenciais

1. FSM Moore (Detector de Sequência 101)

- Projete uma **FSM Moore** que detecta a sequência de bits 101 em uma entrada serial X. A saída Z deve ser **1** apenas quando a sequência foi reconhecida.

2. FSM Mealy (Detector de Sequência 101)

- Projete uma **FSM Mealy** que detecta a mesma sequência 101, mas agora a saída Z deve ser **1 imediatamente** ao reconhecer o último bit, sem esperar mudar de estado.

3. Contador Síncrono Módulo-4 (00 → 01 → 10 → 11 → 00)

- Projete um **contador síncrono** de 2 bits (Q1, Q0) que conta em binário crescente de 0 a 3.

Como Resolver Problemas Combinacionais

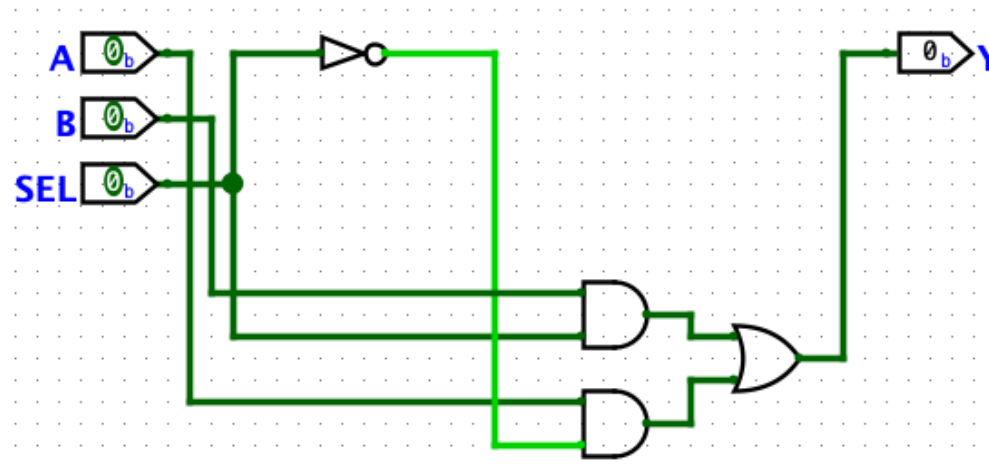
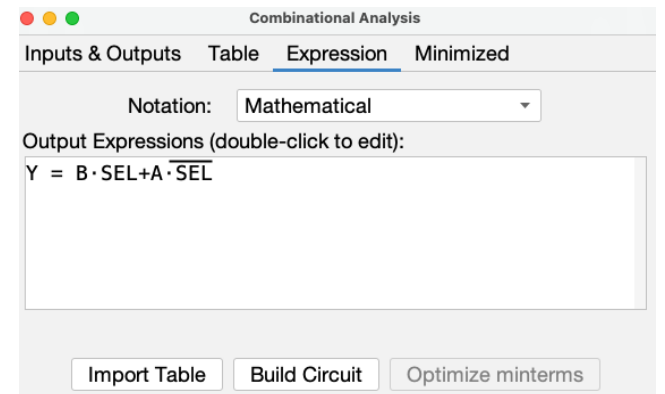
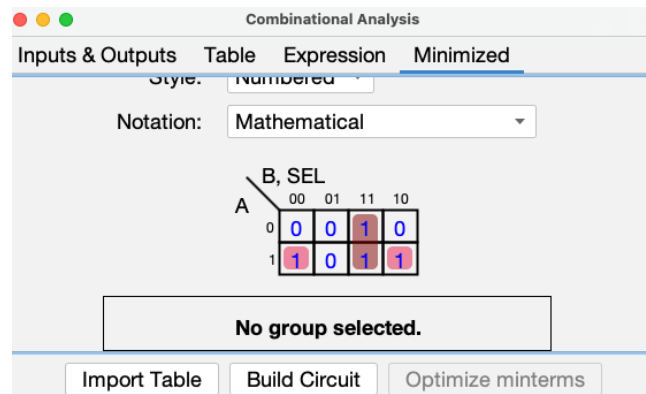
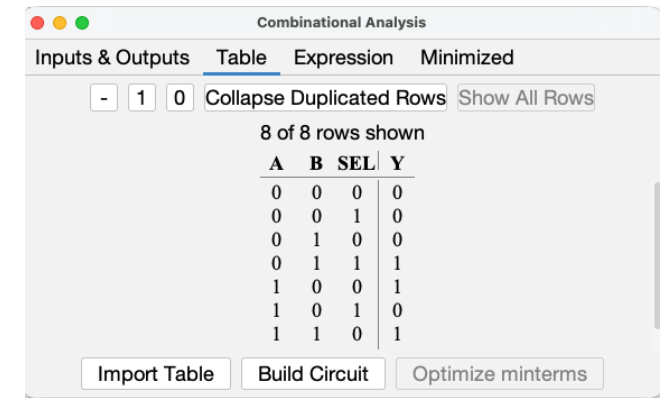
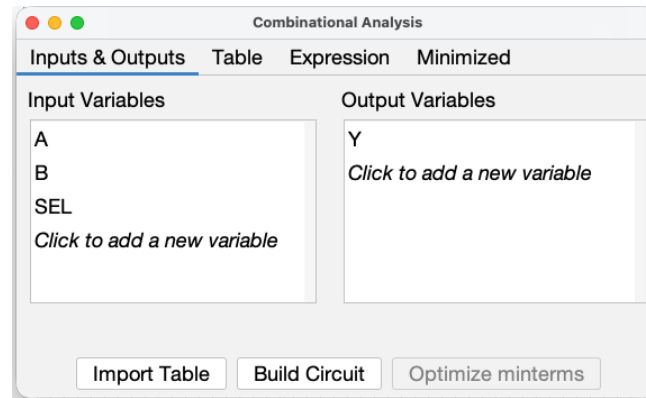
1. Identifique as variáveis de entrada
2. Identifique as variáveis de saída
3. Crie a tabela verdade e selecione os casos verdadeiros
4. Monte o Mapa de Karnaugh para obter a equação lógica simplificada
5. Substitua a equação (SOMA de produtos) em um circuito

Se você usar o Logisim Evolution, você pode pular a etapa 4 e 5

Exemplo MUX 1-bit A,B

Entradas: A, B, S

Saídas: Y

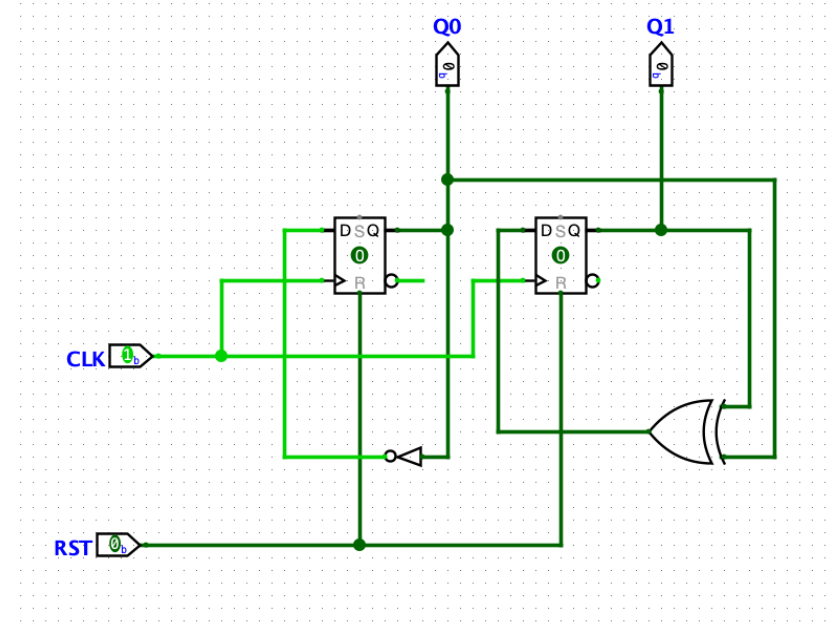
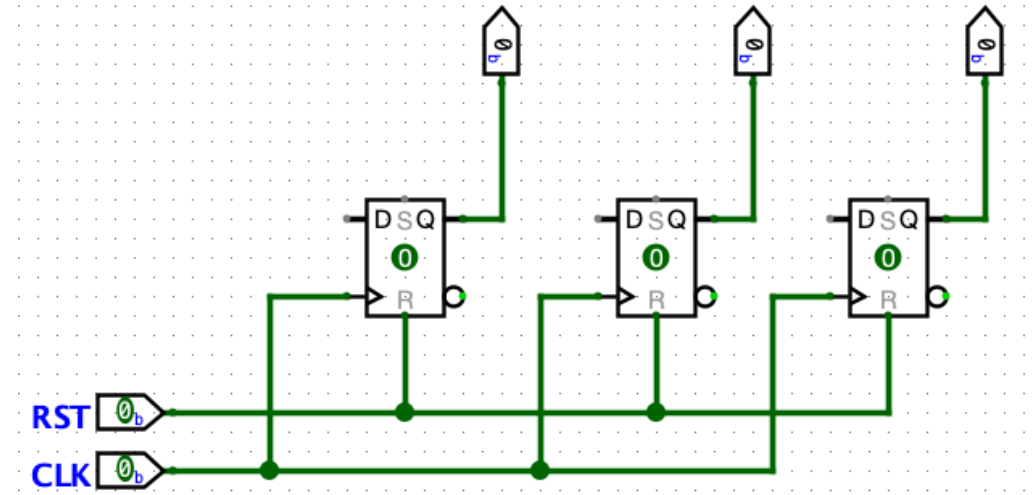


Passos para resolver problemas de Contadores

1. Considere a arquitetura Moore (Huffman Topology)
2. Escolha o numero de flp-flops para segurar a contagem (por exemplo 4 números precisam de 2 bits, ou 2 flip-flops)
3. As saídas dos flip-flops serão a contagem
4. So precisamos criar e simplificar a logica do próximo estado, por exemplo
 1. Quando o estado for 00, o próximo tem que ser 01
 2. Quando o estado for 01 o próximo tem que ser 10
 3. Quando o estado for 10 o próximo vai ser 11Logo nesse caso $D0 = \text{NOT } Q0$, e $D1 = Q0 \text{ XOR } Q1$

Lembrando que para circuitos síncronos e usando o flip-flop D a conexão dos elementos de memoria são os mesmos.

Veja a seguir o contador 0 a 3 (00, 01, 10, 11) síncrono com flip-flop Ds

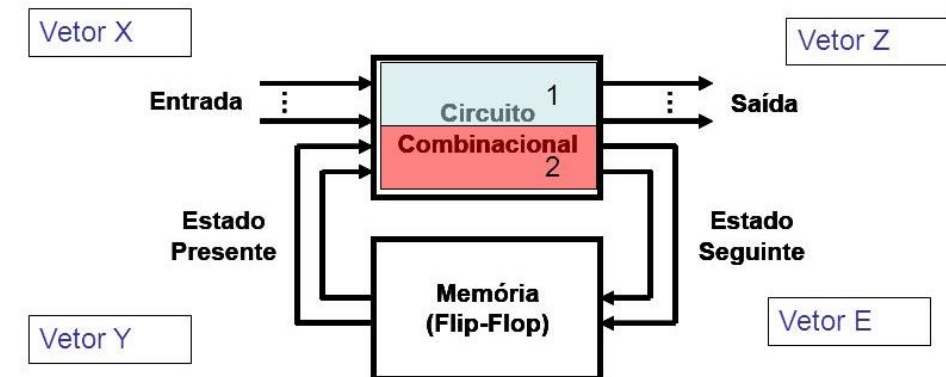
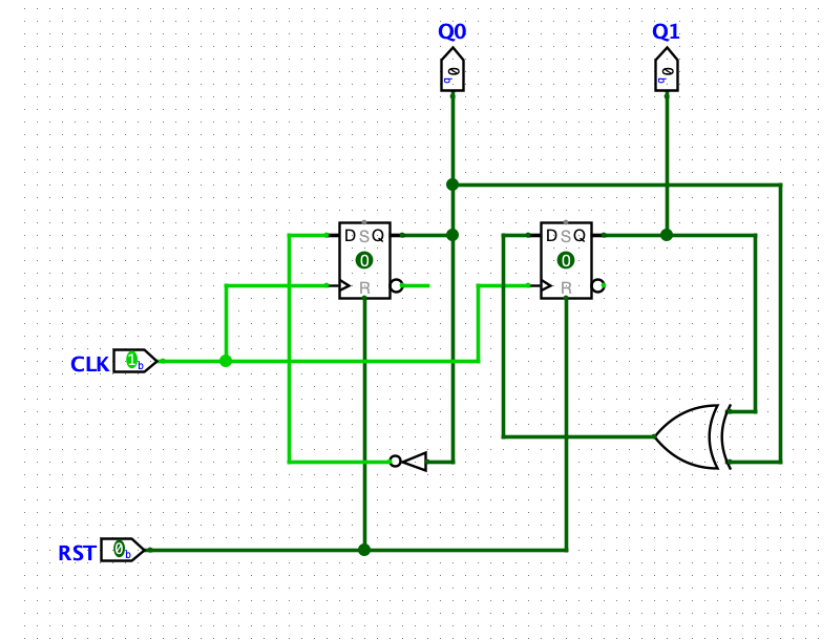


Passos para Resolver Problemas de FSM (Moore / Mealy)

1. Escolha a Arquitetura Moore ou Mealy
2. Determinar o diagrama de transição de estados
3. Codificar os estados em forma binária
4. Escolher o Flip-Flop (Normalmente o D)
5. Simplificar a parte combinacional para o próximo estado
6. Simplificar a parte combinacional para saída

Lembrando que para circuitos síncronos e usando o flip-flop D a conexão dos elementos de memória são os mesmos.

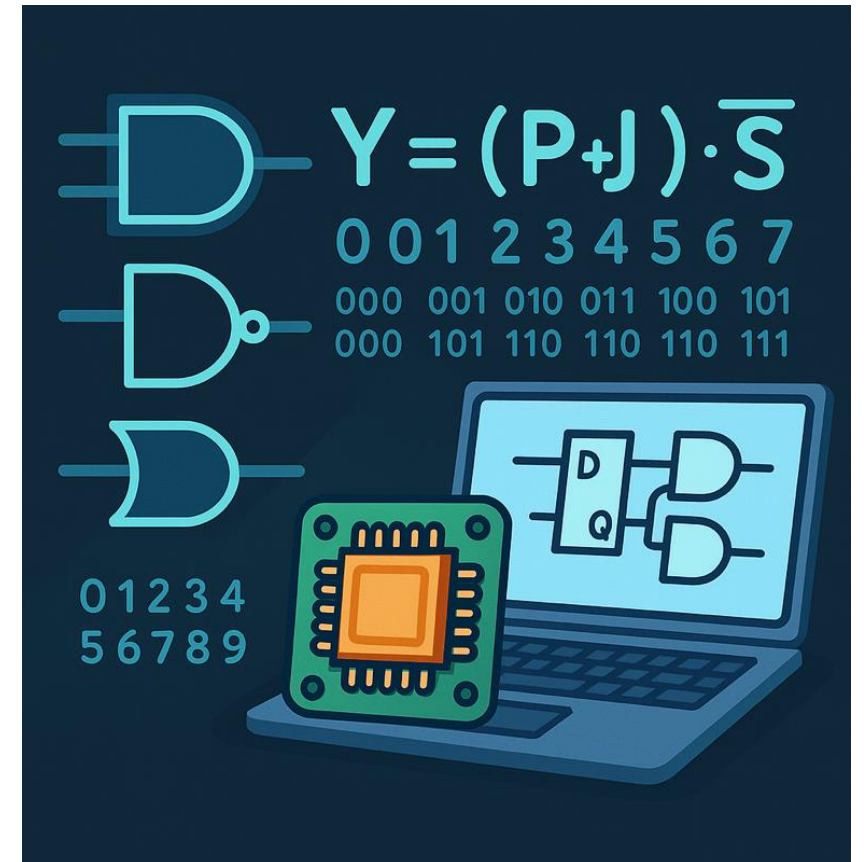
Veja a seguir o contador 0 a 3 (00, 01, 10, 11) síncrono com flip-flop Ds



Fim da Parte 2

A partir deste ponto nos já sabemos:

- Circuitos Combinacionais e Sequenciais
- Logica e numeração booleana
- Simplificação de equações Booleanas
- Usar ferramentas Logisim Evolution para simular/projetar circuitos digitais.



VHDL

Agora o projeto de circuitos eletrônicos, começa a virar “programação”, iremos aprender a linguagem de descrição de hardware chamado VHDL, usada para **modelar, simular e sintetizar** circuitos digitais.

Aplicações:

- Projetar sistemas digitais em **FPGAs e ASICs**
- Testar e simular circuitos antes da fabricação

Estilos de descrição

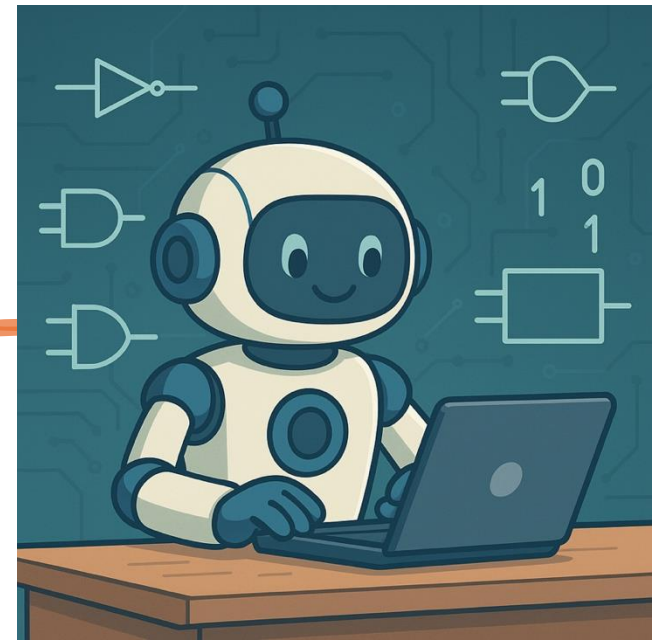
Podemos escrever o código VHDL de diversas formas, com cada uma tendo vantagens para determinado tipo ou tamanho de circuito.

- **Structural** → descreve interconexões entre componentes. (Usado com projetos com sub-módulos)
- **Behavioral** → descreve o comportamento lógico (mais abstrato). (podemos usar if/else ou case) porém o sintetizador decide que circuito vai ser criado.
- **Dataflow** → descreve o fluxo de sinais por equações lógicas.

O Código VHDL não é compilado, ele é sintetizado ou seja o compilador irá criar um circuito que implementa a descrição, ou seja o código VHDL não é interpretado, a não ser em Testes (Testbench).

Testbenches:

São “testes unitários” que exercitam o circuito que você quer implementar usando outro código VHDL (testbench), esse sim se comporta como um programa normal, interpretado pelo computador.





Sintaxe Básica do VHDL

◆ Estrutura Geral

- Inclusão de bibliotecas
- Definição da entidade, portas, direção das portas e tipos, generics (constantes)
- Definição da arquitetura

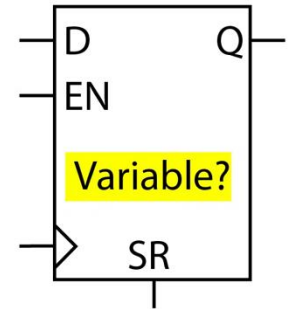
◆ Tipos de dados comuns

- **STD_LOGIC** → um único bit ('0', '1', 'Z', etc.)
- **STD_LOGIC_VECTOR(n downto 0)** → vetor de bits
- **INTEGER** → usado em simulação e contadores
- **BOOLEAN** → verdadeiro/falso

◆ Estruturas de descrição

- **Concurrent** (paralelas): Executa sempre que as entradas mudarem de valor
- **Sequential** (em process): Executa passo a passo, usado pelo sintetizador para “descobrir” que circuito implementa o algoritmo no process.

```
1 process(clk)
2   variable my_var : std_logic;
3 begin
4   if rising_edge(clk) then
5
6     if some_condition then
7       my_var := some_value;
8     end if;
9
10    my_signal <= my_var;
11
12  end if;
13 end process;
```



EXAMPLE 2. Write the VHDL code to implement the function expressed by the following logic equation: $F3 = \overline{L} \cdot \overline{M} \cdot N + L \cdot M$

SOLUTION. The black box diagram and associated VHDL code is shown in Listing 4.7.

Listing 4.7: Solution of Example 2.

```
-- library declaration
library IEEE;
use IEEE.std_logic_1164.all;
-- entity
entity my_ckt_f3 is
  port ( L,M,N : in  std_logic;
        F3    : out std_logic);
end my_ckt_f3;
-- architecture
architecture f3_2 of my_ckt_f3 is
begin
  F3<= ((NOT L) AND (NOT M) AND N) OR (L AND M);
end f3_2;
```



Caso de uso do Behavior/Process

Usamos o modo Behaviour para descrever maquinas de estado, FSM ou contadores, Decisões e Controle, e geração de estímulos no testbench.

Cuidados ao usar

- Sempre listar sinais no **sensitivity list** (ex.: process(clk, rst)), ou usar process(all) em VHDL-2008.
- Em lógica sequencial:
- Usar **clock + reset** corretamente.
- Atribuir valores a todos os sinais para evitar **latches indesejados**.
- Em lógica combinacional:
 - Garantir que todas as condições sejam tratadas (else, others).
 - Evitar lógica muito complexa em um único process → dificulta síntese e leitura.

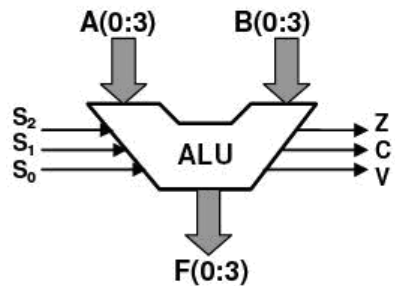
Quando usar Behavioral

- ✓ FSMs (controle de protocolos, automação)
- ✓ Registradores e contadores
- ✓ Controle de sinais complexos (multiplexadores grandes, decodificadores)
- ✓ Testbench e simulação de hardware

 Para lógica simples combinacional (ex.: portas, somadores), prefira **Dataflow** ou **Structural**.

Exercício 1

Criar Código para ALU de 4 bits



Z, C and V are status flags

Z = 1 if F=0

C = Carry or Borrow

V = Overflow

S ₂	S ₁	S ₀	Function (F)
0	0	0	A+B
0	0	1	A-B
0	1	0	A-1
0	1	1	A+1
1	0	0	$A \wedge B$
1	0	1	$A \vee B$
1	1	0	NOT A
1	1	1	$A \oplus B$

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity alu4 is
    port (
        A, B : in  std_logic_vector(3 downto 0);
        S    : in  std_logic_vector(2 downto 0);
        F    : out std_logic_vector(3 downto 0);
        Z, C, V : out std_logic
    );
end alu4;

architecture beh of alu4 is
    signal As, Bs, Fs : signed(3 downto 0);
begin
    As <= signed(A); Bs <= signed(B);

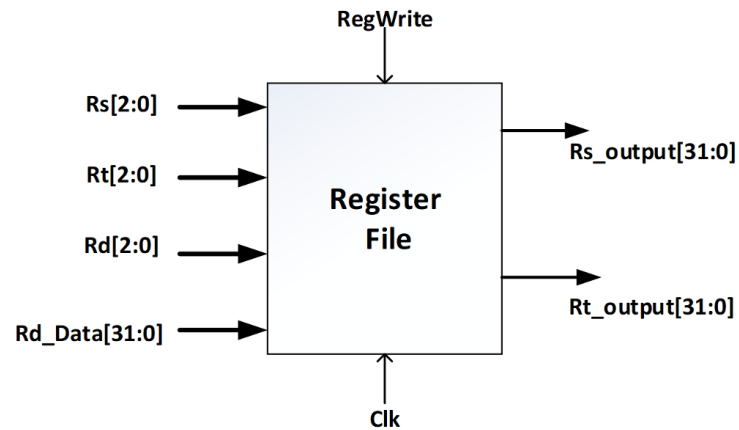
    process(As, Bs, S)
        variable T : signed(4 downto 0);
    begin
        case S is
            when "000" => T := resize(As,5)+resize(Bs,5); -- A+B
            when "001" => T := resize(As,5)-resize(Bs,5); -- A-B
            when "010" => T := resize(As,5)-1; -- A-1
            when "011" => T := resize(As,5)+1; -- A+1
            when "100" => Fs <= As and Bs; T := (others=>'0');
            when "101" => Fs <= As or Bs; T := (others=>'0');
            when "110" => Fs <= not As; T := (others=>'0');
            when "111" => Fs <= As xor Bs; T := (others=>'0');
            when others => Fs <= (others=>'0'); T := (others=>'0');
        end case;

        if S <= "011" then
            Fs <= T(3 downto 0);
            C <= T(4);
            V <= (As(3)=Bs(3) and Fs(3)/=As(3)) when S="000" else
                (As(3)/=Bs(3) and Fs(3)/=As(3)) when S="001" else
                '0';
        else
            C <= '0'; V <= '0';
        end if;
    end process;

    F <= std_logic_vector(Fs);
    Z <= '1' when Fs = 0 else '0';
end beh;
```

Register File

Criar código VHDL para Register File, para 8 registradores de 32 bits.



```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity RegisterFile is
    port (
        Rs, Rt, Rd : in  std_logic_vector(2 downto 0);    -- endereço 0-7
        Rd_Data    : in  std_logic_vector(31 downto 0);    -- write data
        RegWrite    : in  std_logic;                      -- write enable
        Clk         : in  std_logic;

        Rs_output   : out std_logic_vector(31 downto 0);
        Rt_output   : out std_logic_vector(31 downto 0)
    );
end entity;

architecture Behavioral of RegisterFile is
    type reg_array is array(0 to 7) of std_logic_vector(31 downto 0);
    signal regs : reg_array := (others => (others => '0'));
begin

    -----
    -- Escrita síncrona
    -----

    process(Clk)
    begin
        if rising_edge(Clk) then
            if RegWrite = '1' then
                regs(to_integer(unsigned(Rd))) <= Rd_Data;
            end if;
        end if;
    end process;

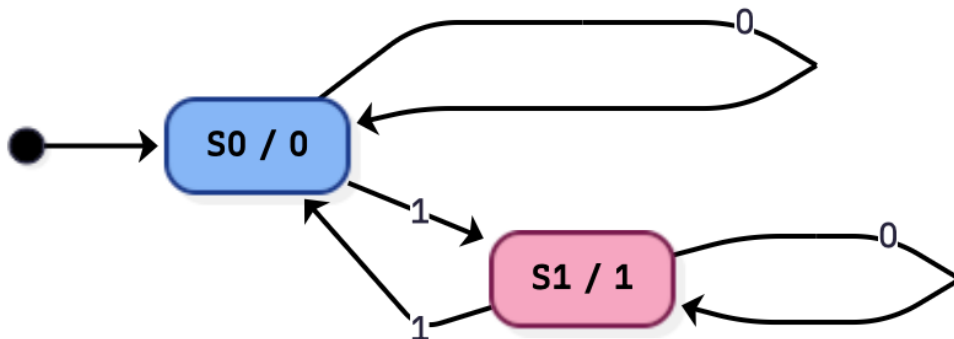
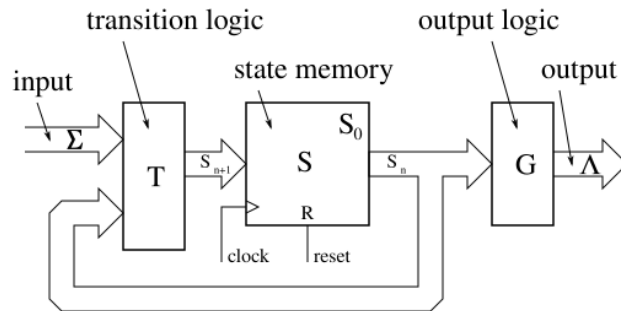
    -----
    -- Leitura assíncrona (combinacional)
    -----

    Rs_output <= regs(to_integer(unsigned(Rs)));
    Rt_output <= regs(to_integer(unsigned(Rt)));

end architecture Behavioral;
```

Maquina de Estado Moore

Implemente essa maquina de estados FSM Moore



```

library ieee;
use ieee.std_logic_1164.all;

entity moore_fsm is
  port(
    clk, reset : in  std_logic;
    x          : in  std_logic;
    y          : out std_logic
  );
end;

architecture beh of moore_fsm is
  type state_t is (S0, S1);
  signal state, next : state_t;
begin

  -- Estado (registrador)
  process(clk, reset) begin
    if reset = '1' then
      state <= S0;
    elsif rising_edge(clk) then
      state <= next;
    end if;
  end process;

  -- Lógica de próxima transição
  process(state, x) begin
    case state is
      when S0 =>
        if x = '1' then next <= S1;
        else             next <= S0;
        end if;

      when S1 =>
        if x = '1' then next <= S0;
        else             next <= S1;
        end if;
    end case;
  end process;

  -- Saída Moore: depende só do estado
  y <= '0' when state = S0 else '1';

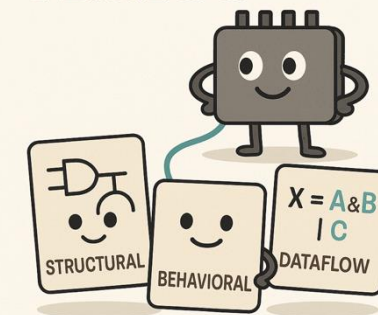
end beh;
  
```

Verilog

Implementar uma função lógica simples:

$$Y = (A \cdot B) + (\bar{A} \cdot C)$$

VERILOG



O Verilog é uma linguagem de descrição de hardware (HDL), usada para modelar, simular e sintetizar circuitos digitais.

Aplicações:

- Projetar sistemas digitais em **FPGAs** e **ASICs**
- Testar e simular circuitos antes da fabricação

Estilos de descrição:

- **Structural** → descreve conexões entre módulos e portas lógicas
- **Behavioral** → descreve o comportamento com construções similares a linguagens de programação (if, case, loops)
- **Dataflow** → descreve o fluxo de sinais usando expressões booleanas e equações

1. Structural (estrutural)

Aqui conectamos explicitamente portas lógicas primitivas (and, or, not).

```
verilog

module func_structural (
    input A, B, C,
    output Y
);

    wire notA, and1, and2;

    not U1 (notA, A);      // inversor
    and U2 (and1, A, B);   // AND entre A e B
    and U3 (and2, notA, C); // AND entre ~A e C
    or U4 (Y, and1, and2); // OR final

endmodule
```

2. Behavioral (comportamental)

Usando bloco always e instruções condicionais.

```
verilog

module func_behavioral (
    input A, B, C,
    output reg Y
);

    always @(*) begin
        if ((A & B) | ((~A) & C))
            Y = 1'b1;
        else
            Y = 1'b0;
        end

endmodule
```

3. Dataflow (fluxo de dados)

Usando atribuição contínua (assign) com expressões lógicas.

```
verilog

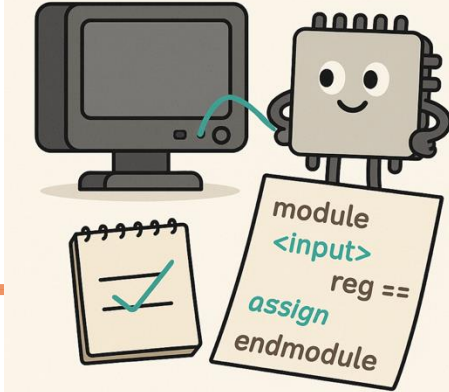
module func_dataflow (
    input A, B, C,
    output Y
);

    assign Y = (A & B) | ((~A) & C);

endmodule
```




Sintaxe Básica do Verilog



◆ Estrutura Geral do Código

- Inclusão de módulos (module ... endmodule)
- Definição de **entradas (input)**, **saídas (output)** e **registradores (reg)**
- Definição da lógica dentro do módulo

◆ Tipos de Dados mais comuns

- wire → usado para representar conexões físicas (combinacional)
- reg → armazena valores (necessário em lógica sequencial)
- integer → usado em simulação e contadores
- parameter → constantes e parâmetros genéricos

◆ Descrição

- **Concorrente** → assign (ligações contínuas)
- **Sequencial** → always @(...) (processos ativados por eventos, como clock/reset)

// Exemplo simples de Verilog

```
module exemplo_basico (  
    input wire A, B, clk, rst,      // entradas  
    output wire Y,                  // saída combinacional  
    output reg Q                    // saída sequencial (flip-flop)  
);  
  
    // Constante (parâmetro genérico)  
    parameter N = 4;  
  
    // Conexão combinacional  
    wire and_out;  
  
    // Descrição concorrente (executa sempre que entradas mudam)  
    assign and_out = A & B;          // operação AND  
    assign Y = ~and_out;            // saída Y é o NOT de (A AND B)  
  
    // Descrição sequencial (ativada por borda do clock/reset)  
    always @(posedge clk or posedge rst) begin  
        if (rst)                    // reset assíncrono  
            Q <= 0;  
        else  
            Q <= Y;                  // registra valor de Y  
        end  
    endmodule
```

Boas práticas e usos

Cuidados ao usar Verilog

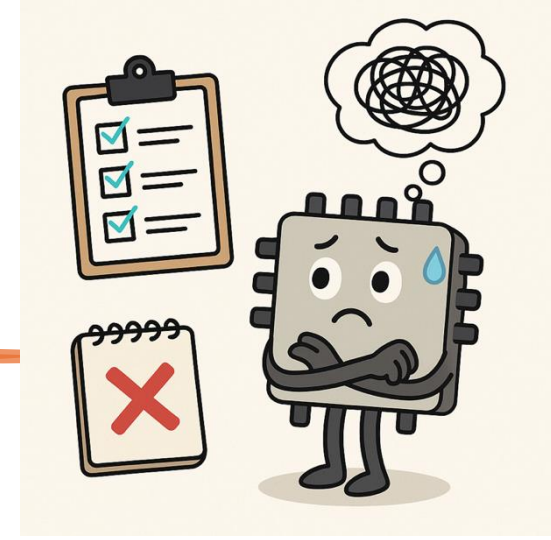
- Usar corretamente **sensibilidade** em blocos always (always @(posedge clk or posedge rst))
- Diferenciar lógica **combinacional** (always @(*)) de lógica **sequencial** (always @(posedge clk))
- Evitar condições incompletas → podem gerar **latches indesejados**
- Inicializações em simulação não equivalem a inicializações em hardware real

✓ Quando usar Behavioral

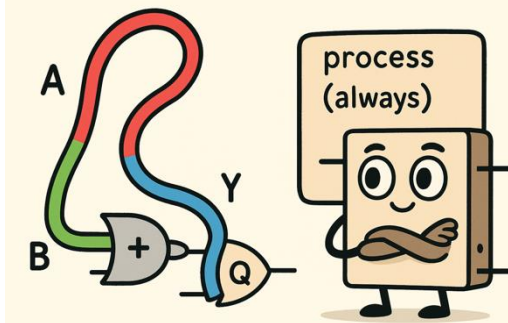
- Máquinas de estado (FSMs)
- Contadores e registradores
- Controle de sinais complexos (ex.: multiplexadores grandes)
- Criação de testbenches para simulação

✗ Quando NÃO usar Behavioral

- Para lógica combinacional simples (somadores, portas lógicas), prefira **Dataflow** ou **Structural**



Wire/Reg vs Signal/Variable



Signal(VHDL) / Wire (Verilog)

- Representa uma **linha de conexão** entre processos ou dentro da arquitetura.
- Atualização é **agendada** (ocorre no final do ciclo de simulação delta).
- Usado em **atribuições concorrentes** ou em processos.

Já a variable(VHDL) e reg(Verilog) podem parecer iguais pois só podem ser usadas dentro de um bloco always (Verilog) ou process (VHDL).

◆ Variable no VHDL

- Existe **apenas dentro de processos, funções ou procedures**.
- Atualização é **imediata** (o valor muda na hora dentro do processo).
- Não é visível fora do processo.
- Útil para **cálculos temporários ou intermediários**.

◆ Reg no Verilog

- É um tipo de **objeto de armazenamento** usado em **blocos procedurais** (always, initial).
- Pode guardar valor **entre ativações** do bloco.
- **Não é igual ao variable do VHDL**, porque pode representar tanto:
- **Flip-flop (registrador real)** → se estiver em always @(posedge clk)
- **Combinacional temporário** → se estiver em always @(*)

📌 Conclusão

- variable (VHDL) e reg (Verilog) **se parecem** porque ambos são usados em blocos procedurais, **mas não são equivalentes**.
- Já wire e signal

vhdl

```
process(a, b)
  variable temp : std_logic;
begin
  temp := a AND b; -- atualização imediata
  y <= temp;       -- sinal recebe no fim do ciclo
end process;
```

verilog

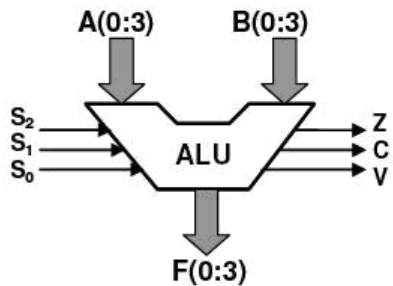
```
reg q;
always @(posedge clk) begin
  q <= d; // flip-flop (armazenamento real)
end
```

verilog

```
reg y;
always @(*) begin
  y = a & b; // combinacional (sem flip-flop)
end
```

Exercício 1

Criar Código para ALU de 4 bits



Z, C and V are status flags

Z = 1 if F=0

C = Carry or Borrow

V = Overflow

S ₂	S ₁	S ₀	Function (F)
0	0	0	A+B
0	0	1	A-B
0	1	0	A-1
0	1	1	A+1
1	0	0	$A \wedge B$
1	0	1	$A \vee B$
1	1	0	NOT A
1	1	1	$A \oplus B$

```
// ALU 4-bit com flags Z (zero), C (carry/borrow) e V (overflow)
// Funções segundo S2 S1 S0:
// 000: A+B   | 001: A-B   | 010: A-1   | 011: A+1
// 100: A&B   | 101: A|B   | 110: ~A    | 111: A^B

module alu4 (
    input  [3:0] A, B,           // operandos
    input  [2:0] S,              // seletor da operação
    output reg [3:0] F,          // resultado
    output reg Z, C, V           // flags
);

// Pré-computações aritméticas (5 bits = inclui carry)
wire [4:0] add_ab = {1'b0,A} + {1'b0,B};
wire [4:0] sub_ab = {1'b0,A} - {1'b0,B};
wire [4:0] inc_a  = {1'b0,A} + 5'b00001;
wire [4:0] dec_a  = {1'b0,A} - 5'b00001;

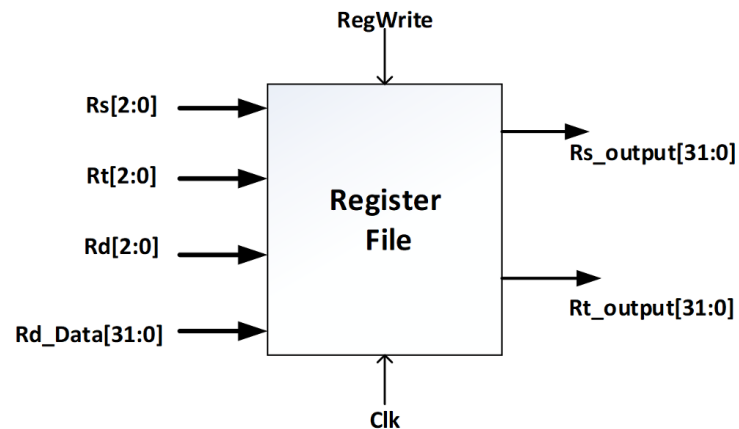
always @* begin
    // Valores padrão
    F = 4'b0;
    C = 1'b0;
    V = 1'b0;

    case (S)
        3'b000: begin                // A + B
            {C,F} = add_ab;
            V = (A[3]==B[3]) && (F[3]!=A[3]);
        end
        3'b001: begin                // A - B
            {C,F} = sub_ab;
            V = (A[3]!=B[3]) && (F[3]!=A[3]);
        end
        3'b010: begin                // A - 1
            {C,F} = dec_a;
            V = A[3] & ~F[3];
        end
        3'b011: begin                // A + 1
            {C,F} = inc_a;
            V = ~A[3] & F[3];
        end
        3'b100: F = A & B;            // AND
        3'b101: F = A | B;            // OR
        3'b110: F = ~A;               // NOT
        3'b111: F = A ^ B;            // XOR
    endcase

    // Flag Z = 1 se resultado for zero
    Z = (F == 4'b0000);
end
endmodule
```

Register File

Criar código VHDL para Register File, para 8 registradores de 32 bits.



```
// Register File 8x32
// - 8 registradores de 32 bits
// - 2 portas de leitura (Rs, Rt)
// - 1 porta de escrita (Rd)
// - Escrita síncrona, leitura combinacional

module register_file (
    input wire      clk,
    input wire      RegWrite,           // habilita escrita

    input wire [2:0] Rs, Rt, Rd,       // endereços 0..7
    input wire [31:0] Rd_Data,         // dado a escrever

    output wire [31:0] Rs_output,      // leitura porta Rs
    output wire [31:0] Rt_output      // leitura porta Rt
);

// 8 registradores de 32 bits
reg [31:0] regs [0:7];

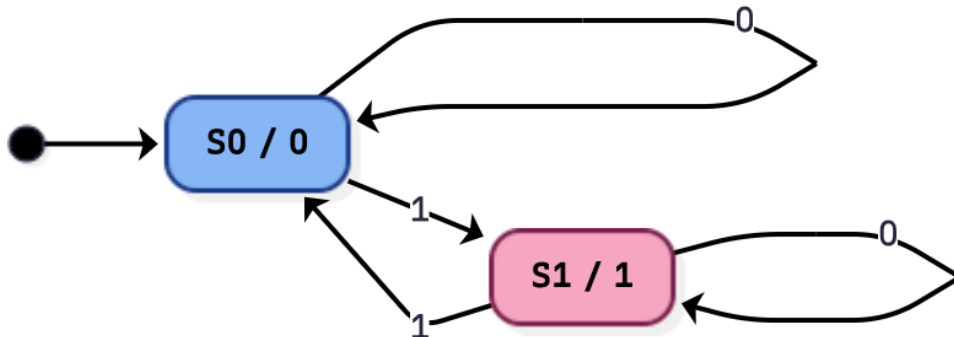
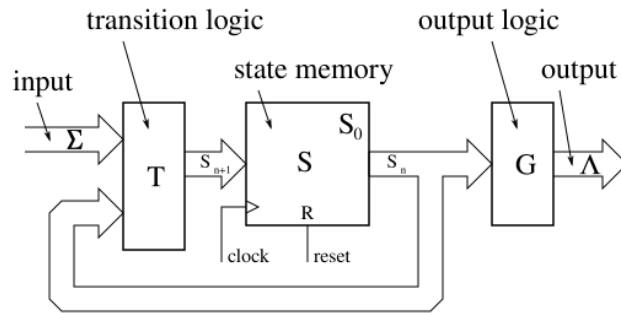
// Escrita síncrona
always @(posedge clk) begin
    if (RegWrite)
        regs[Rd] <= Rd_Data;
end

// Leitura assíncrona (combinacional)
assign Rs_output = regs[Rs];
assign Rt_output = regs[Rt];

endmodule
```


Maquina de Estado Moore

Implemente essa maquina de estados FSM Moore



```
// Moore FSM genérica (exemplo: 2 estados S0/0 e S1/1)
// Entrada  x
// Saída    y (depende só do estado)

module moore_fsm (
    input wire clk,
    input wire reset, // síncrono ou assíncrono, aqui assíncrono
    input wire x,
    output reg y
);

// -----
// 1) Codificação dos estados (fácil de expandir)
// -----
localparam S0 = 1'b0; // saída = 0
localparam S1 = 1'b1; // saída = 1

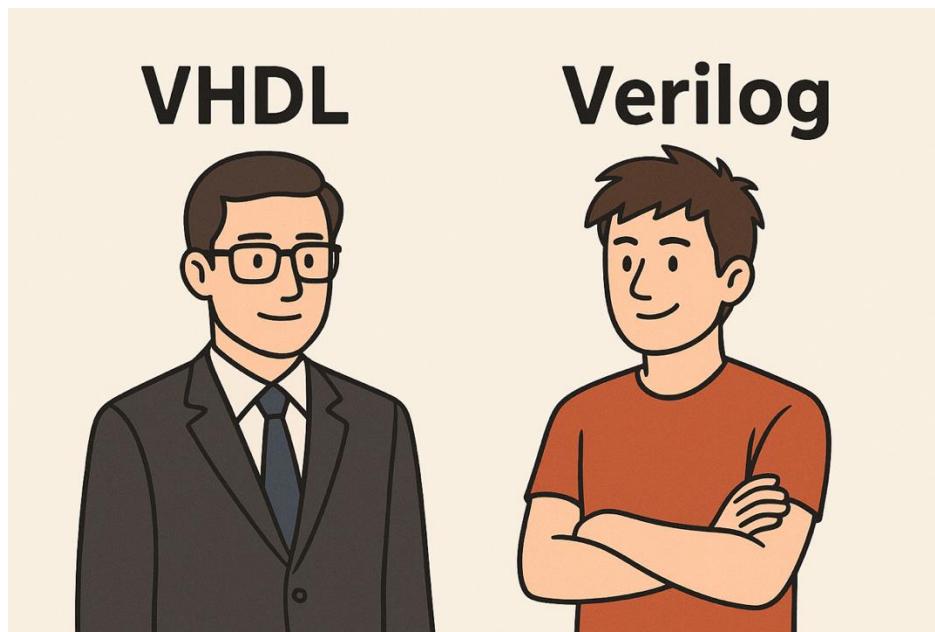
reg state, next_state;

// -----
// 2) Registrador de estado
// -----
always @(posedge clk or posedge reset) begin
    if (reset)
        state <= S0; // estado inicial
    else
        state <= next_state; // próxima transição
end

// -----
// 3) Lógica de próxima transição (depende de estado + entradas)
//    -> aqui implementa o diagrama S0/S1
// -----
always @* begin
    case (state)
        S0: begin
            if (x) next_state = S1; // x=1 vai para S1
            else next_state = S0; // x=0 permanece em S0
        end
        S1: begin
            if (x) next_state = S0; // x=1 volta para S0
            else next_state = S1; // x=0 permanece em S1
        end
        default: next_state = S0; // fallback seguro
    endcase
end

// -----
// 4) Saída Moore (depende só do estado)
// -----
always @* begin
    case (state)
        S0: y = 1'b0;
        S1: y = 1'b1;
        default: y = 1'b0;
    endcase
end

endmodule
```



Aspecto	VHDL	Verilog
Origem	Década de 1980 (ADA → linguagem fortemente tipada, criada pelo DoD/IEEE).	Década de 1980 (inspirada em C, criada pela Gateway Design, depois IEEE).
Estilo	Mais verboso e fortemente tipado (precisa declarar tudo).	Mais conciso e flexível , sintaxe parecida com C.
Aprendizado	Mais rigoroso (força boas práticas, menos erros escondidos).	Mais fácil para iniciantes (menos código para a mesma lógica).
Usos comuns	Muito usado em academia, Europa e projetos de defesa .	Muito usado em indústria, EUA e ASICs comerciais .
Tipos de dados	Ricos (std_logic, boolean, integer, array, records).	Mais simples (wire, reg, integer, parameter).
Estilos de descrição	Structural, Dataflow, Behavioral.	Structural, Dataflow, Behavioral.
Legibilidade	Mais próximo de linguagem de programação fortemente tipada .	Mais próximo de linguagem de programação C .
Ferramentas	Suporte excelente em FPGAs (Altera/Intel, Xilinx, Lattice).	Suporte amplo em ASICs, FPGA e SystemVerilog como padrão moderno.

◆ VHDL vs Verilog

- 📌 **Resumo:**
- **VHDL** → mais rigoroso, ideal para ensino, prototipagem robusta e projetos que exigem segurança.
- **Verilog** → mais prático, mais usado no mercado para ASIC/FPGA, base para **SystemVerilog**.

Limitação da descrição Behavioral



A descrição **behavioral** permite usar construções de **alto nível** (if/else, case, loops, etc.), que se parecem com programação tradicional. Porém, **nem tudo que você escrever é sintetizável** em hardware real.

⚠ Exemplos de limitações:

- **Operações sem correspondência em hardware** → por exemplo, divisão por número arbitrário, chamadas recursivas, delays (#10 no Verilog, wait for 10 ns no VHDL) — funcionam em **simulação**, mas não são sintetizados.
- **Chamadas de sistema** (\$display, \$stop) → apenas para debug/simulação.
- **Laços indefinidos** → while sem limite fixo não pode ser convertido em hardware.
- **Estruturas complexas demais** → podem até ser sintetizáveis, mas o circuito resultante será enorme ou ineficiente.

◆ A síntese sempre cria hardware?

Não. O sintetizador só gera hardware quando:

- O código tem **correspondência direta com lógica digital**.
- Há um mapeamento claro para **portas lógicas, flip-flops, multiplexadores, decodificadores, etc.**

Se o algoritmo for muito abstrato (ex.: “ordenar uma lista”), o sintetizador **não cria automaticamente um processador interno** para executar isso.

Ele tenta expandir em portas lógicas → o que pode resultar em algo gigantesco ou inviável.

✚ Conclusão

- **Behavioral ≠ “qualquer algoritmo”** → só o que tem correspondência com lógica digital.
- **Simulação ≠ Síntese** → no simulador tudo roda como software, mas o hardware gerado pode ser diferente ou até inviável.
- **Melhor prática** → sempre pensar em termos de **circuitos sequenciais (FSMs)** e **blocos combinacionais**.

◆ Exemplo simples não sintetizável

verilog

```
module nao_sintetizavel;

    reg clk;

    initial begin
        clk = 0;
        forever begin
            #10 clk = ~clk; // Delay de 10 unidades de tempo
        end
    end

endmodule
```

◆ Outro exemplo não sintetizável (loop indefinido)

verilog

```
module exemplo_while (
    input [7:0] a,
    output reg [7:0] y
);

    always @(*) begin
        y = 0;
        while (a > y) begin // Loop com limite indefinido
            y = y + 1;
        end
    end

endmodule
```

Tutorial GHDL – Simulação de VHDL

◆ O que é?

- **Simulador open-source de VHDL** (roda no terminal).
- Permite **compilar, simular e depurar** circuitos digitais.
- Integra com **GTKWave** para visualizar formas de onda.

◆ Opções úteis

- `--stop-time=100us` → define tempo da simulação.
- `--vcd=wave.vcd` → gera arquivo VCD (compatível com GTKWave).
- `--wave=wave.ghw` → formato nativo, mais rápido.
- `--assert-level=error` → para simulação em caso de erro.

O Visual Studio tem um plugin visualizador de arquivos VCD (Wavetrace) para verificar os resultados:

Tutorial iVerilog – Simulação de Verilog

◆ O que é?

- Simulador open-source de Verilog/SystemVerilog.
- Permite compilar, simular e depurar projetos digitais.
- Integra com GTKWave (ou qualquer visualizador VCD) para análise de formas de onda.

◆ Opções úteis

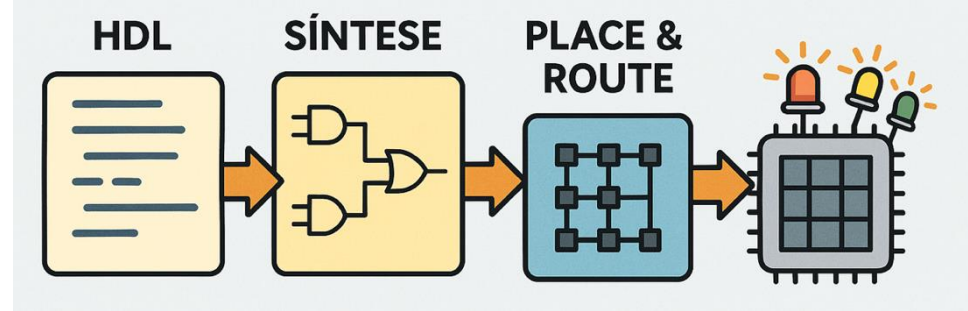
- -g2012 → habilita suporte a SystemVerilog 2012
- -DNOME=VALOR → define macros (útil em parametrização)
- -I<path> → inclui diretórios de headers (.vh)
- -Wall → mostra warnings detalhados

◆ Observações Ondas são geradas pelo testbench usando:

- `$dumpfile("wave.vcd");`
- `$dumpvars(0, tb);`

O Visual Studio Code também possui extensões para abrir VCD (ex.: Wavedrom, Wavetrace).

FPGAs



O que é?

- **FPGA** = chip **reprogramável** contendo milhares ou milhões de blocos lógicos.
- Permite implementar **qualquer circuito digital** sem precisar fabricar um ASIC dedicado.
- Muito usado em **prototipagem**, **sistemas embarcados**, **telecomunicações**, **processamento de sinais** e **Inteligência Artificial**.

Estrutura Básica

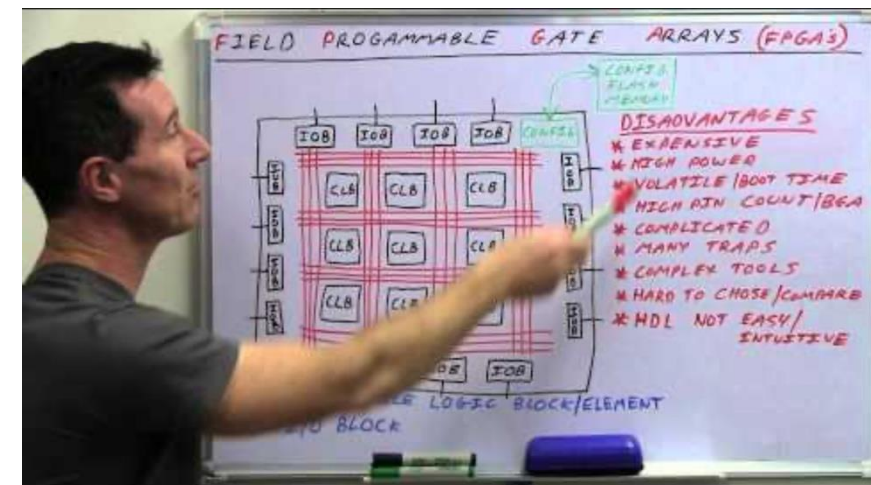
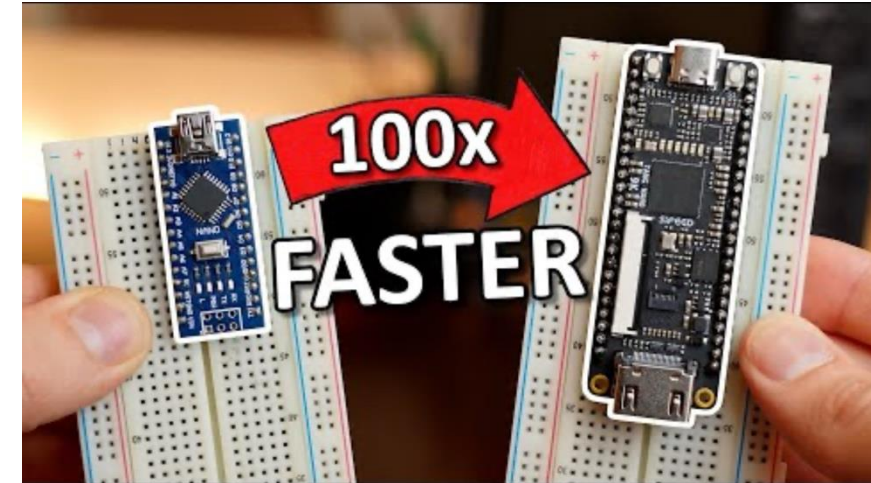
- **LUTs (Look-Up Tables)** → realizam lógica combinacional.
- **Flip-Flops** → armazenam bits e permitem lógica sequencial.
- **DSPs e BRAMs** → blocos dedicados de multiplicação e memória interna.
- **Roteamento programável** → interconecta todos os elementos.

Ferramentas de Desenvolvimento

- **Comerciais** → Xilinx Vivado, Intel Quartus, Lattice Radiant.
- **Open Source** → Yosys + nextpnr + OpenFPGA.

📌 Resumo

- O fluxo de projeto em FPGA segue:
- **HDL → Simulação → Síntese → Place & Route → Bitstream → FPGA real.**



SystemVerilog



◆ O que é?

- É uma **extensão do Verilog** (IEEE 1800), criada para unificar **descrição de hardware (RTL)** e **verificação**.
- Muito usado hoje em ASICs e FPGAs porque adiciona recursos de **linguagem moderna**.
- É compatível com Verilog: todo código Verilog é válido em SystemVerilog.

◆ O que ele tem de melhor em relação ao Verilog

- ☒ **logic** unifica wire e reg → menos confusão.
- ☒ **always_ff, always_comb, always_latch** → blocos mais claros e seguros.
- ☒ **Enums, structs, typedefs** → código mais legível e modular.
- ☒ **Interfaces** → agrupam sinais complexos (como barramentos).
- ☒ **Orientação a Objetos (classes)** → testbenches poderosos.
- ☒ **Randomização e constraints** → verificação automática e geração de cenários.
- ☒ **Assertions (SVA)** → checagem formal de propriedades dentro do código.

📌 Resumo:

- **Verilog** → bom para descrição RTL, simples, conciso.
- **SystemVerilog** → traz **clareza na modelagem + recursos de software moderno** para **verificação e validação**. Hoje é o padrão na indústria de ASICs/SoCs.
- Verilator e a ferramenta OpenSource para simulação com SystemVerilog (porem não suporta a UVM)

systemverilog

```
module exemplo (  
    input logic a, b, clk,  
    output logic y  
);  
  
    always_ff @(posedge clk) begin  
        y <= a & b; // uso de always_ff (sequencial)  
    end  
  
endmodule
```

👉 logic substitui wire/reg → simplifica e evita confusão.

👉 always_ff, always_comb, always_latch → deixam claro o tipo de bloco.

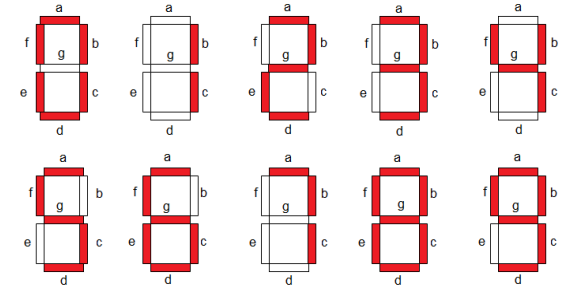
5. Testbench avançado (Classes, Randomização)

systemverilog

```
class Pacote;  
    rand bit [7:0] endereco;  
    rand bit [7:0] dado;  
  
    constraint addr_limit { endereco < 128; } // restrição  
endclass  
  
initial begin  
    Pacote p = new();  
    if (p.randomize())  
        $display("Endereço=%0d, Dado=%0d", p.endereco, p.dado);  
end
```

👉 Recursos de OOP + randomização → fundamentais para UVM (Universal Verification Methodology).

Display de 7 Segmentos (HDL)



Agora iremos implementar em Verilog e VHDL o circuito combinacional para controlar o display de 7 segmentos.

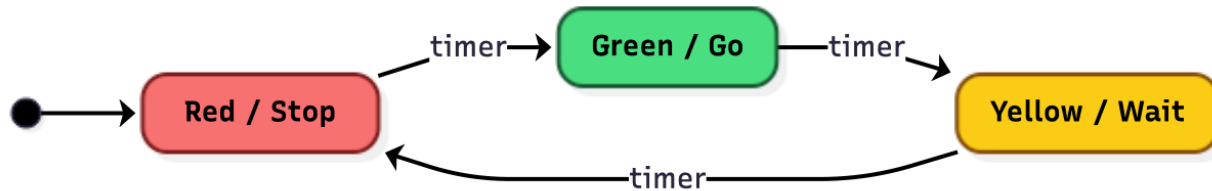
16 of 16 rows shown							
D0[3..0]	A	B	C	D	E	F	G
0 0 0 0	1	1	1	1	1	1	0
0 0 0 1	0	1	1	0	0	0	0
0 0 1 0	1	1	0	1	1	0	1
0 0 1 1	1	1	1	1	0	0	1
0 1 0 0	0	1	1	0	0	1	1
0 1 0 1	1	0	1	1	0	1	1
0 1 1 0	1	0	1	1	1	1	1
0 1 1 1	1	1	1	0	0	0	0
1 0 0 0	1	1	1	1	1	1	1
1 0 0 1	1	1	1	1	0	1	1
1 0 1 0	0	0	0	0	0	0	0
1 0 1 1	0	0	0	0	0	0	0
1 1 0 0	0	0	0	0	0	0	0
1 1 0 1	0	0	0	0	0	0	0
1 1 1 0	0	0	0	0	0	0	0
1 1 1 1	0	0	0	0	0	0	0

```
verilog > ≡ bcd7seg_simple.v
1 // Decoder simples BCD -> Display 7 segmentos
2 // Saída ativa em nível alto (cátodo comum)
3 // Se for ânodo comum, basta inverter com assign seg = ~seg;
4 module bcd7seg_simple (
5     input wire [3:0] bcd, // entrada: número em binário (0..9)
6     output reg [6:0] seg // saída: {a,b,c,d,e,f,g}
7 );
8
9     always @(*) begin
10         case (bcd)
11             4'd0: seg = 7'b1111110; // 0
12             4'd1: seg = 7'b0110000; // 1
13             4'd2: seg = 7'b1101101; // 2
14             4'd3: seg = 7'b1111001; // 3
15             4'd4: seg = 7'b0110011; // 4
16             4'd5: seg = 7'b1011011; // 5
17             4'd6: seg = 7'b1011111; // 6
18             4'd7: seg = 7'b1110000; // 7
19             4'd8: seg = 7'b1111111; // 8
20             4'd9: seg = 7'b1111011; // 9
21             default: seg = 7'b0000000; // apaga (para entradas 10..15)
22         endcase
23     end
24 endmodule
```

```
vhdl > ≡ bcd7seg_simple.vhd
1 -- bcd7seg_simple.vhd
2 library ieee;
3 use ieee.std_logic_1164.all;
4
5 entity bcd7seg_simple is
6     port (
7         bcd : in std_logic_vector(3 downto 0); -- 0..9 (BCD)
8         seg : out std_logic_vector(6 downto 0) -- {a,b,c,d,e,f,g}
9     );
10 end entity;
11
12 architecture rtl of bcd7seg_simple is
13 begin
14     process(bcd)
15     begin
16         case bcd is
17             when "0000" => seg <= "1111110"; -- 0
18             when "0001" => seg <= "0110000"; -- 1
19             when "0010" => seg <= "1101101"; -- 2
20             when "0011" => seg <= "1111001"; -- 3
21             when "0100" => seg <= "0110011"; -- 4
22             when "0101" => seg <= "1011011"; -- 5
23             when "0110" => seg <= "1011111"; -- 6
24             when "0111" => seg <= "1110000"; -- 7
25             when "1000" => seg <= "1111111"; -- 8
26             when "1001" => seg <= "1111011"; -- 9
27             when others => seg <= "0000000"; -- apaga (10..15)
28         end case;
29     end process;
30 end architecture;
```

FSM Semáforo (VHDL)

Esse circuito sequencial implementa uma máquina de estado Moore, pois o circuito depende somente do estado interno, iremos usar código VHDL para implementar.



```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

-- FSM Moore: RED -> GREEN -> YELLOW -> RED
-- Cada estado fica N ciclos de clock (genéricos abaixo)
entity traffic_light_timer is
    generic (
        RED_TICKS    : natural := 100;
        GREEN_TICKS  : natural := 100;
        YELLOW_TICKS : natural := 40
    );
    port (
        clk, reset : in  std_logic;  -- clock e reset assíncrono
        red, yellow, green : out std_logic
    );
end entity;

architecture rtl of traffic_light_timer is
    -- estados
    type state_t is (S_RED, S_GREEN, S_YELLOW);
    signal state, next_state : state_t;

    -- maior tempo para dimensionar o contador
    constant MAX_TICKS : natural :=
        integer'max(RED_TICKS, integer'max(GREEN_TICKS, YELLOW_TICKS));

    -- contador de ciclos no estado atual
    signal tick_cnt : natural range 0 to MAX_TICKS-1 := 0;

    -- função: tempo máximo de cada estado
    function max_ticks(s : state_t) return natural is
    begin
        case s is
            when S_RED    => return RED_TICKS;
            when S_GREEN  => return GREEN_TICKS;
            when S_YELLOW => return YELLOW_TICKS;
        end case;
    end function;

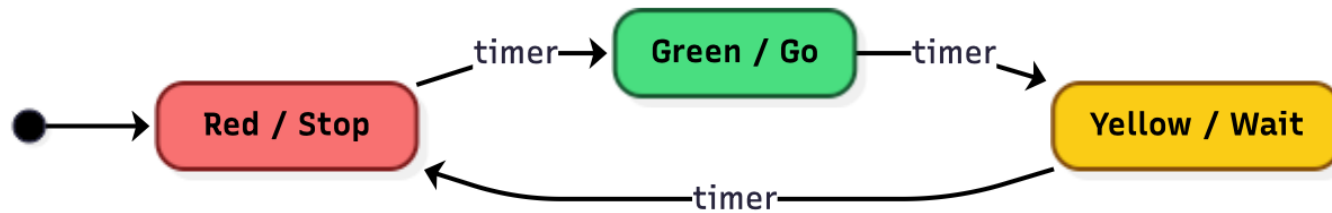
begin
    -- registrador de estado + contador
    process(clk, reset) begin
        if reset = '1' then
            state <= S_RED;
            tick_cnt <= 0;
        elsif rising_edge(clk) then
            state <= next_state;
            if next_state /= state then
                tick_cnt <= 0;
            elsif tick_cnt < max_ticks(state)-1 then
                tick_cnt <= tick_cnt + 1; -- continua contando
            else
                tick_cnt <= 0;
            end if;
        end if;
    end process;

    -- próxima transição
    process(state, tick_cnt) begin
        next_state <= state;
        if tick_cnt = max_ticks(state)-1 then
            case state is
                when S_RED    => next_state <= S_GREEN;
                when S_GREEN  => next_state <= S_YELLOW;
                when S_YELLOW => next_state <= S_RED;
            end case;
        end if;
    end process;

    -- saídas Moore
    red    <= '1' when state = S_RED    else '0';
    green  <= '1' when state = S_GREEN  else '0';
    yellow <= '1' when state = S_YELLOW else '0';
end architecture;
```


FSM Semáforo (Verilog)

Esse circuito sequencial implementa uma máquina de estado Moore, pois o circuito depende somente do estado interno, iremos usar código Verilog para implementar.



```
module traffic_light_timer #(
    parameter integer RED_TICKS    = 100,
    parameter integer GREEN_TICKS  = 100,
    parameter integer YELLOW_TICKS = 40,
    parameter integer CNT_WIDTH   = 16 // largura do contador (ajustar se preciso)
) (
    input wire clk,
    input wire reset, // reset assíncrono ativo em 1
    output reg red,
    output reg yellow,
    output reg green
);

// ----- Estados -----
localparam [1:0] S_RED    = 2'b00,
                S_GREEN    = 2'b01,
                S_YELLOW    = 2'b10;

reg [1:0] state, next;
reg [CNT_WIDTH-1:0] tick_cnt;
reg time_up;

// ----- Função "max_ticks" em forma de case combinacional -----
reg [CNT_WIDTH-1:0] max_ticks;
always @* begin
    case (state)
        S_RED:    max_ticks = RED_TICKS;
        S_GREEN:  max_ticks = GREEN_TICKS;
        S_YELLOW: max_ticks = YELLOW_TICKS;
        default:  max_ticks = RED_TICKS;
    endcase
end

// ----- Registrador de estado + contador -----
always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= S_RED;
        tick_cnt <= {CNT_WIDTH{1'b0}};
    end else begin
        state <= next;
        if (next != state)
            tick_cnt <= {CNT_WIDTH{1'b0}}; // mudou de estado → zera
        else
            tick_cnt <= tick_cnt + 1'b1; // mesmo estado → conta
    end
end

// ----- Lógica de próxima transição -----
always @* begin
    time_up = (tick_cnt == max_ticks-1);
    next = state;
    case (state)
        S_RED:    if (time_up) next = S_GREEN;
        S_GREEN:  if (time_up) next = S_YELLOW;
        S_YELLOW: if (time_up) next = S_RED;
    endcase
end

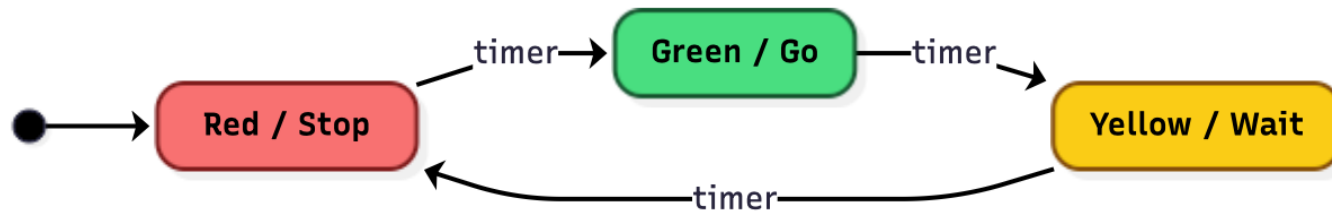
// ----- Sidas Moore -----
always @* begin
    red    = (state == S_RED);
    green  = (state == S_GREEN);
    yellow = (state == S_YELLOW);
end

endmodule
```




FSM Semáforo (System Verilog)

Esse circuito sequencial implementa uma maquina de estado Moore, pois o circuito depende somente do estado interno, iremos usar código Verilog para implementar.



```
// Semáforo com temporizador interno (FSM Moore)
// Estados: RED -> GREEN -> YELLOW -> RED
// Cada estado permanece N ciclos definidos por parâmetros

module traffic_light_timer #(
    parameter int RED_TICKS=100, GREEN_TICKS=100, YELLOW_TICKS=40
)(
    input logic clk, reset,          // clock e reset
    output logic red, yellow, green  // saídas Moore
);

// ----- Estados -----
typedef enum logic [1:0] { S_RED, S_GREEN, S_YELLOW } state_t;
state_t state, next;

// ----- Maior tempo entre os estados (para dimensionar o contador) -----
localparam int MAX_TICKS =
    (RED_TICKS>GREEN_TICKS ?
    (RED_TICKS>YELLOW_TICKS ? RED_TICKS : YELLOW_TICKS) :
    (GREEN_TICKS>YELLOW_TICKS ? GREEN_TICKS : YELLOW_TICKS));

// Contador de ciclos no estado atual
logic [$clog2(MAX_TICKS)-1:0] tick_cnt;
logic time_up;

// ----- Função: tempo máximo de cada estado -----
function automatic int max_ticks(state_t s);
    case (s)
        S_RED:    return RED_TICKS;
        S_GREEN:  return GREEN_TICKS;
        S_YELLOW: return YELLOW_TICKS;
        default:  return RED_TICKS;
    endcase
endfunction

// ----- Registrador de estado + contador -----
always_ff @(posedge clk or posedge reset) begin
    if (reset) begin
        state <= S_RED;    // estado inicial
        tick_cnt <= '0;
    end else begin
        state <= next;      // avança estado
        if (next != state) // mudou de estado -> zera contador
            tick_cnt <= '0;
        else                // continua no mesmo estado -> incrementa
            tick_cnt <= tick_cnt + 1'b1;
    end
end

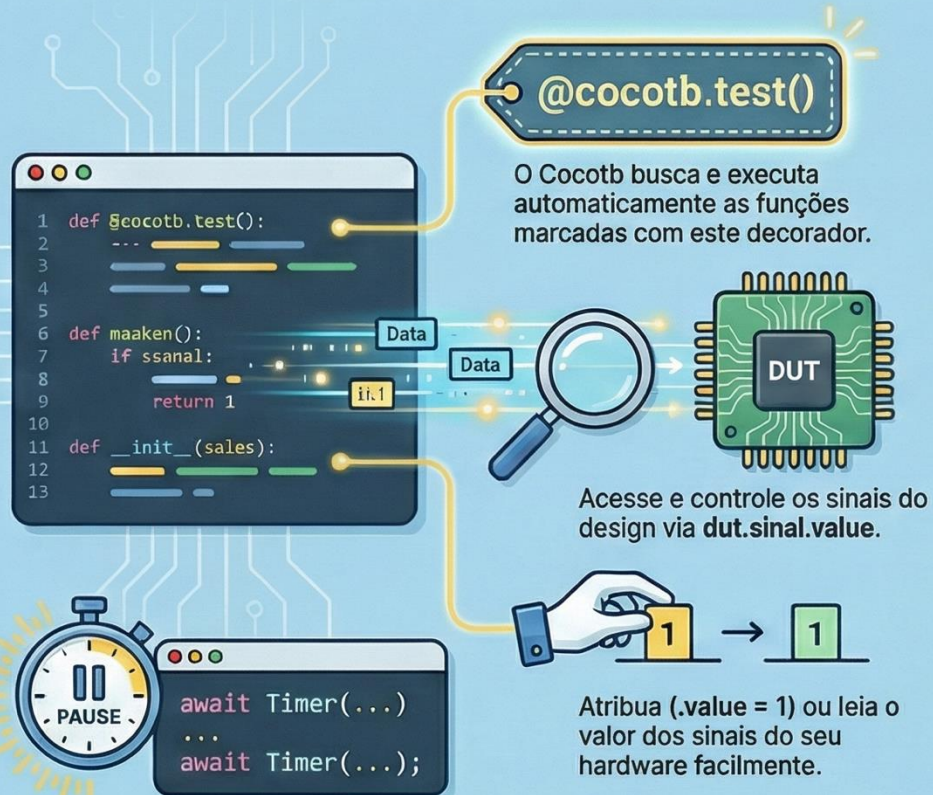
// ----- Lógica de próxima transição -----
always_comb begin
    time_up = (tick_cnt == max_ticks(state)-1);
    next = state;
    unique case (state)
        S_RED:    if (time_up) next = S_GREEN;
        S_GREEN:  if (time_up) next = S_YELLOW;
        S_YELLOW: if (time_up) next = S_RED;
    endcase
end

// ----- Saídas Moore -----
always_comb begin
    red    = (state == S_RED);
    green  = (state == S_GREEN);
    yellow = (state == S_YELLOW);
end
endmodule
```

Cocotb: Teste Seu Hardware com Python

O Cocotb é uma biblioteca de co-simulação que permite criar testbenches para **projetos de hardware (DUTs)** usando Python.

Passo 1: Escreva o Teste em Python



Use `await` para controlar o tempo e aguardar eventos. Espere por gatilhos, como um `Timer`, para sincronizar as ações com o simulador.

Passo 2: Configure e Execute a Simulação



Alternativa Moderna: Use "Python Runners".

Execute testes diretamente com scripts Python, uma alternativa experimental ao Makefile.

CocoTB

cocotb é um framework de **verificação de hardware baseado em Python**

- Permite escrever **testbenches em Python** para:
- **Verilog / SystemVerilog**
- **VHDL**
- Baseado em **corrotinas (async / await)**
- Open-source (BSD), multiplataforma
- Integra com simuladores via **VPI / VHPI / FLI / GPI**

? Por que usar cocotb?

- ✓ Testbench em **Python**, não em HDL
- ✓ Muito mais **produtivo** que SV/VHDL
- ✓ Fácil integração com:
 - NumPy / SciPy / PyTest
 - CI/CD (GitHub Actions, GitLab, Jenkins)
- ✓ Reutilização de código e testes randômicos (estilo UVM)

⚙️ Como o cocotb funciona?

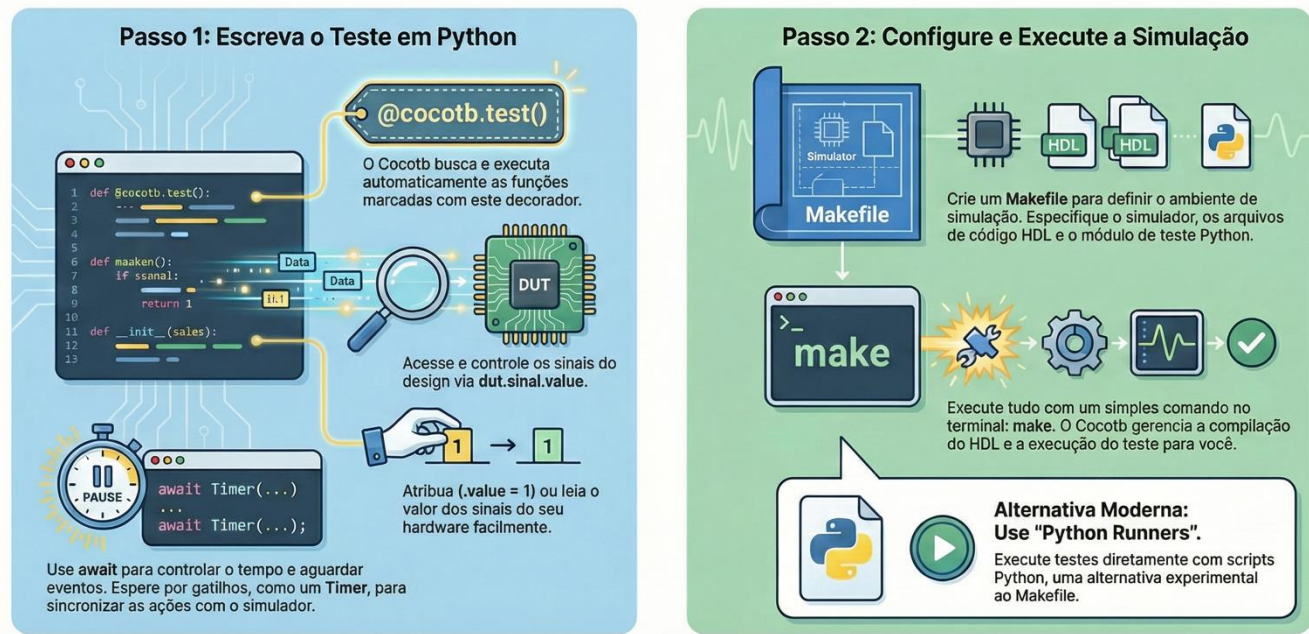
- O **DUT (Design Under Test)** roda no simulador
- O **Python controla o tempo** da simulação
- A cada **await**, o controle volta ao simulador
- Múltiplas corrotinas simulam processos paralelos

📌 Regra mental importante

- O CocoTB precisa de um simulador externo
- Vamos usar o Verilator, devido compatibilidade com Yosys e SystemVerilog

Cocotb: Teste Seu Hardware com Python

O Cocotb é uma biblioteca de co-simulação que permite criar testbenches para **projetos de hardware (DUTs)** usando Python.



🚀 Por que usar cocotb?

- Testes **mais legíveis**, funciona com **VHDL, Verilog e SystemVerilog**
- Fácil gerar:
- estímulos randômicos e sequências complexas
- Integra bem com:
 - CI/CD
 - Yosys
 - Verilator
- Excelente para **ASIC-style verification open-source**
- **Agnostico ao simulador**, pode usar simuladores **Comerciais e OpenSource**

Sintaxe Essencial do CocoTB

Definindo um teste

Todo teste deve ser uma função assíncrona (async) precedida pelo decorador @cocotb.test()

python

```
@cocotb.test()
async def meu_teste(dut): # 'dut' é o objeto que representa seu hardware
```

Manipulação de Sinais

Para ler ou escrever valores no hardware, acesse os nomes dos sinais como atributos do objeto dut.

- Escrita: dut.sinal.value = 10
- Leitura: valor = dut.sinal.value
- Conversão: int(dut.sinal.value) ou dut.sinal.value.integer

python

```
from cocotb.clock import Clock

# Cria um clock de 100MHz
meu_clock = Clock(dut.clk, 10, units="ns")
# Inicia sem bloquear o teste principal
cocotb.start_soon(meu_clock.start())
```

Controle do Tempo (Triggers)

A execução pausa no await até que o evento ocorra no simulador.

- Tempo fixo: await Timer(10, units="ns")
- Borda de sinal: await RisingEdge(dut.clk) ou await FallingEdge(dut.rst)
- Mudança de estado: await Edge(dut.sinal)

python

```
assert dut.saida.value == 1, f"Erro! Obtido: {dut.saida.value}"
```

python

```
dut._log.info("Iniciando a fase de reset...")
dut._log.error("Valor inesperado detectado!")
```

Verificação e logs

- Use os assert padrão do Python para validar o comportamento. Se falhar, o cocotb encerra o teste com erro.
- Em vez de print(), use o logger integrado do cocotb para mensagens formatadas.

```
@cocotb.test()
async def test_counter(dut):
    cocotb.start_soon(Clock(dut.clk, 10, unit="ns").start())

    dut.rst.value = 1
    await RisingEdge(dut.clk)
    dut.rst.value = 0

    for i in range(60):
        await RisingEdge(dut.clk)
        assert int(dut.count.value) == i
```

Exemplo Combinacional

systemverilog

```
module adder (  
    input logic [3:0] a,  
    input logic [3:0] b,  
    output logic [4:0] result  
);  
    always_comb begin  
        result = a + b;  
    end  
endmodule
```

python

```
import cocotb  
from cocotb.triggers import Timer  
  
@cocotb.test()  
async def adder_test(dut):  
    for i in range(4):  
        dut.a.value = i  
        dut.b.value = i  
        await Timer(1, units="ns")  
        assert dut.result.value == (i + i), f"Falha na soma: {i}+{i}"
```


Exemplo Sequencial

systemverilog

```
module counter (  
    input logic      clk,  
    input logic      rst,  
    output logic [3:0] count  
);  
    always_ff @(posedge clk or posedge rst) begin  
        if (rst)  
            count <= 4'b0000;  
        else  
            count <= count + 1'b1;  
        end  
    endmodule
```

python

```
import cocotb  
from cocotb.clock import Clock  
from cocotb.triggers import RisingEdge  
  
@cocotb.test()  
async def counter_test(dut):  
    # Inicializa Clock (100MHz = 10ns de período)  
    cocotb.start_soon(Clock(dut.clk, 10, units="ns").start())  
  
    # Aplica Reset  
    dut.rst.value = 1  
    await RisingEdge(dut.clk)  
    dut.rst.value = 0  
  
    # Verifica a contagem por 5 ciclos  
    for i in range(5):  
        await RisingEdge(dut.clk)  
        expected = i + 1  
        assert int(dut.count.value) == expected, f"Erro no ciclo {i}"  
  
    dut._log.info("Teste do contador finalizado com sucesso!")
```

Exemplo Makefile

O Makefile do cocotb não "roda" o teste diretamente; ele chama o simulador, compila o SystemVerilog em uma biblioteca compartilhada e injeta a biblioteca de runtime do cocotb nela.

Podemos também usar o cocotb sem Makefile, a forma recomendada de usar cocotb sem Makefile é através do pacote cocotb-test ou utilizando o Runner nativo do cocotb (introduzido em versões recentes para facilitar CI/CD e integração com Jupyter).

Para isso, você usará o módulo cocotb.runner.

makefile

```
# Configurações de simulador
SIM ?= verilator
TOPLEVEL_LANG ?= verilog

# Escolha o arquivo de hardware para compilar
VERILOG_SOURCES += $(PWD)/adder.sv
VERILOG_SOURCES += $(PWD)/counter.sv

# Defina o módulo TOP e o arquivo PYTHON de teste
# (Mude aqui para alternar entre os testes)
TOPLEVEL = counter
MODULE    = test_counter

# Flags para o Verilator (Gera ondas .vcd)
EXTRA_ARGS += --trace --trace-structs

include $(shell cocotb-config --makefiles)/Makefile.sim
```

Verificação Formal

? O problema da verificação tradicional? Simulação não explora todos os casos

- Testbenches testam **cenários específicos**
- Corner cases dependem de:
 - Criatividade / sorte / tempo
 - Bugs críticos podem:
 - nunca aparecer em simulação
 - aparecer só depois do tape-out (ASIC 🤖)

📌 Quanto maior o design, menor a cobertura real da simulação

✅ O que é Verificação Formal? Verificação Formal usa **matemática e lógica** para analisar o hardware:

Explora **todos os estados possíveis**

- Não depende de vetores de teste
- Prova propriedades ou gera contraexemplos
- 🔑 Em vez de testar valores, você testa **leis do sistema**

🔍 Dois modos fundamentais do Formal

1. Cover — Exploração do Espaço de Estados : “**Isso pode acontecer?**” “**Existe uma sequência que leva a esse estado?**”

- O solver tenta **encontrar um caminho**
- Você descreve o **estado desejado**
- A ferramenta descobre:
 - Sequências / condições implícitas
 - dependências escondidas
- 📌 Pode ser usado como **resolvedor de problemas**

2. Assertions + BMC — Prova e Caça a Bugs: “**Isso nunca pode acontecer!**”

- Você declara regras que **devem sempre valer**
- O solver tenta **quebrá-las**
- Se falhar → gera **counterexample (trace/VCD)**
- 📌 Base para:
 - bug hunting
 - verificação de FSMs
 - **Sneak Path Analysis**
- provar que estados críticos só ocorrem pelos caminhos esperados



Formal vs TestBenches

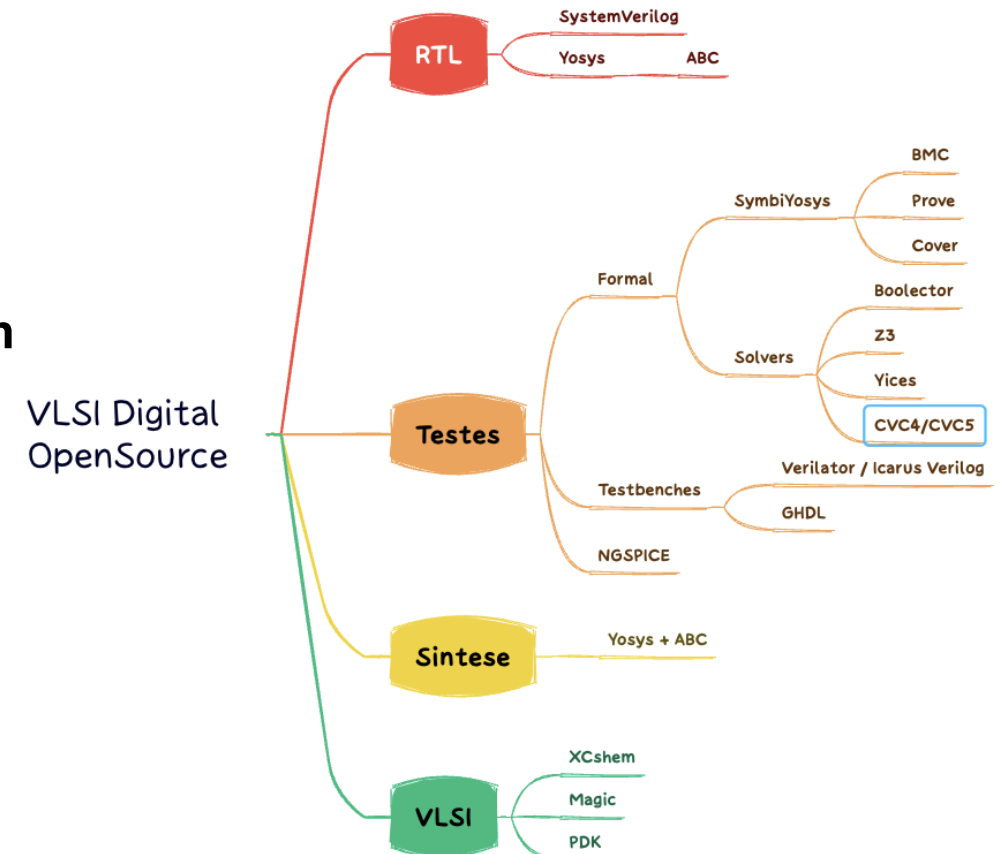
Testes formais não substituem a simulação — eles a complementam. Ainda precisamos de testbenches.

Por que usar Formal desde o design?

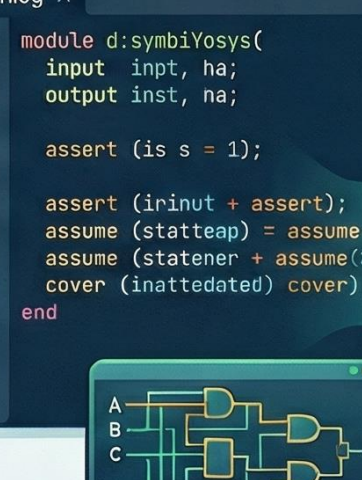
-  **Mais segurança** (especialmente em ASIC)
-  **Bugs aparecem mais cedo**
-  **Entendimento profundo do design**
-  **Menos dependência de testbench complexo**
-  **Camada extra de verificação**

Formal não é só verificação — é uma ferramenta de design

Simulação	Formal
Testa casos	Explora todos os casos
Precisa de estímulos	Precisa de propriedades
Pode perder bugs	Prova ausência de bugs
Baseada em tempo	Baseada em estados
“Funciona?”	“É impossível falhar?”



Tutorial de Verificação Formal com SymbiYosys



```
1 module d:symbiYosys(  
2     input  inpt, ha;  
3     output inst, na;  
4  
5     assert (is s = 1);  
6  
7     assert (irinput + assert);  
8     assume (statteap) = assume(2);  
9     assume (statener + assume(2);  
10    cover (inattedated) cover));  
11 end  
12  
13
```



The logic diagram shows a circuit with three inputs (A, B, C) and one output (V). The circuit is composed of several logic gates (AND, OR, NOT) connected in a complex manner. The inputs A, B, and C are connected to various gates, which eventually lead to the output V. The diagram is a schematic representation of the logic implemented in the Verilog code above.

1. Descreva o Design em Verilog

Codifique seu hardware e inclua propriedades formais usando assert, assume e cover.

```

module d:symbiYosys(
  input inpt, nq;
  output inpt, nq;

  assert (e_s == 1))

  assert (inc == 1 - 0)
  assume (s0 == 0)
;
00-set

assert (ieach == fil);

assume (iss==1);
cover (covr == 0);
end

```

2. O Yosys Sintetiza o Código

O Yosys compila o Verilog para um modelo formal, uma representação matemática para análise.

4. SymbiYosys Orquestra o Processo

Ele gerencia o Yosys e o solucionador, simplificando a execução da verificação.

[illegible]

3. O Solucionador SMT Realiza a Prova

Uma ferramenta de prova matemática (solucionador SMT) verifica exaustivamente se as asserções são sempre verdadeiras.

**VERIFICADO**

CONTRAEXEMPLO ENCONTRADO

5. Analise os Resultados

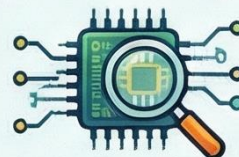
Se uma propriedade falhar, a ferramenta gera um contraexemplo, mostrando uma forma de onda do erro.

Por Que Usar Verificação Formal?



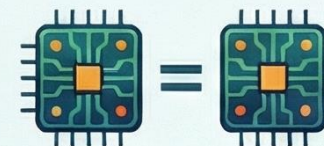
Prova de Correção

Verifica matematicamente que o design está correto para todos os possíveis estados e entradas.



Encontre Bugs Difíceis

Detecta casos extremos e complexos que a simulação tradicional pode não encontrar.



Equivalência Formal

Prove que duas implementações de um design, como antes e depois de uma otimização, são funcionalmente idênticas.

Estrutura de Projeto SimbiYosys

Conceitos fundamentais (antes do código)

◆ **assert:** “Isso nunca pode ser falso”

Usado para:

- invariantes
- regras de transição
- segurança

◆ **assume:** “Considere que o mundo se comporta assim”

Usado para:

- restringir entradas
- modelar reset
- evitar cenários irreais

◆ **cover:** “Existe algum caminho para isso acontecer?”

Usado para:

- exploração
- entender comportamento
- “resolver” problemas

◆ **past:** Valor de uma expressão no clock anterior

👉 Isso expressa uma **relação temporal** entre estados:

- estado atual
- estado anterior

So precisamos de um flag para o primeiro resultado não falhar (past_valid)

3 Estrutura padrão de projeto (REGRA DE OURO)

text

```
project/  
├── design.sv           # RTL puro (DUT)  
├── design_formal.sv    # Formal testbench  
└── design.sby          # Configuração SBY
```

- ❌ Não misture lógica com asserts
- ❌ Não gere clock com #delay
- ✅ Um arquivo = uma responsabilidade

✅ O papel de past_valid

past_valid é um flag que diz ao formal:

“Agora já existe um ciclo anterior válido”

Implementação típica:

```
SV  
  
logic past_valid = 0;  
  
always_ff @(posedge clk)  
    past_valid <= 1;
```

Isso cria:

- step 0 → past_valid = 0
- step ≥1 → past_valid = 1

🧩 Como usamos past_valid corretamente

Sempre que usar \$past, faça:

```
SV  
  
if (past_valid) begin  
    assert(q == $past(q));  
end
```

Ou, mais completo:

```
SV  
  
if (past_valid && !$past(rst) && !rst) begin  
    ...  
end
```

Exemplo Contador 0..9

5.1 Design (bcd_counter.sv)

RTL simples, sem asserts:

```
systemverilog

module bcd_counter (
    input  logic clk,
    input  logic rst,
    input  logic en,
    output logic [3:0] q
);

    always_ff @(posedge clk) begin
        if (rst)
            q <= 4'd0;
        else if (en) begin
            if (q == 4'd9) q <= 4'd0;
            else
                q <= q + 4'd1;
        end
    end

endmodule
```

5.3 Arquivo SBY (bcd_counter.sby)

```
ini

[options]
mode bmc
depth 20

[engines]
smtbmc boolector

[script]
read -formal bcd_counter.sv bcd_counter_formal.sv
prep -top bcd_counter_formal

[files]
bcd_counter.sv
bcd_counter_formal.sv
```

```

(OSS CAD Suite) (base) → sequential git:(main) × sby bcd_counter.sby
SBY 23:30:53 [bcd_counter] Copy '/Users/leonardoaraujo/work/learning_ngspice/exercises/formal_v/sequential/bcd_counter/src/bcd_counter.sv'.
SBY 23:30:53 [bcd_counter] Copy '/Users/leonardoaraujo/work/learning_ngspice/exercises/formal_verification/sequential/bcd_counter/src/bcd_counter_formal.sv'.
SBY 23:30:53 [bcd_counter] engine_0: smtbmc boolector
SBY 23:30:53 [bcd_counter] base: starting process "cd bcd_counter/src; yosys -ql ../model/design_prep.smtc"
SBY 23:30:53 [bcd_counter] base: Warning: Wire bcd_counter_formal.\clk is used but has no driver
SBY 23:30:53 [bcd_counter] base: finished (returncode=0)
SBY 23:30:53 [bcd_counter] prep: starting process "cd bcd_counter/model; yosys -ql design_prep.smtc"
SBY 23:30:53 [bcd_counter] prep: Warning: Wire bcd_counter_formal.\clk is used but has no driver
SBY 23:30:53 [bcd_counter] prep: finished (returncode=0)
SBY 23:30:53 [bcd_counter] smt2: starting process "cd bcd_counter/model; yosys -ql design_smt2.smtc"
SBY 23:30:53 [bcd_counter] smt2: finished (returncode=0)
SBY 23:30:53 [bcd_counter] engine_0: starting process "cd bcd_counter; yosys-smtbmc -s boolector vlogtb engine_0/trace_tb.v --dump-smtc engine_0/trace.smtc model/design_smt2.smt2"
SBY 23:30:53 [bcd_counter] engine_0: ## 0:00:00 Solver: boolector
SBY 23:30:53 [bcd_counter] engine_0: ## 0:00:00 Checking assumptions in step 0..
SBY 23:30:53 [bcd_counter] engine_0: ## 0:00:00 Checking assertions in step 0..
SBY 23:30:53 [bcd_counter] engine_0: ## 0:00:00 Checking assumptions in step 1..
SBY 23:30:53 [bcd_counter] engine_0: ## 0:00:00 Checking assertions in step 1..
SBY 23:30:53 [bcd_counter] engine_0: ## 0:00:00 Checking assumptions in step 2..

```

5.2 Teste Formal (bcd_counter_formal.sv)

Aqui está o coração da verificação:

```

systemverilog

module bcd_counter_formal;

    (* gclk *) logic clk;
    (* anyseq *) logic rst;
    (* anyseq *) logic en;

    logic [3:0] q;

    bcd_counter dut (.*);

    logic past_valid = 0;
    always_ff @(posedge clk)
        past_valid <= 1;

    // Começa em reset
    always_ff @(posedge clk)
        if ($initstate) assume(rst);

    // Reset só ocorre no início
    always_ff @(posedge clk)
        if (past_valid && $past(!rst))
            assume(!rst);

    // q sempre entre 0 e 9
    always_ff @(posedge clk)
        assert(q <= 4'd9);

    // Reset síncrono (efeito no ciclo seguinte)
    always_ff @(posedge clk)
        if (past_valid && $past(rst))
            assert(q == 4'd0);

    // Regras de transição
    always_ff @(posedge clk)
        if (past_valid && !$past(rst) && !rst) begin
            if (!$past(en))
                assert(q == $past(q));
            if ($past(en)) begin
                if ($past(q) == 4'd9)
                    assert(q == 4'd0);
                else
                    assert(q == $past(q) + 1);
            end
        end

    // Exploração
    always_ff @(posedge clk)
        cover(q == 4'd9);

endmodule

```

Exemplo Adder4

Estrutura

text

```
adder4/  
├─ adder4.sv  
├─ adder4_formal.sv  
└─ adder4.sby
```

1) Design (DUT) — adder4.sv

```
systemverilog  
  
module adder4 (  
    input logic [3:0] a,  
    input logic [3:0] b,  
    input logic      cin,  
    output logic [3:0] sum,  
    output logic      cout  
);  
    logic [4:0] full;  
  
    always_comb begin  
        full = {1'b0, a} + {1'b0, b} + cin;  
        sum  = full[3:0];  
        cout = full[4];  
    end  
endmodule
```

2) Formal testbench — adder4_formal.sv

Ideia: como é combinacional, não precisa clock, \$past, past_valid, nada disso.

A estratégia formal aqui é:

- deixar entradas livres (anyseq)
- afirmar que a saída bate com a **especificação matemática**
- adicionar alguns **covers** para gerar traces interessantes

```
systemverilog  
  
module adder4_formal;  
  
    // Entradas livres para o solver  
    (* anyseq *) logic [3:0] a;  
    (* anyseq *) logic [3:0] b;  
    (* anyseq *) logic      cin;  
  
    logic [3:0] sum;  
    logic      cout;  
  
    adder4 dut (.*);  
  
    // Especificação de referência (matemática)  
    logic [4:0] ref;  
    always_comb ref = {1'b0, a} + {1'b0, b} + cin;  
  
    // ASSERT: DUT == spec  
    always_comb begin  
        assert({cout, sum} == ref);  
    end  
  
    // Covers úteis (para ver exemplos no trace)  
    always_comb begin  
        cover(a == 4'hF && b == 4'h1 && cin == 1'b0); // 15+1  
        cover(sum == 4'h0 && cout == 1'b1);           // overflow  
        cover(sum == 4'hA);                             // exemplo qualquer  
    end  
  
endmodule
```

Emulando Erro no Adder

✓ Como "testar" que formal está funcionando?

Faça um bug proposital no DUT, por exemplo troque:

```
SV  
  
full = {1'b0, a} + {1'b0, b} + cin;
```

por (bug):

```
SV  
  
full = {1'b0, a} + {1'b0, b}; // esqueceu cin
```

Rode sby de novo.

👉 Vai dar **FAIL** e o trace vai mostrar um caso onde cin=1 quebra.

ALU 4-bits

A ALU vai suportar 4 operações (2-bit opcode):

- 00 → ADD (a + b)
- 01 → SUB (a - b)
- 10 → AND (a & b)
- 11 → XOR (a ^ b)

Saídas:

- y[3:0]
- flags: z (zero), c (carry/borrow), n (negative = MSB)

3) SBY — mini_alu.sby

ini

[options]

mode bmc

depth 1

[engines]

smtbmc boolector

[script]

read -formal mini_alu.sv mini_alu_formal.sv

prep -top mini_alu_formal

[files]

mini_alu.sv

mini_alu_formal.sv

1) DUT — mini_alu.sv

systemverilog

```
module mini_alu (
    input  logic [3:0] a,
    input  logic [3:0] b,
    input  logic [1:0] op,    // 00 add, 01 sub, 10 and, 11 xor
    output logic [3:0] y,
    output logic      z,      // y == 0
    output logic      n,      // y[3]
    output logic      c       // carry (add) / no-borrow (sub)
);

    logic [4:0] tmp;

    always_comb begin
        tmp = 5'd0;
        unique case (op)
            2'b00: begin // ADD
                tmp = {1'b0, a} + {1'b0, b};
                y    = tmp[3:0];
                c    = tmp[4];    // carry-out
            end

            2'b01: begin // SUB (a - b)
                tmp = {1'b0, a} - {1'b0, b};
                y    = tmp[3:0];
                // Em subtração unsigned, tmp[4] funciona como "no borrow" em muitos fluxos:
                // se a>=b => tmp[4]=1, se a<b => tmp[4]=0 (depende do estilo; vamos validar formalmente via spec).
                c    = ~tmp[4]; // vamos definir c como "borrow"=1 quando a<b (mais intuitivo)
            end

            2'b10: begin // AND
                y = a & b;
                c = 1'b0;
            end

            2'b11: begin // XOR
                y = a ^ b;
                c = 1'b0;
            end
        endcase

        z = (y == 4'd0);
        n = y[3];
    end
endmodule
```

```

module mini_alu_formal;

    (* anyseq *) logic [3:0] a;
    (* anyseq *) logic [3:0] b;
    (* anyseq *) logic [1:0] op;

    logic [3:0] y;
    logic z, n, c;

    mini_alu dut (.*);

    // SPEC de referência
    logic [4:0] ref_add;
    logic [4:0] ref_sub;
    logic [3:0] ref_y;
    logic      ref_c;    // borrow em SUB, carry em ADD
    logic      ref_z;
    logic      ref_n;

    always_comb begin
        ref_add = {1'b0, a} + {1'b0, b};
        ref_sub = {1'b0, a} - {1'b0, b};

        ref_y = 4'h0;
        ref_c = 1'b0;

        unique case (op)
            2'b00: begin // ADD
                ref_y = ref_add[3:0];
                ref_c = ref_add[4];           // carry
            end

            2'b01: begin // SUB (borrow = 1 quando a<b)
                ref_y = ref_sub[3:0];
                ref_c = (a < b);             // definição clara e sem ambiguidades
            end

            2'b10: begin // AND
                ref_y = (a & b);
                ref_c = 1'b0;
            end

            2'b11: begin // XOR
                ref_y = (a ^ b);
                ref_c = 1'b0;
            end
        endcase
    end
endmodule

```

```

[[OSS CAD Suite] (base) → combinational git:(main) × sby mini_alu.sby
SBY 12:39:42 [mini_alu] Copy '/Users/leonardoaraujo/work/learning_ngspice/exercises/formal_verification/combinational/mini_mbinational/mini_alu/src/mini_alu.sv'.
SBY 12:39:42 [mini_alu] Copy '/Users/leonardoaraujo/work/learning_ngspice/exercises/formal_verification/combinational/mini_tion/combinational/mini_alu/src/mini_alu_formal.sv'.
SBY 12:39:42 [mini_alu] engine_0: smtbmc boolector
SBY 12:39:42 [mini_alu] base: starting process "cd mini_alu/src; yosys -ql ../model/design.log ../model/design.yml"
SBY 12:39:42 [mini_alu] base: finished (returncode=0)
SBY 12:39:42 [mini_alu] prep: starting process "cd mini_alu/model; yosys -ql design_prep.log design_prep.yml"
SBY 12:39:42 [mini_alu] prep: finished (returncode=0)
SBY 12:39:42 [mini_alu] smt2: starting process "cd mini_alu/model; yosys -ql design_smt2.log design_smt2.yml"
SBY 12:39:42 [mini_alu] smt2: finished (returncode=0)
SBY 12:39:42 [mini_alu] engine_0: starting process "cd mini_alu; yosys-smtbmc -s boolector --presat --unroll --noprogess engine_0/trace_tb.v --dump-smtc engine_0/trace.smtc model/design_smt2.smt2"
SBY 12:39:42 [mini_alu] engine_0: ## 0:00:00 Solver: boolector
SBY 12:39:42 [mini_alu] engine_0: ## 0:00:00 Checking assumptions in step 0..
SBY 12:39:42 [mini_alu] engine_0: ## 0:00:00 Checking assertions in step 0..
SBY 12:39:42 [mini_alu] engine_0: ## 0:00:00 Status: passed
SBY 12:39:42 [mini_alu] engine_0: finished (returncode=0)
SBY 12:39:42 [mini_alu] engine_0: Status returned by engine: pass
SBY 12:39:42 [mini_alu] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 12:39:42 [mini_alu] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 12:39:42 [mini_alu] summary: engine_0 (smtbmc boolector) returned pass
SBY 12:39:42 [mini_alu] summary: engine_0 did not produce any traces
SBY 12:39:42 [mini_alu] DONE (PASS, rc=0)
[[OSS CAD Suite] (base) → combinational git:(main) ×

```

```

        ref_z = (ref_y == 4'd0);
        ref_n = ref_y[3];

        // ASSERT principal: DUT == SPEC
        assert(y == ref_y);
        assert(c == ref_c);
        assert(z == ref_z);
        assert(n == ref_n);
    end

    // Covers: gerar alguns casos interessantes
    always_comb begin
        // Overflow em ADD (carry=1)
        cover(op == 2'b00 && c == 1'b1);

        // Borrow em SUB (a<b)
        cover(op == 2'b01 && c == 1'b1);

        // Resultado zero em algum op
        cover(z == 1'b1);

        // Caso específico para visualizar
        cover(op == 2'b00 && a == 4'hF && b == 4'h1 && y == 4'h0 && c == 1'b1);
    end
endmodule

```

Lista de Exercícios na Verilog/VHDL

Circuitos Combinacionais

1. Alarme de Porta Aberta (Sistema de Segurança)

- Um sistema de alarme deve acionar uma sirene (S) **se pelo menos uma das duas portas (P1, P2) estiver aberta** e o alarme estiver ativado (A).

2. Controle de Ventilador de Geladeira

- Um ventilador deve ligar (V=1) **quando a temperatura for alta (T=1) ou quando a porta estiver aberta (P=1), mas não se o modo econômico estiver ativo (E=1).**

3. Semáforo para Pedestres

- Um semáforo para pedestres deve acender a luz verde (G=1) quando:
- O botão de pedestre (B) for pressionado **e**
- O sinal de carros estiver vermelho (C=1).

4. Multiplexador de 3 Entradas

- Projete um circuito **multiplexador de 3 entradas** que seleciona um dos sinais I_0, I_1, I_2 para a saída Y, usando 2 linhas de seleção (S_0, S_1).
- 👉 Observação: como o multiplexador clássico é 2^n entradas, aqui temos **3 entradas**, então usaremos as combinações 00, 01, 10. O caso 11 pode ser tratado como **saída 0 (desligada)**.

Sequenciais

1. FSM Moore (Detector de Sequência 101)

- Projete uma **FSM Moore** que detecta a sequência de bits 101 em uma entrada serial X. A saída Z deve ser **1** apenas quando a sequência foi reconhecida.

2. FSM Mealy (Detector de Sequência 101)

- Projete uma **FSM Mealy** que detecta a mesma sequência 101, mas agora a saída Z deve ser **1 imediatamente** ao reconhecer o último bit, sem esperar mudar de estado.

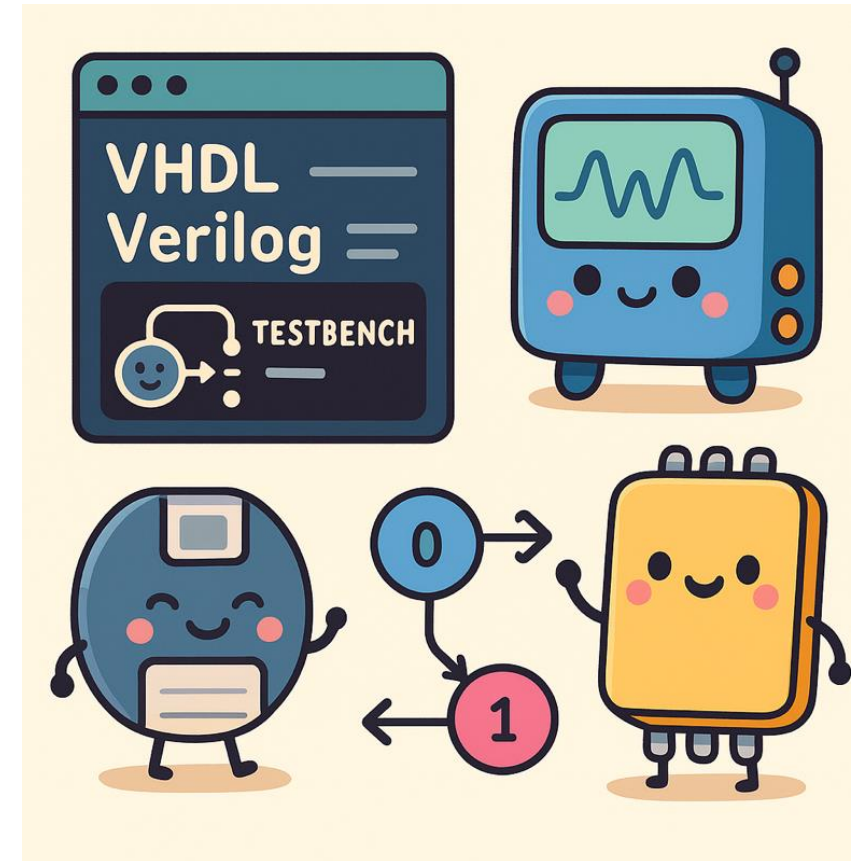
3. Contador Síncrono Módulo-4 (00 → 01 → 10 → 11 → 00)

- Projete um **contador síncrono** de 2 bits (Q1, Q0) que conta em binário crescente de 0 a 3.

Fim da Parte 3

A partir deste ponto nos já sabemos:

- VHDL e Verilog, criar circuitos e testbenches
- Simular circuitos descritos em Verilog e VHDL
- Implementar maquinas de estado, e circuitos combinacionais



LibreLane

LibreLane é uma **infraestrutura de flows** para implementação física de ASICs, que:

- Orquestra **ferramentas open-source de EDA**
- Usa **um único arquivo de configuração (JSON ou Python)**
- Automatiza o caminho **RTL(Verilog/VHDL) → GDS**



 **Objetivo:**

- Tornar o *ASIC Physical Design* acessível, reproduzível e configurável.

A entrada do LibreLane:

- Seu Design / Idea (Verilog / SystemVerilog)
- PDK do Fabricante (Skywater130nm)

A Saida do LibreLane e o arquivo GDS, para ser enviado para foundry.

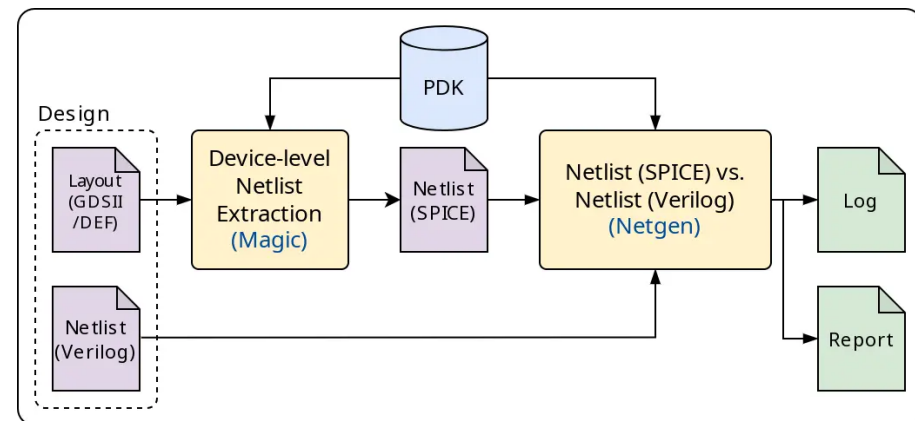
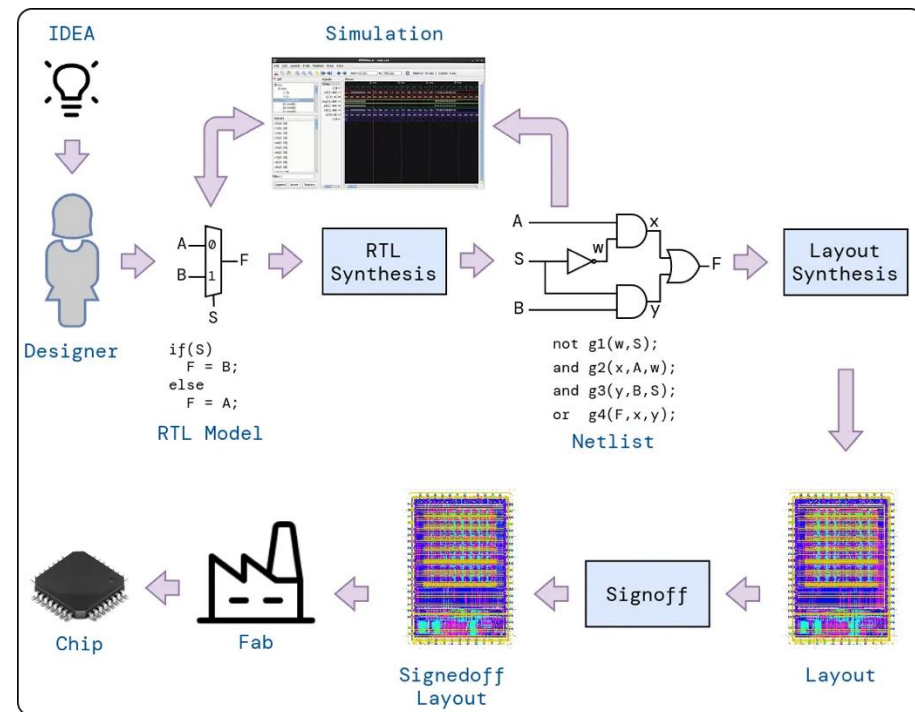


🧠 O que é GDS?

- **GDS (ou GDSII)** é o **arquivo final de layout físico de um chip**, usado para **fabricação em uma foundry**.
- 🖱 Em uma frase: **GDS é o “blueprint” do chip que vai para a fábrica.**

O LibreLane é composto por diversos software OpenSource, como:

- Yosys / ABC (Síntese)
- Verilator (Simulação)
- OpenRoad, responsável pelo
 - Floorplaning / Placement / Routing
- Magic / Klayout
 - DRC / Visualização
- Netgen
 - LVS (Layout vs Schematic)



Fluxo Básico LibreLane

Como dito anteriormente a entrada do LibreLane:

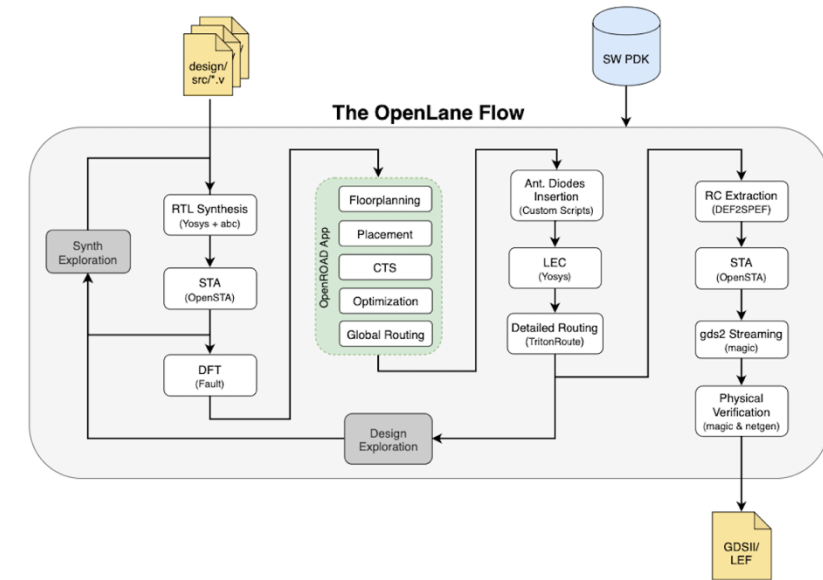
- PDK
- arquivos RTL do projeto.
- Config.json

Após isso só precisamos executar os comandos:

```
librelane adder4/config.json
```

```
librelane --last-run --flow openinklayout adder4/config.json
```

Isso ira sintetizar o projeto, e gerar o arquivo GDS para ser enviado para foundry, por padrão esse será o PDK do skylane.

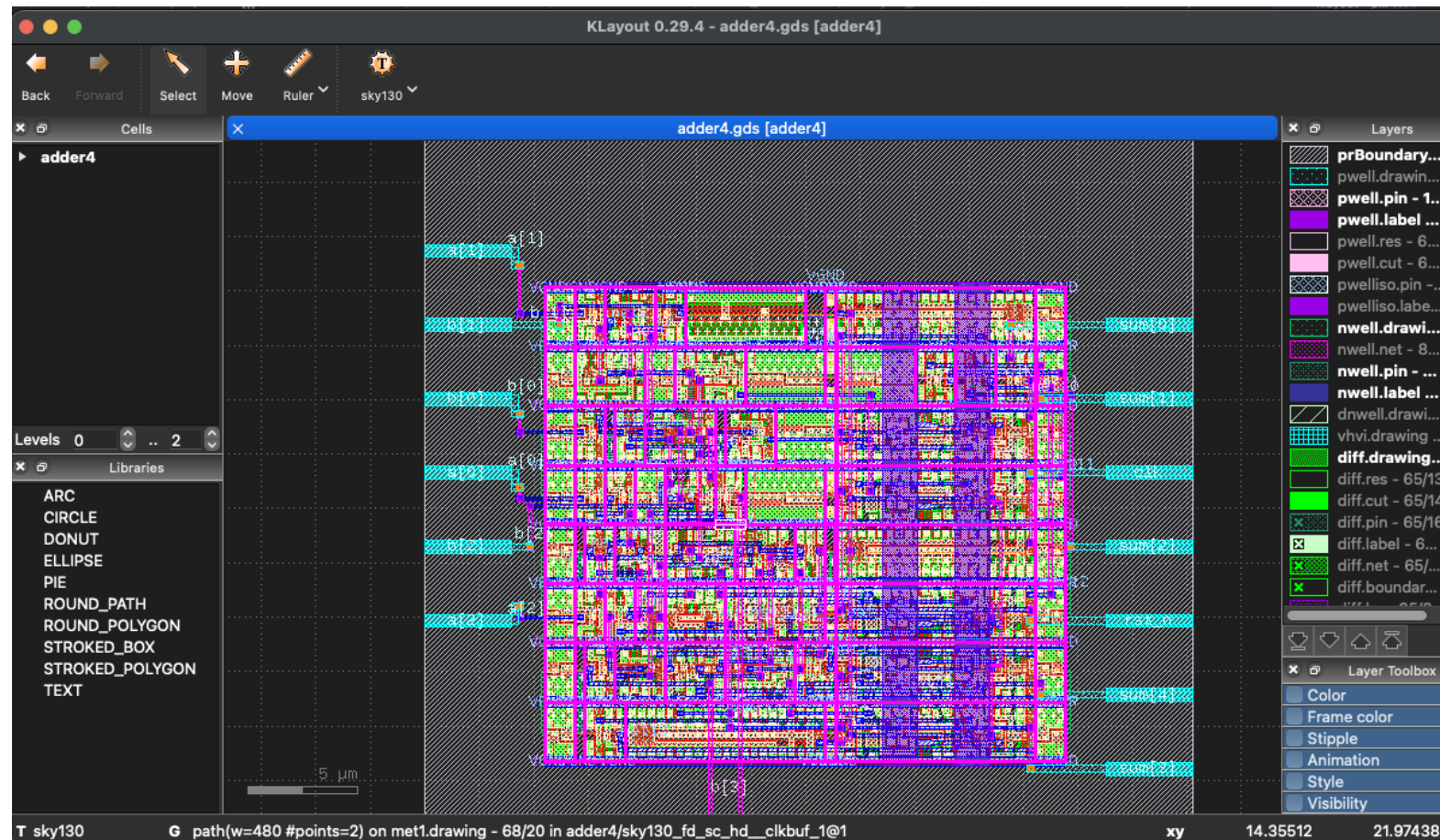


```
{ } config.json x
adder4 > { } config.json > ...
1 {
2   "DESIGN_NAME": "adder4",
3
4   "VERILOG_FILES": [
5     "dir::adder4.sv"
6   ],
7
8   "CLOCK_PORT": "clk"
9 }
10
```

```
adder4.sv x
adder4 > adder4.sv
1 module adder4 (
2   input logic clk,
3   input logic rst_n,
4   input logic [3:0] a,
5   input logic [3:0] b,
6   output logic [4:0] sum
7 );
8
9   always_ff @(posedge clk or negedge rst_n) begin
10     if (!rst_n)
11       sum <= 5'b0;
12     else
13       sum <= a + b;
14   end
15
16 endmodule
17
```

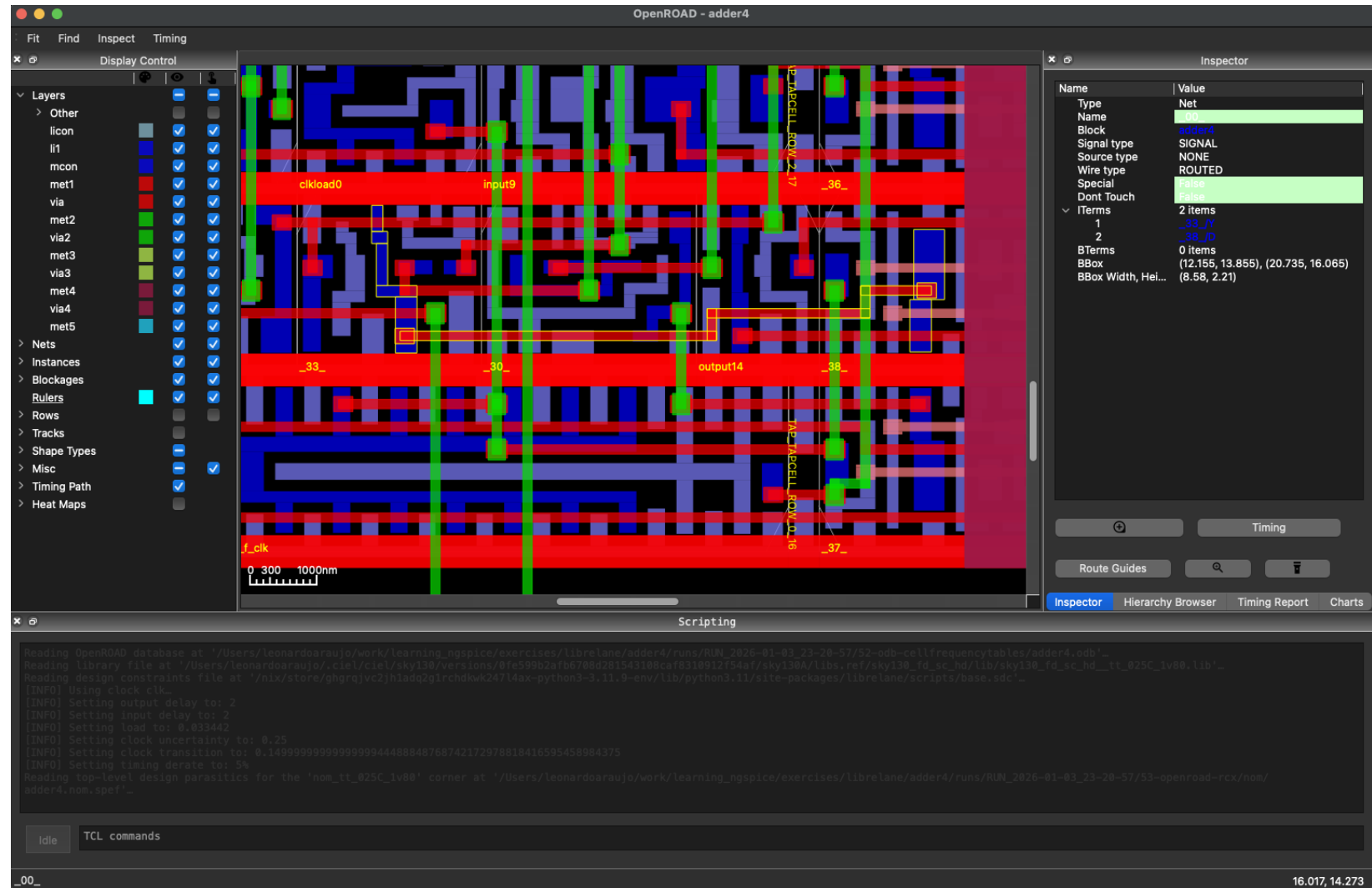
Resultado no KLayout

`librelane --last-run --flow openinklayout adder4/config.json`



Resultado no OpenROAD

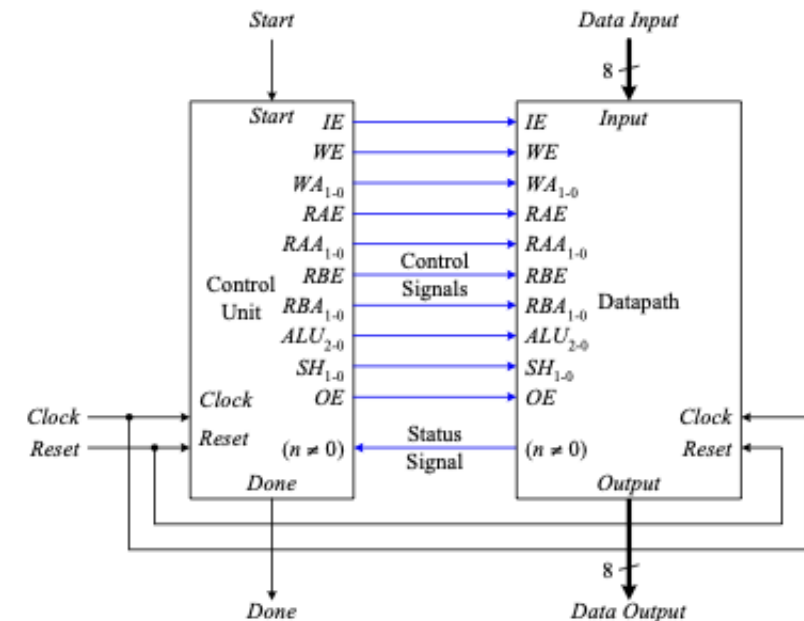
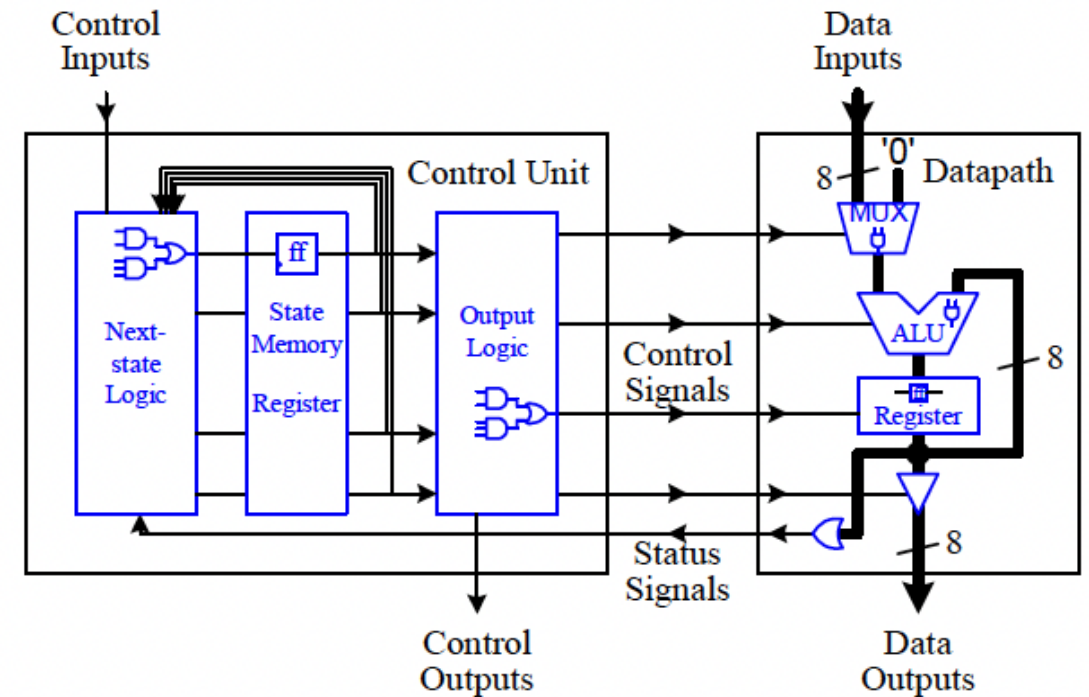
```
librelane --last-run --flow openinopenroad adder4/config.json
```



Processador Básico

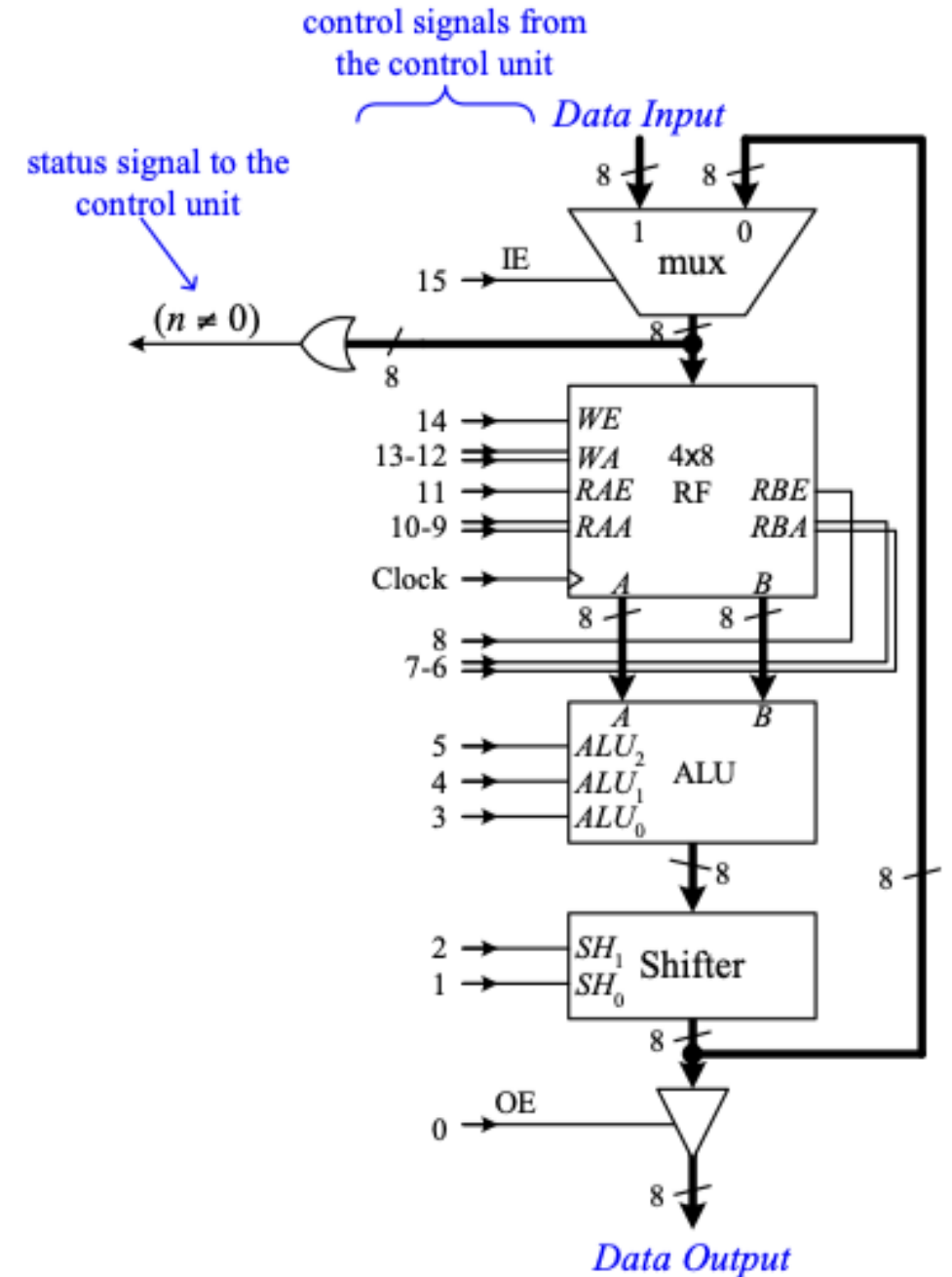
Baseado no primeiro livro sobre desenvolvimento de processadores
“**Enoch O. Hwang Digital Logic and Microprocessor Design With VHDL**”

Iremos desenvolver o mesmo processador em VHDL, Verilog e SystemVerilog



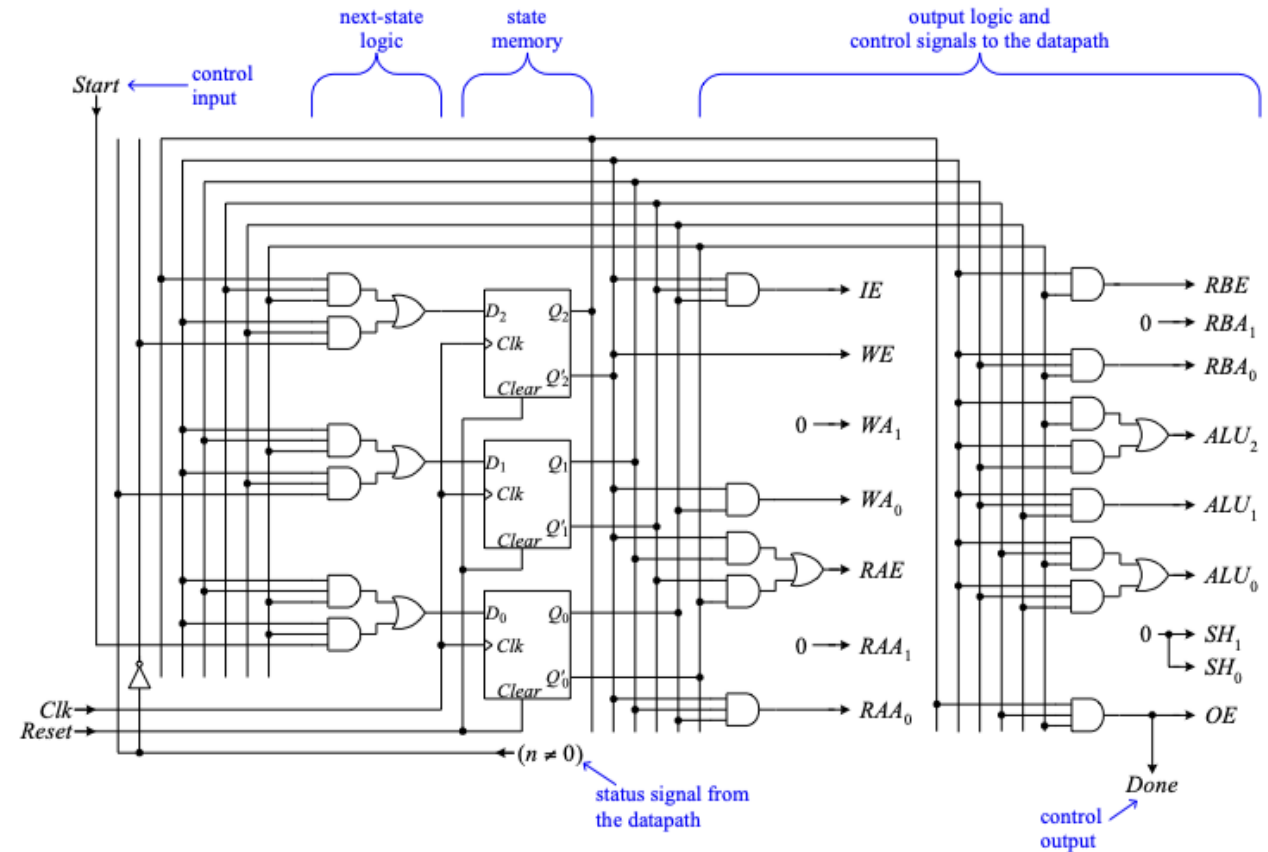
Datapath

Prove para unidade de controle circuitos para execução das instruções. Ele é composto por registradores, ALU, e flags para saber o resultado de determinadas instruções.



Unidade de Controle

Envia sinais para o Datapath para executar uma determinada instrução.



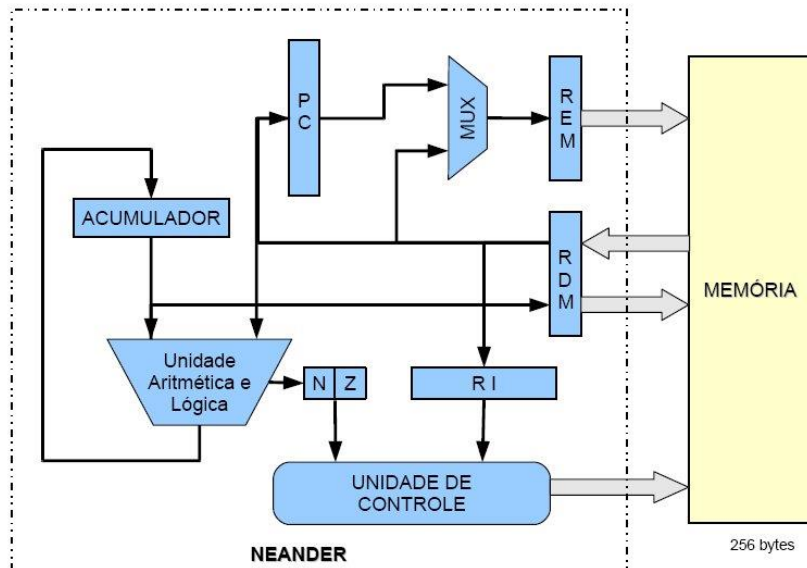
Control Word	Instruction	IE	WE	WA _{1,0}	RAE	RAA _{1,0}	RBE	RBA _{1,0}	ALU _{2,1,0}	SH _{1,0}	OE
		15	14	13-12	11	10-9	8	7-6	5-3	2-1	0
1	$sum = 0$	0	1	00	1	00	1	00	101 (subtract)	00	0
2	input n	1	1	01	0	xx	0	xx	xxx	00	0
3	$sum = sum + n$	0	1	00	1	00	1	01	100 (add)	00	0
4	$n = n - 1$	0	1	01	1	01	0	xx	111 (decrement)	00	0
5	output sum	x	0	xx	1	00	0	xx	000 (pass)	00	1

Oque faremos?

Iremos desenvolver um hardware para o processador Neander, implementar em um Chip real, no projeto Tiny Typeout

Principais Características

- 8 Bits de Largura
- Dados representados em complemento de dois
- Modo de endereçamento direto
- 1 Registrador Acumulador
- PC de 8 bits
- Flags para Zero e Negativo



Código	Instrução	Operação
0000	NOP	Nenhuma operação
0001	STA end	Armazena acumulador no endereço “end” da memória
0010	LDA end	Carrega o acumulador com o conteúdo do endereço “end” da memória
0011	ADD end	Soma o conteúdo do endereço “end” da memória ao acumulador
0100	OR end	Efetua operação lógica “OU” do conteúdo do endereço “end” da memória ao acumulador
0101	AND end	Efetua operação lógica “E” do conteúdo do endereço “end” da memória ao acumulador
0110	NOT	Inverte todos os bits do acumulador
1000	JMP end	Desvio incondicional para o endereço “end” da memória
1001	JN end	Desvio condicional, se “N=1”, para o endereço “end” da memória
1010	JZ end	Desvio condicional, se “Z=1”, para o endereço “end” da memória
1111	HLT	Para o ciclo de busca-decodificação-execução

Referencias

- [Oque e Lógica, Primeiro computador do Mundo](#)
- [Aristototele o Homem mais Importante e Lógica de Aristotele](#)
- [George Boole– Criador da Logica Booleana](#)
- [Thesis Claude Shannon e The Story of Information Theory, em PT, Trailler](#)
- [The Bit Player, The Shannon Limit, Information Theory, Relés oque são e para que servem?](#)
- [Why The First Computers Were Made Out Of Light Bulbs, Allan Turing Machine](#)
- [The Big Misconception About Electricity, How it actually works](#)
- [Understanding Logic Gates, with Dominos, Transistors, Water, More Gates](#)
- [Mapa de Karnaugh, Projeto circuito passo a passo](#)
- [Equação para Circuito, projeto prático, Circuitos Combinacionais \(Playlist\), Display 7 Segmentos](#)
- [Tudo sobre 555, Projetar Oscilador com 555, How a 555 timer works, Crystal de Quartz](#)
- [How does a Flip-Flop Work and Metastability, Computer Organization and Design, Playlist Grande](#)
- [Tudo sobre Flip-Flop, Introdução Latches, Circuitos Sequencias, Contador Sincrono, Contadores](#)
- [Moore vs Mealy e Playlist, FSM Explained, another Vs FSM](#)
- [Playlist completo de Eletronica Digital, Yosys \(lista Open software\) e build](#)
- [VHDL Overview e Playlist PT, Harsh Truth about FPGAs, Yosys, IceStorm and Apio toolchain, Yosys Lecture](#)
- [Icarus Verilog, Mastering Verilog in one hour, SystemVerilog in 90 minutes, Synthesis in VLSI, Yosys Logic Locking](#)
- [Tiny Tapeout, Computers Suck at Division, Processador Neander, Apresentacao Neader, Arquitetura de Computadores](#)
- [The Ridiculous Engineering Of The World's Most Important Machine, Playlist Formal Verification, Formal Methods](#)
- [From top to Transistors: opensource Verilog to ASIC flow, Entrevista Estudantes TinyTapeout](#)